

React + PHP: The Thin Stack

The miniCMS protocol for modern web apps

Simone Pizzi

Third Edition

RUNTIME EDIZIONI

React + PHP: The Thin Stack
The miniCMS protocol for modern web apps
Third Edition — June 2026

© 2026 Simone Pizzi
© 2026 Runtime Edizioni

All rights reserved. No part of this book may be reproduced without the written consent of the publisher, except for brief quotations in reviews.

The trademarks and product names cited are the property of their respective owners.
www.runtimeradio.com

For Valerio Galano: because he's like Neo, he sees worlds in code, and he taught me to think in a different way. I wonder whether I taught him a little something in return.

For Giuseppe Pugliese who, even though he sees the world in his own particular way, takes pride in being a web developer and keeps at his craft with passion.

Contents

The Vision	9
CHAPTER 1: Manifesto	10
The Architecture	15
CHAPTER 2: Architecture & Project Structure	16
CHAPTER 3: Database Strategy	21
CHAPTER 4: Frontend Dependencies	26
The Components	33
CHAPTER 5: Backend Logic (PHP)	34
CHAPTER 6: Frontend Bridge (API.ts)	39
CHAPTER 7: Media & Optimization	47
CHAPTER 8: Advanced Content Editing	55
The Operational Flow	65
CHAPTER 9: Content Lifecycle	66
CHAPTER 10: Security & Auth	71
CHAPTER 11: SEO Pre-rendering with a PHP Entry Point	85
CHAPTER 12: RSS Feed & Syndication	94
CHAPTER 13: Newsletter & Email System	102
CHAPTER 14: Admin Dashboard & Panels	110
The Real-World Cases	117
CHAPTER 15: Database Evolution — From SQLite to MySQL	118
CHAPTER 16: Portfolio & Projects Module	128
CHAPTER 17: Festival Logic — Submissions & Approval Workflow	132
CHAPTER 18: Festival Logic — Voting & Anti-Fraud Protection	135
CHAPTER 19: Festival Logic — Admin Dashboard, Settings & Reporting	139
CHAPTER 20: Social Interactions & Reactions	142
Appendices	149
APPENDIX A — Boilerplate Checklist: Starting a New miniCMS Project	150
APPENDIX B: The Life of a Fork	154
APPENDIX C: Testing & Deployment (an Outline)	158

PART I

The Vision

The why. The philosophy that guides every technical decision.

CHAPTER 1: Manifesto

Why This Protocol Exists

There's an unresolved tension at the center of modern web development.

On one side, the frontend has reached an extraordinary maturity, both aesthetic and functional: React, TypeScript, Tailwind. Fluid animations, reusable components, type safety, hot reload. The development experience has become a pleasure, and the finished product, when it's done well, is visually and functionally better than anything that came before.

On the other side, that revolution brought with it an infrastructural complexity wildly out of proportion to what most sites actually need. Node.js, cloud databases, containers, CI/CD pipelines, microservices, headless CMSs on subscription plans. The technical overhead and the operating cost have become the norm even for sites that would run perfectly well on five-euro-a-month shared hosting.

This protocol grows out of a specific question: **can you have the aesthetic and technical power of React without giving up the simplicity, the control, and the affordability of a PHP backend with SQLite?**

The answer, built on months of real work on real projects, is yes.

The Founding Principle: The Separation of Layers

The Universal Model isn't a technology. It's a mental architecture.

It draws a sharp line between two layers that often get confused.

The Presentation Layer belongs to React. It's the place of form, of interaction, of animation, of typography, of the color palette. It's where aesthetic talent lives, where Tailwind turns visual intent into precise CSS, where framer-motion gives weight and breath to its movements. This layer is compiled, optimized, served as a static asset.

The Data Layer belongs to PHP and SQLite (or MySQL when it's needed). It's the place of persistence, of business logic, of security, of the content lifecycle. It isn't "the backend" in the heavy sense of the word: no framework, no ORM, no external dependencies. Just native PHP, PDO, and a file-based database that needs no server configuration.

These two layers talk through a precise contract: the REST API. The frontend knows nothing about the database. The backend knows nothing about React. Their separation is the source of all the scalability and maintainability in the system.

*
**

A Scale, Not an Absence

“Thin stack” doesn’t mean “no backend.” It means a backend pared to the bone but real, and above all it means a scale. The same skeleton—native PHP, one PDO singleton per request, one file per endpoint, no framework—plays out at different rungs depending on what you need. At the base rung sits SQLite, a database that is a single file, with no server to configure and no secret to guard. One rung up is essential MySQL, when the data or the traffic calls for it. Higher still is engineered MySQL, with hardened connections, shared preludes, and dedicated scaffolding. These aren’t three different architectures: they’re the same model at three heights.

This book is built on four real sites that sit at different points on that scale, and it reads them exactly that way. When a chapter shows “three ways to do the same thing,” it isn’t listing options at random: it’s measuring how much you can strip away, or add, to the same skeleton before it changes nature. The three-rung scale, from the base rung to the engineered top, is the lens for the whole book.

*
**

What This Protocol Is Not

It’s not a framework. It imposes no rigid code structures, requires no specific dependencies, constrains no stylistic choices.

It’s not a traditional CMS. There’s no visual page builder, no prepackaged themes, no plugin marketplace. Every site built with this protocol is unique, handmade, tailored to its purpose.

It’s not an enterprise solution. It isn’t designed to handle millions of simultaneous users, complex data flows, or distributed architectures. It’s designed for sites that need to be excellent, fast, secure, and maintainable by a small team—or even by a single person.

It’s not for someone who wants a site in ten minutes. It’s for someone who wants to understand what they’re building.

*
**

When NOT to Use This Protocol

An honest manifesto must also define its limits. The thin stack trades infrastructural complexity for the discipline of whoever writes it: it removes the framework and puts your attention in its place. When that trade-off isn’t worth it, it’s time to choose another stack. Here are the cases where I’d reach for something else without hesitating.

When the team is large. The conventions here aren’t enforced by a framework; they’re held together by people. With one or two developers it works; past four or five, the

absence of a rigid structure becomes a cost rather than a freedom, and an opinionated framework (Laravel, Next.js with its rules) repays the learning curve.

When you need real time. Live chat, instant push notifications, presence, simultaneous collaboration on a single document: these are workloads that want WebSockets and persistent processes, not the request-response model of PHP on shared hosting. You can force them, but it's an uphill fight.

When the scale is genuinely high. Tens of thousands of requests per second, unpredictable spikes, the need to scale horizontally across many nodes: this calls for queues, distributed caches, and managed databases. Chapter 3 puts hard numbers to this, so the threshold doesn't stay an opinion.

When the data is complex and heavily relational. Distributed transactions, heavy analytical reporting, models with dozens of interconnected entities: at a certain point an ORM and an engineered RDBMS aren't overhead, they're the right tool.

When compliance is an explicit requirement. Formal audits, certifications, regulated environments that demand frameworks and libraries with commercial support and a documented chain of responsibility: here, disciplined do-it-yourself is a risk not worth taking.

The rule that ties these cases together is simple: the thin stack is excellent as long as the complexity of the problem stays below the complexity a framework would impose. When it goes past that, the framework stops being a weight and becomes a safety net. Recognizing that crossover point is part of the same technical honesty that runs through the rest of the book.

✧
✧ ✧

The Values That Guide Every Decision

Total control. Whoever builds a site with this protocol owns every line of their stack. No vendor lock-in, no forced update that breaks production, no dependency on an external service for the site's survival.

Lightness as a principle, not a compromise. SQLite isn't the "cheap" choice next to MySQL: it's the right choice for 90% of use cases. Simplicity isn't a limitation to overcome, it's a goal to reach and defend.

Security as architecture, not as a patch. The security decisions are built into the design of the system: the database outside the public root, a build script that never leaves the database in the deploy, PHP sessions with HttpOnly cookies, passwords never in cleartext. These aren't measures bolted on afterward, they're the structure itself.

More engineered doesn't mean more secure. It's the most uncomfortable lesson these sites teach, and it comes back in almost every chapter. The technically richest site, with the most hardened connections and the most carefully tended infrastructure, is often the one with the most fragile fundamentals: a default password written into the code, no automatic backup, an anti-abuse barrier missing exactly where it's needed. Adding complexity doesn't pay security interest on its own, and sometimes it distracts from

the two lines that actually matter. Distrusting the equation “more engineering equals more security” is one of the threads that hold this book together.

Documentation as part of the code. A system you can’t understand is a system you can’t maintain. This protocol is documented with the same care it’s built with, because knowledge has to stay accessible even when the context changes.

Real experience as the only validator. Every pattern documented in this book was pulled from code running in production. The most important lessons come from real incidents, not hypothetical scenarios: the nighttime WAL crash that forced Runtime Radio into an emergency migration to MySQL, the wave of bots that swamped its entry point. Theory without the scar doesn’t teach enough.

*
**

Who It’s For

For anyone who wants to build a website that’s a living thing: not a template, not a customized WordPress, not a site spat out by a builder.

For the developer who knows React and wants a backend without having to learn a whole framework.

For the freelancer who has to deliver a fast, maintainable, secure site to a client with no budget for cloud infrastructure.

For the author, the musician, the festival, the radio station that wants a digital presence of its own: controlled, independent of the whims of the platforms.

For anyone who believes the web can still be a place made by people, for people, with no middlemen.

*
**

Two Voices: “In the Wild” and “The Canon”

This book does something most technical texts avoid: it shows real code as it is, not as it should be. That’s its strength, but it can confuse, because two different things live on the same page, and they have to be kept apart.

The first voice is “**in the wild**”: the snapshot of what the four sites actually do. Here are the good choices and also the scars, the anti-patterns, the holes left open. When the text recounts that a site saves raw HTML without sanitizing it, or exposes a migration script, it isn’t recommending that: it’s documenting it. This is the body of every chapter, and it’s written without flinching precisely because the scar teaches more than theory.

The second voice is “**the Canon**”: the rule to follow, distilled. Every chapter closes with a box titled **The Canon**, which cleanly separates the prescription from the snapshot. There you’ll find what to do in a new project, cleaned of the imperfections of the real cases. If you have ten seconds and just want the rule, read that box; if you want to understand *why* the rule is what it is, read the chapter that comes before it.

NOTE

The reading rule, in one line The body of the chapter says what the code *does* (“in the wild”); the box at the end says what you *should* do (“the Canon”). When the two diverge, the Canon is always right: the divergence is the point, not a typo.

*
**

How to Use This Book

The book is organized into independent thematic chapters. You don’t have to read it from start to finish: every chapter is a self-contained reference.

To start a new project from scratch, the **Boilerplate Checklist** (Appendix A) is the practical place to begin.

To improve an existing project, the specific chapters (Database Strategy, Security & Auth, SEO Pre-rendering) offer patterns you can apply surgically.

To learn from history, the chapters with the experiential voice (the WAL crash, the bot attack on Runtime Radio, the migration to MySQL) are the most honest reading this book has to offer.

Code doesn’t lie. Neither do scars.

*
**

IMPORTANT The Canon

- Separate the two layers: compiled React for presentation, native PHP with PDO for data, a REST contract between them.
- Pick the right rung of the scale (SQLite base rung → essential MySQL → engineered MySQL), never more than you need.
- Treat security as architecture, not as a patch, and distrust the equation “more engineering equals more security.”
- Document the code as it is, not as it looks: the scar teaches more than theory.
- Use the thin stack as long as the problem’s complexity stays below what a framework would impose; past that point, choose the framework.

*
**

“Perfection is achieved, not when there is nothing more to add, but when there is nothing left to take away.” Antoine de Saint-Exupéry

*
**

Next Chapter: Architecture & Project Structure. Where ideas become folders.

PART II

The Architecture

The foundations. Project structure, database, technology stack.

CHAPTER 2: Architecture & Project Structure

This chapter defines the physical and logical architecture of the system: how the folders are laid out, where the secrets live, how a request finds its PHP file, and which defenses are wired into the structure itself rather than bolted on afterward.

1. Folder Topology and Asset Separation

The Universal Model enforces a clean separation between source files (build-time) and runtime files (the data that persists).

1.1 Physical Structure (Root)

```
/
├── public/                # Public content and API entry point
│   ├── .htaccess         # SPA routing (crucial for React Router)
│   ├── index.php         # SEO Engine (Chapter 11) – PHP entry point
│   └── api/              # Core PHP backend (see 1.2)
├── src/                  # React 19 / TS frontend
├── scripts/              # Utilities (build, clean, migration)
├── .env.local            # Local configuration (VITE_API_URL)
├── package.json          # Dependencies and automation scripts
└── clean-dist.js         # Post-build sanitization (SECURITY)
```

1.2 Anatomy of the API Area (`public/api/`)

The first thing to understand about this folder is what it *doesn't* contain: a router. There's no dispatching `index.php`, no kernel, no route table. Every endpoint is a **standalone PHP file**: `articles.php`, `upload.php`, `reactions.php`. Each one pulls in its building blocks at startup (the database connection, the session prelude if there is one, the headers) and handles its own request. This is the founding choice of the miniCMS, and the one that makes the “thin stack” literally thin: the URL `/api/articles.php` is the file `articles.php`, with nothing in between. The price is some repetition across files (the same `require` at the top of each); the payoff is that every endpoint reads, tests, and moves on its own, without forcing you to understand a routing system.

Alongside the endpoints live the runtime folders, which change depending on the database engine you choose:

- **.data/**: holds the **SQLite** database, so it exists only on the sites that use SQLite (like DISINTELLIGENZA). It must contain an `.htaccess` with `Deny from all`, because a `.sqlite` file reachable over the web is the entire database, downloadable by anyone. Sites

migrated to MySQL don't have this folder: their database lives on the MySQL server, outside the docroot by nature.

- **.cache/**: JSON files generated for performance (the content lists, see Chapter 9), invalidated on every write.
- **uploads/**: the assets uploaded by users (images, audio). It should be excluded from source-code backups, and on the better-defended sites it hosts its own `.htaccess` that **turns PHP off** (the first anti-RCE barrier, Chapter 7).

1.3 Configuration and Secrets: Three Approaches Without Libraries

Secrets never sit in versioned source. But the way to keep them out is a small scale of its own, and the three sites occupy it at three different points, all without a configuration library.

```
// SPW config.php (gitignored) – paired with a versioned config.example.php
define('DB_HOST', 'localhost');
define('DB_NAME', '...');
define('SITE_URL', 'https://...');    // canonical constant, anti host-pois
$oning
```

```
; SR .env (gitignored) – read by db_credentials.php via parse_ini_file(); al
$ onside it, .env.example
DB_HOST=localhost
TELEGRAM_BOT_TOKEN=...
SMTP_HOST=...    ; the .env is the hub for ALL the site's secrets
```

SimonePizziWebSite uses a `config.php` with `define()` calls, ignored by git and accompanied by a `config.example.php` committed as a reference. SitoRuntime uses a `.env` file read with `parse_ini_file()`, and it's the hub for all its secrets (database, Telegram token, SMTP credentials). DISINTELLIGENZA has **no** configuration at all: the path to the SQLite file is written directly into `db.php`, because a file-based database has no credentials to guard. It's the “twelve-factor” principle (configuration separated from code) applied without any library, with one radical exception: whoever sits at the base rung of the scale has no secret to manage in the first place.

**

2. Security at Build Time: The Clean-Dist Logic

One of the biggest risks is overwriting the production database during deployment. The system prevents this with a script (`clean-dist.js`) run after the build, which:

1. scans the `dist/api/` folder;
2. recursively removes every file with a `.sqlite`, `.sqlite3`, `.db`, Or `.bak` extension;
3. warns the operator with a security log (□ SECURITY: Removed...);
4. guarantees, as a **design rule**, that the `dist/` is “database-free.” The database is initialized on the server or migrated by hand, never overwritten by the automatic build.

The exact pattern varies by site:

- **SimonePizziWebSite**: `"postbuild": "node clean-dist.js"` (automatic npm hook);

- **DISINTELLIGENZA:** "build": "tsc -b && vite build && node clean-dist.js && move dist\\index.html dist\\index_react.html" (renames index.html to enable the PHP SEO Engine, Chapter 11);
- **SitoRuntime:** "build": "tsc -b && vite build && node scripts/remove-db-from-dist.js" (dedicated script).

WARNING

The build can remove defenses too, not just databases This script has a side effect that comes back in Chapter 14: by removing the `.data/` folder from the distribution, it also carries away the `Deny` from all `.htaccess` committed inside it. The static defense never reaches the server, and has to be recreated at runtime. The lesson holds for any build pipeline: always ask what the build *removes*, not just what it adds.

**

3. URL Routing: SPA vs. API

For React Router to coexist with the PHP APIs on Apache, the standard calls for an `.htaccess` in the public root:

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /
  # If the request points to a real file or directory, serve it directly (so
  ↳ the APIs work)
  RewriteCond %{REQUEST_FILENAME} -f [OR]
  RewriteCond %{REQUEST_FILENAME} -d
  RewriteRule ^ - [L]
  # Otherwise route everything to index.html (or index.php if present)
  RewriteRule ^ index.html [L]
</IfModule>
```

Apache serves `index.php` before `index.html` by default priority. This is the mechanism that lets the SEO Engine (Chapter 11) intercept bot requests without touching the rewrite rules.

**

4. Dynamic File-System Initialization

The PHP backend doesn't take the runtime folders for granted: it creates them when they're needed.

```
// Auto-scaffolding: create the runtime folders, and immediately protect the
↳ DB one
$paths = [__DIR__ . '/.data', __DIR__ . '/.cache', __DIR__ . '/uploads'];
foreach ($paths as $path) {
  if (!is_dir($path)) {
    mkdir($path, 0755, true);
    if (basename($path) === '.data') {                                // only
↳ the file-based DB
      file_put_contents($path . '/.htaccess', "Order allow,deny\nDeny
↳ from all");
    }
  }
}
```

The site that runs on SQLite (DISINTELLIGENZA) generates the `.data/` folder and its `Deny` from all `.htaccess` **at runtime**, inside the connection code: the protection is produced by the application, not pre-deployed, with a `<Files>` block in the global `.htaccess` as a second safety net. The MySQL sites have no file-based database to hide, but they apply the same principle (create at runtime what the deploy doesn't bring) to `cache`, `uploads`, and the `backups` folder (Chapter 14). The shared rule: don't trust that a folder exists, and don't trust that a static defense makes it all the way to the server.

*
**

5. Environment Management

- **Development:** the frontend runs on the Vite dev server, which proxies to the local PHP to avoid CORS errors. `SimonePizziWebSite`, which in development goes through a different port (`localhost:8888`), is the only one that has to handle this on the client side too (Chapter 6).
- **Production:** the frontend points to `/api`, a relative same-origin path, so cookie authentication works without CORS and without domain configuration.

*
**

6. The Fork Pattern (FDCA from DISINTELLIGENZA)

FDCA and DISINTELLIGENZA share an **identical** PHP structure: same files, same logic, because FDCA was born from a fork of the more mature project. It's a deliberate choice: when two projects have the same functional base (a festival with voting), you start from a copy of what already works.

The advantage is independence: no shared dependency, each project evolves on its own. The risk is the exact flip side of that advantage: every bugfix and every improvement has to be reapplied by hand on both branches, and when the fix concerns security, forgetting it means leaving a hole open in one site while you close it in the other. That's exactly what happened here (the upload RCE chain from Chapter 7 was mapped on DISINTELLIGENZA but lives unchanged in FDCA), and the reason the life of a fork deserves an appendix of its own.

IMPORTANT The Canon

- One file per endpoint, no central router: every PHP endpoint pulls in its building blocks and stands on its own.
- Keep secrets out of versioned code (`config.php/.env` in `.gitignore`, with a committed `.example`).
- Keep the file-based database and sensitive folders outside the docroot, or protected by a `deny .htaccess` generated at runtime: the build can remove static defenses.

- Separate the `dist/` from the sources and have `clean-dist` strip the `.sqlite` files from the distribution.

**

Next Chapter: Database Strategy. Locks, indexes, migrations, and the real story of the WAL.

CHAPTER 3: Database Strategy

The database is the heart of the miniCMS. This chapter defines the strategies for connection, integrity, and migration, drawing a careful line between two things that are easy to confuse: what the Model **recommends** and what the real sites **actually do**. The two don't always line up, and saying so out loud is more useful than passing off a prescription as a snapshot.

1. Connection Architecture (PDO, One Engine at a Time)

The system uses PDO with a memoized singleton (the connection opens only once per request, Chapter 5). The engine underneath, though, isn't abstracted away: SQLite and MySQL open the connection with different options, and the three sites pick those options at three different levels.

1.1 The Connection Options: The Snapshot

Before the prescription, here's what happens in the real code. The three sites set PDO option sets of escalating paranoia, and it's a good example of the three-rung scale from Chapter 1.

Site	Engine	Real PDO options
DISINTELLIGENZA	SQLite (live)	only ERRMODE=EXCEPTION + DEFAULT_FETCH_MODE=ASSOC, via setAttribute(). No PRAGMA in connect().
SimonePizziWebSite	MySQL	ERRMODE + FETCH + EMULATE_PREPARES=false (real prepared statements)
SitoRuntime	MySQL	ERRMODE + FETCH + ATTR_TIMEOUT=5 + MYSQL_ATTR_INIT_COMMAND (SET NAMES)

The most surprising metric is the first row: the only site **actually** running on SQLite today (DISINTELLIGENZA) sets none of the “optimal” PRAGMA directives that most manuals take for granted. It opens the file and that's it, with two options. It's a minimal choice that works for its load, but it isn't the most robust setup possible, and that's why the prescription below should be taken for what it is: a piece of advice, not a description of the existing code.

1.2 The Connection Options: The Prescription

For an SQLite deployment on shared hosting, the Model recommends three PRAGMA directives on top of the base options:

```
// RECOMMENDED for SQLite on shared hosting (not what DIS sets today)
self::$pdo = new PDO("sqlite:" . $dbPath);
```

```
self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
self::$pdo->exec("PRAGMA journal_mode = DELETE;"); // more stable than WAL
↳ on shared hosting
self::$pdo->exec("PRAGMA busy_timeout = 5000;"); // waits instead of fai
↳ ling under concurrent writes
self::$pdo->exec("PRAGMA foreign_keys = ON;"); // referential integrit
↳ y (off by default in SQLite)
```

The 5000ms `busy_timeout` makes a concurrent write wait instead of failing right away (handy when the newsletter and the admin panel write at the same time). The `foreign_keys = ON` is needed because SQLite, unlike MySQL, leaves foreign keys disabled by default. But the line that carries a scar is the first one.

WARNING

Why DELETE and not WAL: the lesson comes from an incident SitoRuntime, back when it still ran on SQLite, tried `journal_mode = WAL` in production to improve performance under load. On shared Apache hosting, WAL did damage: the `.sqlite-wal` lock file would hang around and corrupt reads. It took an emergency script (`emergency_revert_wal.php`) to get back to `DELETE`, and not long after, that disaster pushed the migration to MySQL (the full story is in Chapter 15). The lesson, for anyone staying on SQLite: on shared hosting, choose `DELETE`, not `WAL`. It's a recommendation born from a real failure, not a matter of taste.

1.3 Hiding the File-Based Database (SQLite Only)

SQLite users face a problem that MySQL users don't: the database is a file inside the site's folders, and a file reachable over the web is the entire database, downloadable by anyone. In **DISINTELLIGENZA**, the defense is generated at runtime by the connection code: on the first `connect()`, if the `.data/` folder doesn't exist, it gets created, and a `Deny` from all `.htaccess` is written inside it.

```
// DISINTELLIGENZA db.php - the file-based DB's protection is created by the
↳ app, not pre-deployed
$dir = dirname($dbPath); // ../.data
if (!is_dir($dir)) mkdir($dir, 0755, true);
$htaccessPath = $dir . '/.htaccess';
if (!file_exists($htaccessPath)) {
    file_put_contents($htaccessPath, "Require all denied\n"); // second sa
↳ fety net: <Files> in the global .htaccess
}
```

This pattern belongs to the SQLite sites, not the MySQL ones. SimonePizziWebSite and SitoRuntime have migrated to MySQL: they have no `.data/` folder with the database inside, because their data lives on the MySQL server, outside the docroot by nature. Generating the protection at runtime, rather than trusting it to a committed file, has a concrete reason that comes back in Chapter 14: the build script can strip the `.data/` folder out of the distribution, and with it the `deny .htaccess`, which would otherwise never reach the server.

2. Strategic Indexing

Read queries have to be instant. The Model calls for these indexes:

Table	Column	Type	Purpose
news / articles	slug	UNIQUE	Article lookup by URL (SEO)
news / articles	published_at	DESC	Fast chronological ordering
news / articles	status	INDEX	Draft/published filtering
speakers	sort_order	ASC	Manual drag-and-drop ordering
podcasts	slug	UNIQUE	Quick access to series
projects	category	INDEX	Portfolio category filtering
projects	sort_order	ASC	Manual portfolio ordering

After bulk loads, it helps to run `ANALYZE`; to recompute the access statistics and optimize the query plan.

*

**

3. The Migration Lifecycle

Schema changes have to be atomic, idempotent, and protected. Those are the requirements; the real sites meet them unevenly, and the difference matters.

- **Atomicity:** every migration uses a transaction (`beginTransaction`).
- **Idempotency:** the script checks whether a column or table exists before creating it (`IF NOT EXISTS, PRAGMA table_info`), so rerunning it breaks nothing.
- **Protection:** migration scripts **should** be unreachable by unauthenticated web requests. Here reality diverges: `SitoRuntime` denies them by prefix in the `.htaccess` or keeps them inside `admin.php` (gated), while `DISINTELLIGENZA` leaves its `update_db_*.php` and `migrate_media.php` reachable over HTTP without a gate. That's a real security debt (Chapter 15), not a detail.
- **Naming:** no site has a `schema_version` table. The version lives in the **file names** (`update_db_v0.4.2.php`, `update_db_v0.5.4.php`), aligned with the version in `package.json`. The migration history is read from the folder, not from a record in the database.

*

**

4. Data Normalization

To avoid inconsistencies between PHP, JS, and the database, the Model normalizes before saving:

- **Dates:** `Y-m-d H:i:s` format, for SQL and JS compatibility. Mind the time zone: the `published_at <= NOW` visibility check is done on strings, and a date written with the wrong separator shifts the threshold (Chapter 5 and Chapter 9).
- **Numbers:** integers, or rounded to zero decimals, to avoid floating-point bugs.
- **Booleans:** `INTEGER (0 or 1)` in SQLite, `TINYINT(1)` in MySQL.

*

**

5. Maintenance and Integrity

- **VACUUM**: run it after bulk deletions to recompact the file and shrink it (relevant on SQLite).
- **Backups**: every migration **should** be preceded by a copy of the database in a protected folder. Here too, reality isn't uniform: SimonePizziWebSite has an automatic backup outside the docroot, DISINTELLIGENZA makes a .bak copy before destructive actions, and SitoRuntime (the site that suffered the crash) has no automatic backup at all. The “cure without prevention” paradox is in Chapter 14.
- **optimize_db.php**: SitoRuntime includes a maintenance script (VACUUM, ANALYZE, integrity check); despite its “throwaway” header, it's actually non-destructive (it only adds idempotent indexes).

*
**

6. When to Move to MySQL

The full story, with the real scripts, is in Chapter 15. One point needs clearing up right away, because it's a widespread half-truth: SitoRuntime's migration was **not** decided by a traffic threshold reached at a calm pace. It was the reaction to an incident (the WAL crash that night) that made SQLite suddenly unreliable on that hosting. The threshold wasn't a number on a chart, it was a corrupted database at 3 a.m.

The counterpoint is just as instructive: DISINTELLIGENZA, a festival with public voting, **still runs on SQLite** in production today, no trouble. SQLite isn't a rung to abandon the moment you can: it's the right choice as long as it holds, and “as long as it holds” depends on the load and the hosting, not on a universal rule. You move to MySQL when a concrete constraint forces it, not out of superstition.

6.1 The Numbers, Honestly

“As long as it holds” isn't an answer you can leave to gut feeling. You need orders of magnitude, with the caveat that they stay that: they depend on the hardware, on the shared host's disk, and on the shape of the queries, and they have to be measured against your own case, not taken as guarantees.

The key point is that in SQLite, **reads and writes don't scale the same way**. Reads are concurrent and very fast: a read-heavy site (a blog, a portfolio, a radio station with news and podcasts) handles thousands of reads per second from the page cache and hundreds of read queries per second straight to the file without breaking a sweat, because multiple requests can read the same database at once. Writes, though, are **serialized**: SQLite allows one writer at a time and locks the whole file during a write. That's where the ceiling is.

In practice, on typical shared hosting:

Signal	SQLite is comfortable	Worth weighing MySQL
Concurrent writes	up to a few dozen per minute, sporadic	sustained dozens per second, or regular concurrent spikes
Load profile	read-dominant (90%+), isolated writes	frequent, simultaneous writes (forms, votes, queues)
Write concurrency	one process at a time is enough	multiple processes writing together (newsletter + admin + public)
Topology	a single application server	you need to scale across multiple nodes sharing the data
Symptom in the logs	no database is locked	recurring busy timeout or database is locked

The real threshold isn't a number but a **symptom**: when the logs start showing lock errors or busy timeout despite the configured busy_timeout, the database is telling you that write contention has outgrown what a file-based database can handle on that hosting. That's the moment for MySQL, and not an instant sooner. Runtime Radio got there through an incident (Chapter 15); most sites never get there, and that's exactly right.

IMPORTANT The Canon

- SQLite with journal_mode=DELETE (never WAL on shared hosting), busy_timeout, and foreign_keys=ON; PDO in ERRMODE_EXCEPTION.
- Index slug (UNIQUE), published_at (DESC), and status; run ANALYZE after bulk loads.
- Migrations atomic, idempotent, and unreachable by unauthenticated web requests; keep a schema_version table (the real sites don't have one: it's a debt not to inherit).
- Back up before every migration, in a protected folder outside the docroot.
- Move to MySQL when the logs show recurring database is locked/busy timeout, not out of superstition.

*
**

Next Chapter: Frontend Dependencies. The dependency matrix, the rules for choosing, and the cost of every library.

CHAPTER 4: Frontend Dependencies

Frontend dependencies are architectural choices, not just lines in `package.json`. Every library you add has a cost (bundle size, update surface, integration complexity) and has to earn its place. This chapter documents the choices made across the four reference sites, with the reasoning behind them, and it gives equal space to an absence: the libraries the Model chooses **not** to use.

1. The Core Stack (Every Project)

These dependencies are present in all four sites, no exceptions:

Library	Version	Role
react + react-dom	^19.2.x	UI framework: components, state, rendering
react-router-dom	^7.x	Client-side routing, React Router v7 (stable API)
typescript	~5.x	Compile-time type safety
vite	^7.x	Build tool: HMR dev server, optimized bundle
@vitejs/plugin-react	^5.x	Vite plugin for the JSX/TSX transform
lucide-react	^0.5xx	Tree-shakeable SVG icons, native to React

React 19 brings improvements to concurrent rendering and Server Components. For the Universal Model (a client-side SPA only), the practical difference from v18 is small, but staying on the latest stable version is the right call for new projects.

1.1 The Absence That Counts: No Data-Fetching Libraries

Before listing what's there, it's worth listing what's missing, because that's as much a choice as the rest. None of the four sites use a data-fetching or state-management library: no Axios, no React Query, no SWR, no Redux, no Zustand. The bridge to the PHP backend is the `api` object, a thin wrapper over native `fetch` (Chapter 6); shared state lives in the router (SimonePizziWebSite, with React Router's loaders) or is component-local `useState` (SitoRuntime and DISINTELLIGENZA).

It isn't laziness, it's consistency with the philosophy of Chapter 1: every dependency you don't add is an update surface you don't have to maintain, a bundle that doesn't grow, a behavior you don't have to learn. `fetch` does almost everything a CMS needs; the little it lacks (a single place for errors, an interceptor for an expired session) takes a few lines to write, and Chapter 6 tells the story of where the sites chose not to write them.

2. Tailwind CSS: v3 vs. v4

The four sites use different versions of Tailwind, with noticeably different configuration.

SitoRuntime: Tailwind v3 (classic setup)

```
// devDependencies
"tailwindcss": "^3.4.17",
"autoprefixer": "^10.4.22",
"postcss": "^8.5.6"
```

It requires explicit `tailwind.config.js` and `postcss.config.js` files. The entry CSS:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

SimonePizziWebSite, DISINTELLIGENZA, FDCA: Tailwind v4

```
// devDependencies
"tailwindcss": "^4.x",
"@tailwindcss/vite": "^4.x" // or "@tailwindcss/postcss"
```

Tailwind v4 uses the native Vite plugin: no separate `postcss.config.js`, no mandatory `tailwind.config.js`. The CSS:

```
@import "tailwindcss";
```

For new projects, the choice is v4. Migrating from v3 takes some care with renamed classes (for example, `shadow-sm` became `shadow-xs`).

@tailwindcss/typography

Present in every site as a devDependency. It adds the `prose` class, which applies polished typographic styles to blocks of dynamically generated HTML (the output of a rich-text editor, or of a Markdown converter).

```
<div class="prose prose-invert max-w-none">
  <!-- HTML generated by Tiptap or showdown -->
</div>
```

3. Content Editing and Rendering

Tiptap v3: the flagship sites' rich-text editor

Used in: SimonePizziWebSite and SitoRuntime (the two sites with an editorial workflow).

The rich-text editor on the two flagship sites is **Tiptap v3**, not a monolithic editor but a set of `@tiptap/*` packages that compose like building blocks: about a dozen modules for the core and the extensions actually in use.

```
"@tiptap/react": "^3.x",
"@tiptap/pm": "^3.x",
"@tiptap/starter-kit": "^3.x",
"@tiptap/extension-image": "^3.x",
```

```
"@tiptap/extension-text-style": "^3.x",
"@tiptap/extension-color": "^3.x",
"@tiptap/extension-text-align": "^3.x",
"@tiptap/extension-table": "^3.x",
"@tiptap/extension-youtube": "^3.x"
```

Tiptap produces **HTML** (not Markdown), which is saved raw in the database and sanitized at render time with DOMPurify (the full treatment, including the link-insertion guards and the sanitization choke point, is in Chapter 8). SitoRuntime got there through a migration: it used Quill before, and the code still carries the marks of that switch (a conversion shim, a few leftover `react-quill` type defs). Those leftovers aren't the active editor, they're scars from the migration to Tiptap.

DISINTELLIGENZA uses no editing library at all: its editor is hand-built with `contentEditable` and `execCommand`. It's the base rung of the scale, with the security consequence Chapter 8 spells out (it's the only site without DOMPurify).

showdown: Markdown → HTML

Used in: SimonePizziWebSite, SitoRuntime, DISINTELLIGENZA, for content written in Markdown.

```
"showdown": "^2.1.0",
"@types/showdown": "^2.0.6"
```

```
import showdown from 'showdown';
const converter = new showdown.Converter({ tables: true, strikethrough: true
  ↪ });
const html = converter.makeHtml(markdownContent);
```

HTML generated from untrusted content always has to be sanitized (see below): showdown and DOMPurify travel as a pair.

dompurify: XSS sanitization

Used in: SimonePizziWebSite, SitoRuntime (always paired with the editor or the Markdown converter). **Absent in DISINTELLIGENZA**, and that's the security hole Chapter 8 documents.

```
"dompurify": "^3.3.x",
"@types/dompurify": "^3.0.5"
```

```
import DOMPurify from 'dompurify';
const safeHtml = DOMPurify.sanitize(rawHtml);
// Only now is it safe to use dangerouslySetInnerHTML
<div dangerouslySetInnerHTML={{ __html: safeHtml }} />
```

WARNING

Raw HTML plus `dangerouslySetInnerHTML` without DOMPurify is an XSS vulnerability It holds for the output of showdown, of Tiptap, of any source. Saving raw HTML is a legitimate choice (it preserves formatting), but render time is where sanitization becomes mandatory. DISINTELLIGENZA skips this step, and Chapter 8 shows what that means.

4. Animations and Visual Effects

framer-motion

Used in: all four sites.

```
"framer-motion": "^12.x"
```

```
import { motion } from 'framer-motion';
<motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} t
  ↪ransition={{ duration: 0.4 }}>
  {/* content */}
</motion.div>
```

framer-motion is heavy (about 100KB gzipped). It's justified when the animations are part of the site's visual identity; where they're marginal, native CSS transitions are the right choice.

typewriter-effect

Used in: SimonePizziWebSite. A typewriter effect for animated headlines, specific to a personal portfolio.

tailwindcss-animate

Used in: DISINTELLIGENZA. A Tailwind plugin with classes like `animate-in`, `fade-in`, and `slide-in-from-bottom`: a lightweight alternative to framer-motion for simple CSS-based animations.

5. SEO and Meta Tags

react-helmet-async

Used in: SimonePizziWebSite, DISINTELLIGENZA. It manages `<title>`, `<meta>`, and `<link>` from the React side, for users with JavaScript enabled.

```
"react-helmet-async": "^2.0.5"
```

The original version (`react-helmet`) is no longer maintained and has memory-leak problems with React 18+; `react-helmet-async` is the active fork, compatible with React 19.

Social and search crawlers don't run JavaScript, so the meta tags that matter to them are produced by the **PHP SEO Engine** (Chapter 11). The two complement each other, they aren't alternatives: SitoRuntime, in fact, drops `react-helmet-async` entirely and delegates the whole of SEO to the server-side engine.

6. Utilities

clsx + tailwind-merge

Used in: SimonePizziWebSite, DISINTELLIGENZA.

```
"clsx": "^2.1.1",
"tailwind-merge": "^3.x"
```

`clsx` composes `className` conditionally; `tailwind-merge` resolves conflicts between Tailwind classes (`text-sm text-lg` becomes `text-lg`). Together they give you the classic `cn()` helper.

date-fns

Used in: DISINTELLIGENZA, to format the festival dates in the frontend. The other sites handle dates on the PHP side (`date()`, `strtotime()`), keeping the client simpler.

7. Build-Time Optimization

sharp

Used in: SimonePizziWebSite, as a dependency but only in the post-build scripts.

```
"sharp": "^0.34.5"
```

`sharp` is a Node.js library for high-performance image manipulation; it's used in the build scripts to optimize static assets. It's a dependency (not a `devDependency`) because the `postbuild` needs it at build time, but it never enters the React bundle: no `.tsx` file imports it. The runtime WebP conversion of uploads, on the other hand, is all on the PHP side with GD (Chapter 7): `sharp` and GD cover two different moments.

8. Per-Site Dependency Matrix

Library	SitoRuntime	DISINTELLIGENZA	FDCA	SimonePizziWebSite
React 19	yes	yes	yes	yes
react-router-dom v7	yes	yes	yes	yes
framer-motion	yes	yes	yes	yes
showdown	yes	yes	no	yes
dompurify	yes	no	no	yes
lucide-react	yes	yes	yes	yes
Tailwind v3	yes	no	no	no
Tailwind v4	no	yes	yes	yes
@tailwindcss/typography	yes	yes	yes	yes
react-helmet-async	no	yes	yes	yes
@tiptap/* (editor)	yes	no	no	yes
clsx + tailwind-merge	no	yes	yes	yes
sharp	no	no	no	yes
date-fns	no	yes	yes	no
typewriter-effect	no	no	no	yes
tailwindcss-animate	no	yes	yes	no
<i>data-fetching libraries</i>	no	no	no	no

The last row is the most important one in the Model: no site has one.

9. Rules for New Projects

1. **Start from the minimum:** core stack, framer-motion, lucide-react. Add only when the feature genuinely calls for it.
2. **No data-fetching library:** `fetch` and an `api` object are enough (Chapter 6). Adding React Query or Axios is almost always weight that doesn't pay off in a thin-stack CMS.
3. **showdown always requires dompurify** when the content is user-generated or comes from the database.
4. **Tiptap for rich editing, a `<textarea>` for the rest:** pull in the `@tiptap/*` packages only where editors need a visual editor; for simple input, a `textarea` keeps the bundle small.
5. **react-helmet-async for client-side SEO, the PHP engine for crawlers:** they complement each other, they aren't alternatives.
6. **Tailwind v4 for new projects:** simpler configuration; if you migrate from v3, check the renamed classes.

*

**

Related chapters: Chapter 2 (project structure) for the Vite configuration; Chapter 6 (Frontend Bridge) for the `api` object over `fetch`; Chapter 8 (Advanced Content Editing) for Tiptap and sanitization; Chapter 11 (SEO) for the relationship between `react-helmet-async` and the PHP engine.

IMPORTANT The Canon

- Start from the minimum: no data-fetching library (`fetch` and an `api` object are enough), no ORM, no state manager until you genuinely need them.
- For rich editing, use Tiptap (raw HTML saved + DOMPurify at render); a `<textarea>` for everything else.
- Every dependency has a cost: add it only if it pays off, and tell build time (`sharp`) apart from runtime (`GD`).

*

**

Next Chapter: Backend Logic (PHP). Unified CRUD, buffer handling, and input sanitization.

PART III

The Components

The bricks. Backend, frontend, media, editor: the building blocks of the system.

CHAPTER 5: Backend Logic (PHP)

The miniCMS backend is the engine that processes the data and stands guard. This chapter defines what an endpoint is made of, how it starts up, and how it handles identifiers, the time zone, and errors. The through-line is always the same: no framework, one PHP file per responsibility, and three ways of shaping the same skeleton depending on how much the site has chosen to engineer.

1. Identifier Handling (Slugs)

URLs have to be readable and unique. The Model generates the slug on the server and guarantees its uniqueness against collisions.

1.1 Base algorithm

```
function createSlug($string) {
    // lowercase, trim, and strip non-alphanumeric characters (except the hy
    ↪ phen)
    return strtolower(trim(preg_replace('/[^A-Za-z0-9-]+/', '-', $string)));
}
```

1.2 Advanced algorithm (normalizing Italian accents)

The base pattern produces malformed slugs for accented words (caffè becomes caff-).

The **SimonePizziWebSite** variant solves it with an explicit map before the cleanup:

```
function generateSlug($title, $pdo) {
    $accents = ['à','è','é','ì','ò','ù','À','È','É','Ì','Ò','Ù','â','ê'
    ↪ 'î','ô','û','ä','ë','ï','ö','ü'];
    $replacements = ['a','e','e','i','o','u','a','e','e','i','o','u','a','e'
    ↪ 'i','o','u','a','e','i','o','u'];
    $title = str_replace($accents, $replacements, $title);
    $slug = strtolower(trim(preg_replace('/[^A-Za-z0-9-]+/', '-', $title)))
    ↪ ;

    // anti-collision: if the slug already exists, append a timestamp suffix
    $stmt = $pdo->prepare("SELECT COUNT(*) FROM articles WHERE slug = ?");
    $stmt->execute([$slug]);
    if ($stmt->fetchColumn() > 0) $slug .= '-' . time();
    return $slug;
}
```

For sites with Italian content, always use the advanced pattern. (The three slug philosophies across the sites, accents kept or stripped, are detailed in Chapter 9.)

2. Time Zone Handling

On international hosting (a server in Los Angeles, say), `date()` and `time()` use the server's time zone, and that breaks the `published_at <= NOW` visibility logic, which for an Italian site has to reason in Italian time.

```
// at the start of every endpoint with time logic
date_default_timezone_set('Europe/Rome');
$ita_now_str = date('Y-m-d H:i:s'); // Italian time for the SQL comparison
↳ s
```

The rule is simple, applying it isn't. SimonePizziWebSite forces the time zone in **every** endpoint; SitoRuntime and DISINTELLIGENZA do it only in some (`index.php`, `news.php`), and not elsewhere. The result is a publication threshold that shifts depending on which file evaluates it.

WARNING

Force the time zone everywhere, or it's useless Forcing the time zone in a single endpoint gives a false sense of safety: a scheduled article can read as already published to one file and still in the future to another, because the same `published_at` string is compared against a `NOW` computed in different zones. SitoRuntime carries the scar of this in a `debug_time.php` that documents an incident over the date separator (the space turning into a τ). The logic of comparing dates, and the three ways to get it wrong, is in Chapter 9; here the bootstrap rule is enough: if you force the time zone, do it in the shared prelude, not endpoint by endpoint.

3. Anatomy of an Endpoint: the Router on REQUEST_METHOD

There's no central router. Every file in `public/api/` is a standalone endpoint, and it dispatches on its own based on the request's HTTP verb. It's the pattern that makes the thin stack readable: the URL `/api/articles.php` is the file `articles.php`, and inside that file is everything that concerns it.

```
$method = $_SERVER['REQUEST_METHOD'];

if      ($method === 'GET')    { /* read: page, limit, slug, id, category, a
↳ dmin */ }
elseif ($method === 'POST')  { Auth::check(); /* create */ }
elseif ($method === 'PUT')   { Auth::check(); /* full replacement */ }
elseif ($method === 'PATCH') { Auth::check(); /* partial update: toggle is
↳ _visible, sort_order */ }
elseif ($method === 'DELETE') { Auth::check(); /* delete */ }
```

The gate isn't uniform across the file but **selective per branch**: the read `GET` stays public, and the branches that mutate state go through `Auth::check()`. In the older projects (DISINTELLIGENZA, SitoRuntime before the refactor), the mutations all traveled as `POST` with an `action` field in the body; the pattern with separate verbs is more readable and pairs better with an expressive TypeScript client.

4. The Three Bootstrap Styles

Before dispatching the request, an endpoint has to start up: open the connection, start the session, send the headers, handle the CORS preflight if there is one. Here the three sites occupy three rungs, and it's a good portrait of the scale from Chapter 1.

SimonePizziWebSite: inline prelude. Every file includes its building blocks at the top (require 'db.php', require 'auth_helper.php', the headers) and opens the connection right away. The auth_helper.php wraps session_start(), the Content-Type, and the Auth class:

```
// auth_helper.php – session, headers, and Auth in a single include
require_once 'db.php';
session_start();
header('Content-Type: application/json');

class Auth {
    public static function check() {
        if (!isset($_SESSION['user_id'])) {
            http_response_code(401);
            echo json_encode(['status' => 'error', 'message' => 'Non autoriz
↵ zato']);
            exit;
        }
    }
}
```

Concentrating session_start() and header() in a single file reduces the risk of “headers already sent” from stray spaces or BOMs scattered across files.

SitoRuntime: shared cors.php prelude. The site serves a frontend that in development comes from a different origin, so it puts a cors.php ahead of everything to handle headers, the Content-Type, and the OPTIONS preflight, then opens the connection lazily (getDB() with a static, copied into every file). The actual CORS is **not** open to everyone: it’s an allowlist of known origins, reflecting the Origin when it’s allowed.

```
// cors.php – shared prelude: an allowlist, not "*"; answers the preflight a
↵ nd exits
$allowed = ['https://runtimeradio.com', 'https://www.runtimeradio.com', 'htt
↵ ps://runtimeradio.it'];
$origin = $_SERVER['HTTP_ORIGIN'] ?? '';
if (in_array($origin, $allowed, true)) {
    header("Access-Control-Allow-Origin: $origin");
    header('Vary: Origin');
}
header('Content-Type: application/json');
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') { http_response_code(204); exi
↵ t; }
```

DISINTELLIGENZA: minimal inline, no CORS. It develops and ships same-origin, so it has no need for CORS headers: the .htaccess ensures the frontend and the API sit on the same origin, and the session cookie travels on its own. The bootstrap is pared to the bone.

NOTE

Access-Control-Allow-Origin: * almost never The asterisk is fine only for a public, read-only API with no cookies. The moment there’s a session, reflecting an allowlist of origins is the right choice, and it has to come with an explicit decision about Access-Control-Allow-Credentials (SitoRuntime doesn’t emit it, and as a result authentication stays de facto same-origin: the detail, with its effect on the client, is in Chapters 6 and 10).

The connection error: three responses

What happens when the database doesn't answer is a small maturity test, and the three sites pass it differently.

```
// SPW: correct HTTP code + JSON + log for diagnosis
catch (PDOException $e) {
    http_response_code(500);
    error_log('DB: ' . $e->getMessage());           // stays in the lo
    gs, never goes to the client
    echo json_encode(['status' => 'error', 'message' => 'Database non dispon
    ibile']);
    exit;
}
```

SimonePizziWebSite answers 500, in JSON, and writes the exception to the server logs. SitoRuntime answers 503 in JSON but **doesn't log**, and so it loses the diagnostics exactly when they'd be useful. DISINTELLIGENZA does the thing you shouldn't: `die("Connection failed: " . $e->getMessage())`, which prints the exception message (paths included) to the client, with no HTTP code and outside the JSON format.

WARNING

A connection error must never speak to the client A `PDOException` message can contain filesystem paths, database names, details that come in handy to an attacker. The rule: the correct HTTP code (500 or 503), a generic message to the client, and the real detail only in the server logs via `error_log()`. Printing `$e->getMessage()` to the page, the way DISINTELLIGENZA does, is information disclosure for free.

5. Media Processing

Uploading a file isn't a simple `move_uploaded_file`, it's a transformation. The upload's type maps to different folders, each with its own policy: images are public and resized, podcast audio goes through admin only, and participant audio goes to an isolated folder that (in the festival's case) is openly accessible. Every admin image is normalized to a maximum width (1920px), preserving the alpha channel of PNG and WebP. The full treatment, including the security pitfalls of uploads (validating the real bytes, the naming, the RCE chain of the public upload), is in Chapter 7: here it's enough to know that the file passes through GD before it lands on disk.

6. Input and Output Security

- **File names:** every upload is renamed with `uniqid()` and cleaned of special characters, so the user doesn't get control over the name (and therefore the extension). The why is in Chapter 7.
- **JSON integrity:** every response is preceded by `header('Content-Type: application/json')`, and on an error it carries the right HTTP code (400, 401, 403, 500) along with a descriptive JSON message.
- **`FILTER_SANITIZE_STRING` is deprecated** (as of PHP 8.1): in its place, `strip_tags(trim($var))`.

7. Buffer Handling

A PHP `Notice` or `Warning` printed in the middle of a response makes it invalid JSON, and the frontend breaks. The defense is twofold: `display_errors = 0` in production (errors go to the logs, not the page) and, where needed, `ob_start()` to control what goes out. It's the same principle as the connection error in §4: the client gets clean data, the diagnostics stay on the server.

IMPORTANT The Canon

- One file per endpoint with a router on `REQUEST_METHOD`; a gate that's selective per branch (public GET, mutations behind `Auth:::check`).
- CORS via an allowlist of origins with `Vary`, never `*` together with credentials.
- A connection error is written to the logs and answers the client generically: never `getMessage()` in cleartext.
- Force the time zone in **every** endpoint with time logic, and in production `display_errors = 0`.

**
**

Next Chapter: Frontend Bridge (APIs). The connection between React and PHP, and the three ways to read a payload that has no stable contract.

CHAPTER 6: Frontend Bridge (API.ts)

React talks to PHP through a single point. Not `fetch` calls scattered across the components, but an `api` object that gathers them all: one method per action (`login`, `getNews`, `uploadImage`, `submitVote`), grouped by domain with a few comments, imported everywhere with `import { api }.` No Axios, no React Query, no Redux, no global store. It's the thin-stack version of the data access layer: a flat facade over `fetch`, consistent with the model's philosophy (Ch. 4), as few dependencies as possible.

Under that shared surface there's a shared problem, and it's the real lens of the chapter: the PHP API has no uniform contract. The endpoints, grown by accretion, answer with different envelopes (a bare array, or `{ data, total }`, or `{ success, ... }`), and sometimes with an HTTP 200 even when things went wrong. None of the three sites version that contract. Each one just reads defensively whatever arrives, and the way it shoulders that is what tells them apart.

The three responses read well as a scale of *investment*. `SimonePizziWebSite` invests in the state layer: React Router's loaders as a data layer, and the "Double Read" of the payload. `SitoRuntime` invests in client-side security: a CSRF token kept in a module variable. `DISINTELLIGENZA` invests in a codemod, a script that patches the client after the fact. Three ways of holding React and PHP together when neither side has a stable contract. And as in the earlier chapters, the order of investment doesn't line up with solidity: the one who spends the most isn't the one who gets it least wrong.

The fourth site, `FDCA`, doesn't enter here. The fork rewrote and shrank the frontend into a public storefront, and it has no `api.ts` and no `fetch` to `/api`: where on the backend it was byte-identical to `DIS`, on the bridge it's simply absent. A shell disconnected from the CMS, which lives in the chapter on forking, not this one.

*
**

1. One Client, Thin Over `fetch`

Before the differences, the traits the three sites share. The client is a namespace object, not a class: no instance, no dependency injection, you import it and call it. There's no data-fetching library: shared state either lives in the router (`SPW`) or is component-local `useState` (`SR`, `DIS`), and the "fetching" is native `fetch`. Where possible the methods are typed (`NewsArticle[]`, `UserRole`), so the caller knows what to expect, but the real contract is dictated by the PHP, and it doesn't always match the declared types.

Authentication, finally, travels on the `HttpOnly` session cookie (Ch. 10): the client carries no bearer token and no persistent auth state. From this follows a detail that immediately splits the three sites, the base URL and the cookie.

```
// SPW api.ts:1-12 – base URL that switches prod/dev, shared config carrying
↳ the cookie
export const API_URL = import.meta.env.PROD ? '/api' : 'http://localhost:888
↳ 8/api';

const fetchConfig: RequestInit = {
  credentials: 'include', // propagates the PHP session cookie (in dev it
↳ crosses localhost:8888)
  headers: { 'Accept': 'application/json', 'Content-Type': 'application/js
↳ on' }
};
```

```
// SR api.ts:4 – fixed base, no switch: always same-origin
const API_BASE = '/api'; // no localhost:8888, and-crucially-no credenti
↳ als: 'include'
```

NOTE

credentials: 'include': when it matters and when it's noise SPW puts `credentials: 'include'` on every call, and it has to: in development the frontend and the backend sit on two different origins (`localhost:8888`), and without that option the session cookie wouldn't go out. SR and DIS never set it, because they develop and ship same-origin, where the cookie travels on its own under `fetch`'s default policy. There's an exact symmetry with the server side (Ch. 10): SR's CORS doesn't emit `Access-Control-Allow-Credentials`, so authentication is de facto same-origin on both sides, and there `credentials: 'include'` would be a line that does nothing. The rule: that option matters when the client and the API sit on different origins; same-origin, it's noise.

✱
✱✱

2. The Contract Isn't Uniform: Three Ways to Read the Payload

This is the heart of the chapter. The endpoints answer with different envelopes, and the client has to survive that inconsistency. The three techniques are the standout of the cluster.

SPW lives the hardest case: *the same* endpoint sometimes answers with a bare array, sometimes with a paginated object `{ data, total }`. The answer is to read the payload in two possible shapes, and this, and only this, is the pattern the model calls the “Double Read.”

```
// SPW loaders.ts:30-31 – the same endpoint is sometimes an array, sometimes
↳ { data, total }
const articlesData = Array.isArray(articlesRes) ? articlesRes : articlesRes.
↳ data;
const projectsData = Array.isArray(projectsRes) ? projectsRes : projectsRes.
↳ data;
```


SR doesn't have the same ambiguous endpoint, it has a mosaic: each endpoint has *its own* envelope, and the client knows it by heart. On two of them, though, where the response is a bare array on success but a `{ success:false, error }` object on failure, it puts in a type guard.

```
// SR – each endpoint has its own envelope, read by known shape:
// news.php → { success, data, meta }      admin?action=list → { success,
↳ articles, total }
// speakers.php / podcasts.php → BARE ARRAY, or { success:false, error } o
↳ n error
if (Array.isArray(res)) setPodcasts(res);    // Podcasts.tsx:14 – type guard
↳ against the error case
```

DIS takes the opposite road: it doesn't normalize at all. It returns the JSON as-is (“zero envelope”) and leaves it to the caller to know its shape.

```
// DIS api.ts – “zero envelope”: the client doesn't touch the shape, it pass
↳ es it raw to the caller
return await res.json();
```

TIP

Elastic contracts: Double Read, type guard, zero envelope Same root, three remedies. When an API grows by accretion and no one versions the shape of the responses, the client has to decide how much work to do in place of the missing contract. SPW reads the same payload in two possible shapes (the true “Double Read”: bare array *or* `{ data, total }`). SR knows the envelope of each endpoint and, where the shape is ambiguous, puts in an `Array.isArray` guard that tells the good data from the error. DIS doesn't normalize: it returns the JSON as-is and trusts the caller. None of the three is wrong; they're three points on the scale of how much server inconsistency the client agrees to absorb. A note on terminology: the “Double Read” is *this*, reading the shape of the success payload. It's not cloning the response to pull out an error message, which is a different thing with a different story (the next section).

✱
✱✱

3. When the Backend Is Wrong: the Lying 200 and the Lost Message

Sometimes the backend doesn't cooperate, in two distinct ways. The first: it answers HTTP 200 even on a logical failure, putting the error in the body. That's the case with some of DIS's endpoints (the newsletter), and the client patches it by hand, method by method.

```
// DIS api.ts:341-343 – some endpoints return HTTP 200 even on error: a per-
↳ method patch
const data = await res.json();
if (data.status === 'error') throw new Error(data.message);    // not a singl
↳ e interceptor
```

Mind the attribution, because it's easy to mistake this for a general rule: this is DIS's pattern, not a universal prescription of the model. It's the consequence of a backend

that doesn't always use the HTTP codes, and the bridge handles it case by case. The second way the backend fails to cooperate is the opposite: it answers with a precise error, and the client throws it away. To pull out that message when it's there, you have to clone the response.

```
// the block that EXTRACTS the error message from the body (response cloning
↳ ) - NOT the "Double Read"
if (!res.ok) {
  let err = 'Errore imprevisto dal server';
  try { const j = await res.clone().json(); err = j.message || err; } catc
↳ h (e) {}
  throw new Error(err);
}
return await res.json();
```

In DIS this block appears, identical, in twenty-five methods. Not because someone wrote it twenty-five times, but because a script injected it.

```
// DIS fix_api.cjs:4-23 - regex-replace: find every fetch and append the blo
↳ ck, if it's missing
const regex = /(const res = await fetch([\s\S]*?))/g;
code = code.replace(regex, (match, p1, offset, string) => {
  const nextLine = string.substring(offset + p1.length).trim().split('\n')
↳ [0];
  if (nextLine.startsWith('if (!res.ok)')) return match; // skip the one
↳ s that already have it → the "slipped-through" methods
  const check = '\n if (!res.ok) { /* ... res.clone().json() ... */ throw new
↳ Error(err); }';
  return match + check;
});
```

WARNING

The codemod that patches the client: power and traces DIS didn't hand-write its client's error handling: it generated it with a codemod. `fix_api.cjs` looks for every `const res = await fetch(...)` and, if there isn't already an `if (!res.ok)`, appends a block that clones the response and pulls out `message`. It works, and in a few seconds it covers the whole file. But it leaves three fingerprints. The first: the same identical block, word for word, everywhere. The second: the "slipped-through" methods, the ones that already had their own `if (!res.ok)` with a generic message (`login` with "Login failed," `uploadFile` with "Upload failed"), which the script skipped, leaving the bridge uneven. The third: a duplicated line, the residue of a second pass. It's a real case of client maintenance by automatic source transformation, with its upsides and its side effects. That block is exactly what a manual might sell as "the standard" for error handling: here it's the product of a machine, not a choice, and it's worth being able to recognize that.

What's left is the most widespread problem, and the most instructive because it's common to all three: the backend's error message that gets lost. The server (Ch. 10) went to the trouble of crafting precise statuses and texts, a 429 with "Troppi tentativi, riprova tra quindici minuti" (too many attempts, try again in fifteen minutes). The client throws it away, at three different points in the chain.

```
// SPW api.ts:22 – the error body (even a talkative 429) is discarded here,
↳ in the client
if (!res.ok) throw new Error('Login fallito');
```

```
// SR LoginForm.tsx:18-25 – here the body is preserved by api.login, but the
↳ UI throws it away anyway
catch (err) { setError('Login fallito. Controlla le credenziali.');
```

```
}; } // e
↳ rr.message ignored
```

In SPW the message is lost in the client, in SR in the UI, in DIS in patches (the methods the codemod touched preserve it, the slipped-through ones don't). Three different points, the same outcome: the rate-limited user always sees “Login fallito” and never the 429 that would tell them how long to wait.

WARNING

Read the body on the error branches too `res.ok` tells you *whether* the request succeeded, not *why* it failed; the why is in the body. Throwing it away, as all three sites do on login, wastes the work done on the server to make errors talkative, and leaves the user facing a useless message. The lesson is simple and often ignored: the body has to be read also, and especially, when the response isn't ok. Where you do it matters less than doing it: the point is not to replace a 429 “try again in fifteen minutes” with a generic “error.”

✱
✱✱

4. The CSRF Token on the Client

This is the investment that sets SitoRuntime apart, and the most sophisticated mechanism in the cluster. SR's backend (Ch. 10) returns a `csrf_token` in the body of `login` and `check_auth`, and demands the `X-CSRF-Token` header on all mutations. The client handles the handshake in the most minimal way possible: a module-level variable.

```
// SR api.ts:6-10 – the CSRF token lives in a module-level variable, in memo
↳ ry
let csrfToken = ''; // not localStorage, not Context
↳ t, not component state
function csrfHeaders() { return csrfToken ? { 'X-CSRF-Token': csrfToken } :
↳ {}; }
```

```
// SR api.ts:31,37 – captured from the login/check_auth body, sent back only
↳ on mutations
if (data.csrf_token) csrfToken = data.csrf_token;
// ...and on every POST/DELETE: headers: { 'Content-Type': 'application/json',
↳ n', ...csrfHeaders() }
```

The token is captured at login, reinjected on writes, and cleared at logout. SPW has none of this (the server-side Origin/Referer check is enough for it), and DIS has no CSRF at all: it's entirely SR's solution. But keeping it in a module variable has a hidden price.

WARNING

The CSRF token and the reload: a coupled guarantee The variable lives in memory: on a page reload it vanishes, and the first mutation afterward would go out

without X-CSRF-Token, taking a 403. The system holds only because the admin component reruns `checkAuth()` every time it mounts, and that call re-returns the token. It's a real dependency, but one declared nowhere: if tomorrow an admin page mounted an editor without going through that `checkAuth`, the writes would fail opaquely, with a 403 that on screen is indistinguishable from a save error. If a token has to survive a reload, put it where it actually survives (a `sessionStorage`, or an explicit handshake on every startup), not at the mercy of a side effect of a component mounting.

**

5. Protecting the Admin Area: Loader or Component

Protecting the restricted routes on the client is two schools doing the same thing. SPW uses a declarative guard: a loader mounted on the parent `/admin` route, which checks the session before the page mounts and redirects if it's missing.

```
// SPW loaders.ts:10-20 — one guard, N child pages: the session is checked b
↪ efore render
export const adminAuthLoader = async () => {
  const session = await api.checkSession();
  if (!session || !session.user) return redirect('/admin/login');
  return session;
};
```

SR and DIS use an imperative guard inside a component instead: `checkAuth` on mount, a state machine that shows the login until there's a user.

```
// SR Admin.tsx:74-189 — guard inside the component, run on mount
useEffect(() => { checkAuth(); }, []);
if (loading) return <Loader />;
if (!user) return <LoginForm onLogin={handleLogin} />; // no user → logi
↪ n form
```

The full architectural comparison, with its consequences (for example DIS's guard that checks the user but not the role), is in Ch. 14. Here only one thing matters.

NOTE

Loader or component: two schools, one real defense The practical difference between the two is a flash: with the loader you never see restricted content before the redirect; with the component there's an instant of "loading" before the verdict. But it's a difference of experience, not of security. In all three sites the real defense is the server-side gate (Ch. 10), and the client guard only serves to avoid showing a door the server would keep shut anyway. Hiding a page is user experience; preventing an action is security, and that has to be done on the server.

**

6. The Session That Expires While You Work

There's a hole that runs through all three sites, and it may be the most important one in the chapter because no one covers it. The admin-area guard fires once: when you

navigate (SPW) or when the page mounts (SR, DIS). But the session can die *afterward*, while you're writing an article, even just because a password change made elsewhere invalidated it via `session_version` (Ch. 10).

```
// in all three: the mutation gets a 401/403, the client shows a generic err
↳ or
// and does NOT redirect. There's no single place to recognize the status an
↳ d force a re-login.
```

WARNING

Handling session expiry in the thin stack With a dead session, the save takes a 401, the client shows “error while saving” and leaves you on a page that no longer works, with your work still on screen and no prompt to log back in. None of the three sites has an interceptor that recognizes the 401/403 and forces a re-login; in SR it's even worse, because a 403 from an expired CSRF token is identical, on screen, to any other error. It's the flip side of error handling living scattered across every method instead of in a single layer: a small centralized `request(path, opts)`, on top of removing the repeated `fetch` wrapper, would be the natural place to handle the expired session once and for all, along with the Double Read and the headers.

*
**

7. Upload and Pagination: the Same Three Hands

Two surfaces remain where the three philosophies show in miniature. The first is the upload. The bridge sends a `FormData`, and in all three there's the same trick: you remove the `Content-Type` header, otherwise the browser doesn't write the multipart boundary and the file never arrives. The trimmings change.

```
// SPW api.ts:438-509 — for FormData you REMOVE Content-Type; alternatively
↳ XHR for the progress bar
const { headers, ...rest } = fetchConfig;
const res = await fetch(`${API_URL}/upload.php`, { ...rest, method: 'POST',
↳ body: formData,
  headers: { 'Accept': 'application/json' } }); // variant: XMLHttpRequest
↳ st + xhr.upload.onprogress
```

SPW offers the progress bar via `XMLHttpRequest`; SR sends the `FormData` with the `X-CSRF-Token` but no progress, a simple spinner; DIS sends it with no progress and no CSRF. The server side of all this (the validation, the WebP conversion, DIS's public-upload RCE chain) is in Ch. 7.

The second surface is pagination, and it's tied tightly to the Double Read of §2. It's precisely the `{ data, total }` contract that makes reading in two shapes possible, and necessary: without a `total` you don't know how many pages are left.

```
// SPW useFetchArticles.ts:32-49 — the Double Read's { data, total } feeds t
↳ he load-more, with dedup
const data = Array.isArray(res) ? res : res.data;
const total = !Array.isArray(res) && res.total !== undefined ? res.total : d
↳ ata.length;
// ...accumulate the pages, dedupe by id, hasMore = (merged list).length < t
↳ otal
```

SPW accumulates the pages deduping by `id` and computes `hasMore` on the merged length. SR does the load-more without dedup, trusting the backend not to repeat. DIS doesn't paginate at all. And there's a silent trap: when the endpoint returns a bare array, with no `total`, SPW falls back on `data.length` as the total, and `hasMore` becomes `false` even when there would be more pages. It's the direct consequence of reading a contract that isn't guaranteed. The server side (the `{ data, total }`, the `COUNT` and the `LIMIT/OFFSET`) is in Ch. 9.

*
**

In Summary

The bridge between React and PHP is, in all three, the same object over `fetch`, but it grew around an API with no stable contract, and each one added its own touch. SPW in the state layer, with the loaders and the Double Read of the payload. SR in the CSRF token kept in a module variable. DIS in a codemod that patched the error handling after the fact, leaving its fingerprints (the repetition, the slipped-through methods, the duplicated line). Three different investments, and not one that makes the bridge truly solid: the backend's error message is lost in all three, and in all three the thing that would help most is missing, a single place to handle the expired session. The moral isn't "pick the richest client." It's that a wrapper over `fetch` is never just transport: it's where an imperfect API becomes, or fails to become, a reliable experience. The decisions that matter (reading the body on errors too, where the token lives, what to do when the session dies) all live there.

IMPORTANT The Canon

- A single `api` object over `fetch`, auth via the session cookie; for multipart uploads, remove the `Content-Type` header and let the browser set it.
- Define a stable payload shape and read it in one place: a uniform contract beats the "Double Read."
- Handle the CSRF token on mutations and centralize the reaction to 401/403 (an interceptor) for expired sessions.
- Read the response body on the error branches too: don't lose the backend's message (e.g. the 429).

*
**

Next Chapter: Media & Optimization. The upload as an attack surface: validating files for real, turning PHP off in public folders, and the chain that leads from an image to code execution.

CHAPTER 7: Media & Optimization

This chapter follows a file from the disk of whoever uploads it to the disk of the server: how it arrives, how it's checked, how it's slimmed down, where it ends up, and how it's served back. It's a short path, and that's exactly what makes it dangerous. In the three sites the skeleton is the same (an `upload.php` that receives a `multipart`, validates it, optimizes it with GD, and puts it on disk), but it's the cluster where security scales **counterintuitively**.

The defense against code execution from an upload runs from **three independent barriers** (SimonePizziWebSite) to **a single one** (SitoRuntime) to **almost zero on a public upload** (DISINTELLIGENZA), where the chain that leads from the image to a shell is verified. And two details flip your intuition: the “most minimal” naming (SR, which throws the file name away) is the **most secure**, while the “most accommodating” one (DIS, which keeps the name and the extension) is what opens the door. The question that runs through the chapter is a single one: *how much can you take away from an upload system before it becomes insecure*. And DIS shows what lies one step past that line, because an upload open to the public changes all the rules.

A note from the field: this is about **media**, not cache or SEO. The cache of the content lists lives in the content lifecycle (Ch. 9); the prerendering of metadata for bots is the PHP entry-point chapter (Ch. 11). This chapter stays on the file: uploading it, validating it, optimizing it, serving it.

*
**

1. The Shared Skeleton

Before the divergences, the traits the three sites share.

An `upload.php` receives **one file at a time**: POST only, `multipart/FormData` (sent by the client of Ch. 6), `$_FILES['file']`. No media-management library, no external service; the file arrives in the same request that processes it. The image optimization happens **synchronously, inside the endpoint**, via GD, behind an `extension_loaded('gd')` guard that degrades gracefully if GD is missing: no queue, no worker, no cron. The name is made collision-proof with `uniqid()`. The “library” and the “deletion” live in a separate file (`media.php`), distinct from the upload itself. And the reference to the file, inside the content, is always a **URL string** (`cover_image`, `audio_file`), not a foreign-key relation.

That last trait has a shared consequence, **dangling media**: deleting a file doesn't notify whatever cited it, and none of the three keeps a count of references. An image used as a cover, once deleted, leaves a broken `<img>` and no one notices until the page is opened.

```
// SPW upload.php – the canonical flow: gate, validate, optimize, record, re
↳ spond
Auth::check(); // only the admin uploads (S
↳ PW/SR; DIS does NOT, see §5)
// 1) validate extension + real bytes 2) choose subfolder 3) collision-pro
↳ of name
// 4) resize/WebP via GD 5) INSERT into `media` 6) echo { status, url, id,
↳ name }
```

✱
✱ ✱

2. Validating a File: the Extension Isn't Enough, and Neither Is the Content-Type

The endpoint's first job is to decide whether the file is really what it claims to be. SPW and SR do it in **two layers**: first the extension against a whitelist, then the **real bytes** read with `finfo/mime_content_type` against a whitelist of MIME types. It's the “don't trust the file name” principle: a `shell.php.jpg` passes the first filter but is stopped by the second, because its bytes aren't those of an image. That same real MIME also decides the subfolder, so the classification never trusts the declared extension.

```
// SPW upload.php:25-54 – two layers: extension, then the real bytes
$allowedExts = ['jpg', 'jpeg', 'png', 'webp', 'gif', 'pdf', 'zip', 'rar', 'mp3'];
if (!in_array($fileExt, $allowedExts)) { /* 400 */ }

$realMime = mime_content_type($file['tmp_name']); // reads the bytes, not
↳ the extension
$allowedMimes = ['image/jpeg', 'image/png', 'image/webp', 'image/gif', 'applicat
↳ ion/pdf', 'application/zip', /* ... */];
if (!in_array($realMime, $allowedMimes)) { /* 400: disguised content blocked
↳ */ }
```

DIS makes a different choice, and here the chapter's security story begins: it trusts `$_FILES['type']`, that is, the Content-Type **the browser declares**. No `finfo`, no extension whitelist, only the value that whoever sends the request controls entirely.

```
// DIS upload.php:100 – the "validation" looks at the client-declared MIME (
↳ spoofable)
if (!in_array($file['type'], $allowed)) { // $file['type'] = brows
↳ er header
    http_response_code(400); die(json_encode(['status'=>'error', 'message'=>'
↳ Invalid file format']));
}
```

WARNING

`$_FILES['type']` is not validation The Content-Type inside `$_FILES` isn't computed by the server: the client writes it into the `multipart` request, and it's faked with one line of `curl`. Validating against that value means asking the attacker whether their file is allowed. The real defense is to read the file's opening bytes (`finfo_file` / `mime_content_type`) and compare them against a whitelist of MIME types, using the *real* type to decide where to save it too. SPW and SR do this; DIS doesn't, and that “doesn't” is the first link in a chain we'll see in §5.

✱
✱ ✱

3. The File Name Is a Vector

The name the file ends up with on disk is a security decision too, and the three sites make it in three ways that map exactly onto three levels of risk. SPW prepends a `uniqid()` to a base stripped of dots, so a `shell.php.jpg` executable via `mod_mime` can never be born. SR makes the most radical move: it discards the original name entirely and uses a pure `uniqid()`, so no user string enters the final name. DIS keeps the name **and** the extension, and dots stay in the set of allowed characters.

```
// SPW upload.php:73-75 – base with NO internal dots, then uniqid: no double
↳ extension
$safeBase = preg_replace('/[^A-Za-z0-9\-\_]/', '', pathinfo($fileName, PATHIN
↳ FO_FILENAME));
$newFileName = uniqid() . '-' . $safeBase . '.' . $fileExt;
```

```
// SR upload.php:62 – total erasure: the user name is thrown out, only uniqi
↳ d + technical extension remain
$baseId = uniqid('', true); // no user input in the file
↳ name
```

```
// DIS upload.php:110-111 – keeps name and extension (dots allowed): the wea
↳ kest
$filename = uniqid() . '_' . basename($file['name']);
$filename = preg_replace('/[^a-zA-Z0-9\-\_]/', '', $filename); // the "." s
↳ tays -> extension preserved
```

TIP

The less you trust the name, the safer you are: the scale of erasure This is the most counterintuitive point in the chapter. The “most accommodating” choice—keeping the user-provided filename—is the riskiest, because it leaves the attacker in control of the final extension. The “most brutal” choice, deleting the name and replacing it with a server-generated identifier, is the safest, because it removes every foothold. SR sits at the protective extreme not because it added a defense, but because it removed a surface: the file name isn’t data to preserve, it’s input to neutralize.

4. Defense in Depth: Three Barriers, One, Zero

Validation and naming are application defenses, and they live inside `upload.php`. But what happens if that point is bypassed, or if tomorrow a second way to write into the folder gets added? Here comes the barrier that doesn’t depend on the PHP code: the `.htaccess` of the `uploads/` folder that **turns the PHP engine off**. If Apache no longer interprets `.php` in there, a malicious file that gets uploaded stays an inert file, whatever happened before.

```
# SPW public/uploads/.htaccess – the FIRST anti-RCE barrier: no PHP executio
↳ n in here
php_flag engine off
<FilesMatch "\.(php|phtml|phar|cgi|pl)$">
  Require all denied
</FilesMatch>
```

It's easy to look at the `uploads/.htaccess` as a matter of **cache control** (Expires, max-age), a performance detail, and stop there. Its critical use is another: it's the shutdown of PHP, and it's the first of SPW's three independent barriers. We've already seen the other two: the naming that doesn't generate executable names (§3) and the validation on the real bytes (§2). Three safety nets for the same risk, and each covers the other's gap: if the `.htaccess` weren't read, the naming saves you; if the naming failed, the `.htaccess` saves you; disguised content is stopped by the MIME check.

SR has only one of these safety nets, the application validation: there's no `uploads/.htaccess` (the folder is created at runtime and isn't in the repository), and the global `.htaccess` doesn't turn PHP off. As long as `upload.php` stays the only write path, it holds; but there's no second safety net. DIS has practically none of the three.

WARNING

A single barrier is not defense in depth “Validate the upload well” is necessary, not sufficient. Defense in depth means every single barrier can fail without the system falling, because there's another one behind it. Turning PHP off in the upload folder costs two lines of `.htaccess` and turns a possible flaw in the validation from “code execution” into “a useless file on disk.” It's the barrier with the best cost-to-protection ratio in the whole chapter, and it's also the one SR and DIS don't have.

**
**

5. The Perfect Storm: the RCE Chain from a Public Upload

The conditions seen so far, taken one at a time, would be manageable. In DIS they add up, and on a front the other two sites don't have: the **public upload**. Being a festival site, DIS accepts the audio tracks participants upload during sign-up, and to lower the friction that upload doesn't require a login. The gate is decided by type, and for two types it's simply absent.

```
// DIS upload.php:64-98 – the gate is per-type and INCONSISTENT: audio_parti
↪ cipant is public
if ($type === 'image') {
    if (!isset($_SESSION['user_id'])) { http_response_code(401); die(...); }
    ↪ // GATED
    $uploadDir = __DIR__ . '/../uploads/images/';
} elseif ($type === 'audio_participant') {
    $uploadDir = __DIR__ . '/../uploads/audio/participants/';
    ↪ // NO auth gate
} elseif ($type === 'audio_podcast') {
    if (!isset($_SESSION['user_id'])) { http_response_code(401); die(...); }
    ↪ // GATED
    $uploadDir = __DIR__ . '/../uploads/audio/podcasts/';
}
```

Now line up the four links. First: `type=audio_participant` doesn't ask for a login. Second: the validation looks only at `$_FILES['type']`, the MIME the browser declares (§2), which is fakeable. Third: the naming keeps the name and the extension (§3). Fourth: there's no `uploads/.htaccess` and the global `.htaccess` doesn't turn PHP off (§4), it only denies `.sqlite` and `.bak` files. The result is a single request.

```
# The chain: a public POST drops an executable PHP script
curl -F 'type=audio_participant' -F 'file=@shell.php;type=audio/mpeg' https:
↳ //site/api/upload.php
# -> no login required -> the declared "audio/mpeg" MIME passes the white
↳ list
# -> saved as /uploads/audio/participants/<uniqid>_shell.php -> Apache ex
↳ ecutes it
```

WARNING

A public upload changes all the rules An upload behind a login is a hygiene problem; a public upload is an attack surface open to the internet, and it has to be treated with the utmost suspicion. DIS’s four weaknesses, on their own, would be minor findings. Together, on an endpoint with no authentication, they produce remote code execution: the worst case. The defense isn’t one countermeasure but their sum: authenticate where you can, validate on the real bytes, neutralize the name, and above all turn PHP off in the folder, so that even if everything else gives way the file stays inert (Ch. 10). When the upload is public (the festival sign-ups, Ch. 17), these aren’t recommendations: they’re the minimum.

There’s an unpleasant corollary. FDCA, the fork of DIS, has a byte-identical `upload.php`: it inherits the chain intact, the public upload, the weak naming, and the absence of PHP-off, unchanged. A security flaw copied with a `git clone` multiplies without anyone rewriting it. And note the reversal of the usual refrain “more engineered means more fragile”: here the leanest site (SR, which removed everything removable) is more secure than the most “welcoming” one (DIS, which added the opening to the public without adding the defenses that opening requires).

*
**

6. Optimizing the Image: WebP, Resize, and What the Book Promised Too Much Of

Past the validation, the image gets slimmed down. Here the choices diverge, and it’s easy to flatten them into a single rule the real code doesn’t follow.

SPW and SR **convert** raster images to WebP via GD (quality 82, resizing if the width exceeds 1920px), then delete the original. DIS **doesn’t convert**: it only resizes, keeping the starting format (a PNG stays a PNG, smaller). The “mandatory WebP transcoding, the official standard” is therefore the pattern of two sites out of three, not all of them.

```
// SPW upload.php:83-119 – WebP conversion + resize, synchronous in the endp
↳ oint, behind a GD guard
if (in_array($realMime, ['image/jpeg', 'image/png', 'image/gif']) && extension
↳ _loaded('gd') && function_exists('imagewebp')) {
    if ($origW > 1920) { /* imagecopyresampled to 1920px, alpha preserved */
↳ }
        if (@imagewebp($img, $webpDestination, 82)) { unlink($destination); /* t
↳ he original goes away */ }
    }
}
```

Three details deserve a sharp clarification. The GIF is a choice, not an automatism: SPW flattens it into a single frame (the animation is lost), SR **keeps it animated** by excluding

it from the conversion. The GD re-encode, as a side effect, **strips the EXIF**: the geolocation and the device model disappear without anyone asking, a small privacy defense gained for free (SR notes it, the others don't). And the size constraint touches only the **width** above 1920px: there's no height limit, so a very tall, narrow image stays huge.

NOTE

WebP+resize in the thin stack: the variants, and its limits The optimization is synchronous: it happens inside the upload request, with no queue or worker. On a large image the user waits for GD before seeing the response, but for a low-traffic site (with the client's progress bar, Ch. 6) it's a reasonable trade-off. The real variants are three: convert to WebP flattening the GIF (SPW), convert preserving the animated GIF (SR), only resize leaving the format as it is (DIS). None is "the right one" in absolute terms; the wrong one is declaring universal a rule that two lines of code contradict.

✱
✱✱

7. The Library and the Deletion: When the Disk Is the Database

Once the file is uploaded, you need to list it and be able to delete it. On this axis SPW parts ways with the other two. SPW keeps a `media` table with two name columns: `file_path`, the technical URL the browser loads, and `filename`, the original human name. That second name serves a courtesy: `download.php`, a public proxy that streams the file with `readfile`, gives the user back `relazione.pdf` instead of `64f1a2-relazione.pdf`.

SR and DIS have no table: the library is the filesystem, read with `scandir` (flat in SR, recursive in DIS). The disk **is** the media database. Simple, but with a cost: no original name, no saved MIME, ordering by the file's physical date. And the dangling media, common to all, here becomes outright impossible to track, because there isn't even a starting point from which to count the references. In DIS the orphans are competition records: data that gets lost.

The deletion is the most delicate point, because it's a mutation that touches the disk, and it has to be protected on two fronts: the path and the request. On the path, the three sites scale again. SPW resolves the path with `realpath` and checks that it really sits inside `/uploads/`, not even trusting its own database. SR uses only `basename()`. DIS just rejects `..` with a `stripos`, then attempts the `unlink` on several candidate paths.

```
// SPW media.php:31-39 - path guard with realpath: doesn't even trust the DB
$physicalPath = realpath(__DIR__ . '/../' . $filePath);
$uploadsBase = realpath(__DIR__ . '/../uploads');
if ($physicalPath && $uploadsBase && str_starts_with($physicalPath, $uploads
↳ Base)) {
    unlink($physicalPath);                // only if REALLY contained
↳ in /uploads
}
```

```
// SR media.php - no auth_utils, a bare session, and NO validateCsrf on the
↳ delete
```

```

session_start();
if (!isset($_SESSION['user_id'])) { http_response_code(401); exit; } // no
↳ CSRF, no role check
$filename = basename($_input['filename'] ?? ''); // the only traversal defe
↳ nse
if (file_exists($uploadDir.$filename)) unlink($uploadDir.$filename);

```

WARNING

The media delete: the path guard and the missing token On the second front, the request, SR and DIS have the same hole: the deletion **has no CSRF protection**. In SR `media.php` doesn't even include the authentication prelude of the rest of the site, it runs on a bare session and doesn't check the role; in DIS the CSRF protection doesn't exist at all. It means a malicious site can make the logged-in admin delete files with a forged request, mitigated only by the cookie's `SameSite` (Ch. 10). It's the typical inconsistency: the upload demands the token, the delete doesn't, and yet deleting is as destructive as uploading. An endpoint that changes state has to be protected as such, always, even when it lives in a "utility" file that gets less attention.

✱
✱✱

8. Evolving the Storage Without Stopping the Site

Every site carries the scars of a storage migration: from raster to WebP, from a flat folder to subfolders. You read them in the one-shot scripts, and they tell the same sequence: first the flat uploads, then a batch conversion of what was already there, then the sorting into subfolders, finally the realignment of the references still pointed at the old path.

The difference that matters is how much protection these powerful scripts have, the ones that move files and rewrite the database. SPW's follow the "upload via FTP, run from the browser, delete right away" pattern, with dry-run active by default (you look before you touch). SR's live inside `admin.php`, behind the login. DIS's, `migrate_media.php`, **has no gate at all**: anyone, from the internet, can trigger the mass relocation of the files and the updating of the rows. The full mechanics of these migrations, and the discipline they require, is the chapter on database evolution (Ch. 15); here the symptom is enough, along with the warning that powerful maintenance exposed over HTTP is a back door as much as the upload is a front one.

✱
✱✱

In Summary

The media side of the three sites starts from the same skeleton and diverges on a single axis that truly matters: how many independent defenses stand between an uploaded file and code execution. SPW has three (PHP off in the folder, naming that doesn't generate executables, validation on the real bytes), and each covers the other's failure. SR has one, the application validation, robust as long as it stays the only write path. DIS has almost none, and on a public upload at that: the sum of an absent gate, a MIME trusted from the client, a preserved name, and PHP not turned off is the RCE chain

that the FDCA fork inherited intact too. The optimization choices (WebP or resize only, flattened or animated GIF) and the storage ones (a `media` table or a bare filesystem) matter for performance and maintainability, but they don't move the risk. The risk is in the file name, in the bytes you don't check, and in the engine you didn't turn off. The chapter's rule is that an upload shouldn't be made "richer": it should be made redundant, because the first barrier, sooner or later, gives way.

IMPORTANT
The Canon

- Validate files by magic bytes (`finfo`), never by the client-declared MIME.
- Rename with `uniqid()` and discard the original name; rebuild the extension from an allowlist.
- Defense in depth: an `.htaccess` with PHP-off in the upload folders (the first anti-RCE barrier), plus validation and naming. A single barrier is not defense in depth.
- Never an ungated public upload into an executable folder; the media delete goes through a path guard (`realpath/containment`) and a CSRF token.

Next Chapter: Advanced Content Editing & Media Integration. The content editor, and how the HTML it produces is kept safe at the moment of showing it.

CHAPTER 8: Advanced Content Editing

The content editor is where a CMS becomes tangible: it's where the author writes, formats, pastes, drops in an image or a video. It looks like an interface detail, but it carries two underlying questions this chapter holds together. The first is how much editor you actually need, because it ranges from a library of a dozen packages down to zero dependencies. The second, more serious, is where the defense against XSS lives: the content the author composes is HTML, that HTML ends up in the database, and sooner or later someone prints it to the page. If no one cleans it up, a `<script>` written by whoever has access to the editor becomes code executed in every reader's browser.

The three sites answer on three rungs. SimonePizziWebSite (SPW) has a hardened Tiptap editor, with insertion guards and sanitization at render. SitoRuntime (SR) uses the same Tiptap engine, with the same defense at render, but with weaker guards and one extra story behind it: a migration from Quill that left scars in the code. DISINTELLIGENZA (DIS) drops to the handcrafted rung, a hand-driven `contentEditable`, and here the XSS defense disappears entirely. The editor scale and the security scale, we'll see, don't line up: the real dividing line isn't which engine you use, but whether you clean up the HTML at the moment of printing it.

NOTE

Two misunderstandings to clear up right away. It's easy to think of this editor as "a native-React solution that gives up the heavy external dependencies." That's true only for DIS. The two flagships (SPW and SR) use **Tiptap v3** (ProseMirror), that is, a dozen `@tiptap/*` packages: an external dependency that's anything but light. And the "Paste Protection," which at first glance looks like a defense capable of removing "scripts and dangerous attributes," is really just a cosmetic cleanup of the paste. The real XSS defense is elsewhere, at the moment of render, as we'll see in this chapter.

*
**

1. The Editor: a Three-Rung Scale

The same need, composing rich HTML, is solved at three levels of dependency.

1.1 SPW: a "hardened" Tiptap v3

SPW factors out a reusable `RichTextEditor` component around Tiptap v3, with a focused set of extensions: formatted text, images, color, alignment, tables, YouTube embeds in `nocookie` mode. Each extension brings its default Tailwind classes, so the saved HTML is already styled for the public render.

```
// src/components/admin/RichTextEditor.tsx:95-138 (excerpt)
const editor = useEditor({
  shouldRenderOnTransaction: true,    // v3: needed for the toolbar's act
  live states
  extensions: [
    StarterKit.configure({ heading: { levels: [1, 2, 3, 4] } }),
    Image.configure({ HTMLAttributes: { class: 'max-w-full h-auto rounded
d-xl my-4 ...' } }),
    TextStyle, Color,
    TextAlign.configure({ types: ['heading', 'paragraph'] }),
    Table.configure({ resizable: true }), TableRow, TableHeader, TableCe
  ],
  content: value,
  onUpdate: ({ editor }) => { onChange(editor.getHTML()); updateCounts(editor
  .getText()); },
});
```

1.2 SR: the same Tiptap, but with a migration behind it

SR uses the same engine, but embedded directly inside `ArticleEditor.tsx` (the toolbar and `useEditor` in the same file), and it keeps a second copy in `SpeakerEditor.tsx` for the speakers' bios. The rich editing isn't centralized in a single component as in SPW: it's duplicated per domain, and a change to the configuration has to be replicated in two places.

Its identity, though, lies elsewhere: SR migrated from Quill to Tiptap, and the migration is written into the code. The old articles contain “bare” YouTube `<iframe>`s left by Quill, which Tiptap can no longer edit. A function rewraps them into the format Tiptap recognizes, before handing them to the editor:

```
// src/components/admin/ArticleEditor.tsx:29-36
// Converts Quill's bare iframes into div[data-youtube-video] that Tiptap ca
↵ n edit.
const QUILLYT_RE = /<iframe\b(?:\s|>)*\bsrc=["'](\https?:\\\/(?:www\.)?youtube(
?: ??:\nocookie?)\?.com\/embed\/[^\s">]+["'])(?:\s|>)*(<\/iframe>|\/>)/gi;
function prepareForEditor(html: string): string {
  return html.replace(QUILLYT_RE, (_m, src) =>
    '<div data-youtube-video=""><iframe src="' + src + '" allowfullscreen="tr
↵ ue" frameborder="0"><\/iframe><\/div>');
}
// usage: setLoadedContent(prepareForEditor(res.article.content || ''));
```

Two other traces of the old library remain: Quill's `ql-video` CSS class, kept inside the Tiptap configuration, and the `react-quill.d.ts` file still in the repo. Three converging clues of an editor switch never fully cleaned up.

TIP

Changing the editor with content already written: the compatibility shim When you change the editor engine, the old content stays in the previous format. You have two roads: a one-time migration that rewrites every record in the DB, or a shim that converts the old format “on the fly,” every time you open content to edit it. SR chose the second, with `prepareForEditor()`. The upside is that you don’t touch the database and don’t risk a mass migration gone wrong; the price is that the shim stays in the

code forever, or at least as long as a single old, never-reopened article exists. It's a debt you pay in small installments instead of all at once.

1.3 DIS: the handcrafted editor

DIS gives up the library entirely. The editor is a `<div contentEditable>` driven by `document.execCommand`, the now-deprecated DOM API. The only external dependency is showdown, and it serves only to convert pasted markdown.

```
// src/components/RichTextEditor.tsx:94-102 (excerpt)
const exec = (command: string, value?: string) => {
  document.execCommand(command, false, value);      // deprecated API
  if (editorRef.current) onChange(editorRef.current.innerHTML);
};
// <div ref={editorRef} contentEditable onInput={handleInput} className="pro
↵ se prose-invert ..." />
```

It works, and in today's browsers it still runs. But it's the most fragile option: `contentEditable` produces different HTML from one browser to another, `execCommand` is deprecated, and its future behavior isn't guaranteed.

NOTE

How much editor you actually need DIS demonstrates the absolute minimum, zero packages for editing, and at the same time shows its price: inconsistent HTML, a deprecated API, and as the “source of truth” the DOM's raw `innerHTML` instead of the controlled output of a serializer like `getHTML()`. The scale isn't “more packages means better.” It's a trade-off choice: SPW and SR pay a dozen dependencies to have a controlled schema and a rich UX; DIS pays nothing and accepts the fragility. What DIS can't afford to remove, as we'll see in §3, isn't the editor: it's the defense at render.

✱
✱ ✱

2. The HTML Is the Source of Truth, and the Server Saves It Raw

Underneath the three implementations there's a shared architectural choice. None of the three keeps a JSON state or a proprietary AST to re-serialize: the editor emits HTML (Tiptap's `getHTML()`, or the `contentEditable`'s `innerHTML`) and that HTML is exactly what ends up in `articles.content` or `news.content`. The content is already in the form it will be shown in.

And the server doesn't touch it. Consistent with what we saw in Ch. 9 on the content lifecycle, the backend saves the HTML body as-is, with no `strip_tags` or `htmlspecialchars`:

```
// SPW articles.php:252 – the content goes into the DB raw
$content = $data['content'] ?? '';                      // no strip_tags / ht
↵ htmlspecialchars
$stmt = $pdo->prepare("INSERT INTO articles (title, slug, content, ...) VALU
↵ ES (?, ?, ?, ...)");
$stmt->execute([$title, $slug, $content, ...]);          // HTML as-is
```

This isn't an oversight, it's the model: the rich content is kept faithful and the defense moves to the moment it's printed. The whole weight of security, then, rests on a single gesture. Let's see where.

**
**

3. Rich-Content Security: Where the XSS Defense Lives

The `content` is saved raw, and writing is reserved for the authenticated (admin or editor login, as in Ch. 10). So this is a stored XSS from an authenticated author: it isn't the anonymous visitor injecting the script, but whoever has access to the editor. The risk is still real, because a compromised editor account, or a careless copy-paste from a hostile source, is enough to plant dangerous HTML in the database. The question is: is there a point where that HTML gets cleaned up before reaching the reader's browser?

3.1 The choke point: DOMPurify at render time

In SPW the answer is a single function, called right before the public component's one `dangerouslySetInnerHTML`. It passes the HTML through DOMPurify and, with a hook, allows only YouTube `<iframe>`s, removing any other:

```
// src/components/SingleArticle.tsx:28-45
const sanitizeArticleHtml = (html: string): string => {
  DOMPurify.addHook('uponSanitizeElement', (node, data) => {
    if (data.tagName === 'iframe') {
      const src = (node as HTMLInputElement).getAttribute('src') || '';
      if (!src.startsWith('https://www.youtube.com/embed/') &&
          !src.startsWith('https://www.youtube-nocookie.com/embed/'))
        ↪ {
          ↪ node.parentNode?.removeChild(node); // non-YouTube ifram
        }
    }
  });
  ↪ try {
    ↪ return DOMPurify.sanitize(html, {
      ↪ ADD_TAGS: ['iframe'],
      ↪ ADD_ATTR: ['style', 'allowfullscreen', 'frameborder', 'allow', '
    ↪ src'],
    ↪ });
  } finally {
    ↪ DOMPurify.removeHooks('uponSanitizeElement'); // no leftover glob
  }
};
```

Two details deserve attention. The hook is always removed in the `finally`, so it doesn't stay registered globally between one render and the next. And having granted the `style` attribute widens the surface: it's needed for the editor's inline colors, and it's DOMPurify that cleans up the dangerous CSS, but it's a trade-off between visual fidelity and security to keep in mind.

SR does the same thing, with DOMPurify, on all the detail pages (article, speaker, podcast). Same philosophy, same choke point at render time.

DIS doesn't. `NewsDetail.tsx` injects the raw HTML, and `dompurify` isn't even among the project's dependencies:

```
// src/pages/NewsDetail.tsx - no sanitization
<div dangerouslySetInnerHTML={{ __html: news.content }} /> // the DB's HTML
↳ L injected as-is
```

WARNING

Where to sanitize content HTML: the choke point at render time Saving raw and cleaning at render is a legitimate choice, on one condition: that the render really cleans up, and that it's the only way that content reaches a browser. SPW and SR meet the condition with DOMPurify. DIS doesn't, and it's the only one of the three where the stored XSS meets no choke point: raw on write, raw at render. Its only defense is the admin/editor boundary on access to the editor. Zero defense in depth. The dividing line between the two secure sites and the exposed one isn't the editor (Tiptap vs. contentEditable): it's the presence or absence of this function.

3.2 The insertion guards: a second level, not the first

There's a second defense, more visible to the author but less decisive. When you insert a link from the toolbar, SPW validates its URL on the spot, blocking dangerous schemes:

```
// src/components/admin/RichTextEditor.tsx:36-42
const isSafeLinkUrl = (url: string): boolean => {
  const trimmed = url.trim();
  return /^https?:\/\//i.test(trimmed) || trimmed.startsWith('/')
    || trimmed.startsWith('#') || /^mailto:/i.test(trimmed); // blocks
  ↳ javascript: and data:
};
```

SR and DIS don't have this filter. SR does a bare `setLink({ href: url })`; DIS takes the URL from a `prompt()` and passes it to `createLink` without looking at it, so a typed `javascript:alert(1)` ends up in the HTML exactly as-is:

```
// DIS RichTextEditor.tsx:104-107
const promptLink = () => {
  const url = prompt('Inserisci URL:');
  if (url) exec('createLink', url); // no filter: 'javascript:...' would
  ↳ be inserted as-is
};
```

TIP

Two levels, one real choke point The insertion guard (`isSafeLinkUrl`) and the render sanitization (DOMPurify) aren't interchangeable. The first covers only what comes through the toolbar, and it doesn't touch HTML that's pasted or built some other way: it's defense in depth and good UX, because it warns the author right away instead of letting the URL through to the public page. But the real barrier, the one that intercepts any HTML however it got into the DB, is the render. If you have to pick one, pick the choke point at render. Having both is better; having only the first is an illusion of security.

3.3 Cleaning the paste isn't defending against XSS

That leaves the old "Paste Protection" misunderstanding to clear up. DIS, when it intercepts a paste, does two things: if it recognizes markdown it converts it with `showdown`,

otherwise it cleans the HTML of inline styles and the `classes` from Word or Wikipedia. Useful for not importing unwanted fonts and backgrounds. But it isn't security:

```
// DIS RichTextEditor.tsx:63-85 (excerpt) - COSMETIC cleanup
const doc = new DOMParser().parseFromString(htmlText, 'text/html');
doc.querySelectorAll('[style]').forEach(el => { el.style.backgroundColor='';
↳ el.style.color=''; });
doc.querySelectorAll('*').forEach(el => el.removeAttribute('class')); // r
↳ removes styles/classes, NOT script/handlers
document.execCommand('insertHTML', false, doc.body.innerHTML);
```

It removes `style` and `class`, but it lets `<script>`, the `on*` attributes, and `javascript:` URLs through. The comment in the code talks about “enemy classes,” not XSS, and it’s honest: it’s cosmetics. It has to be told apart, clearly, from a real sanitization. Removing Word’s formatting improves how the text looks; it protects no one.

**

4. Anchor Box: the Four content Emitters

Here a thread opens that runs through four chapters, and that’s worth pinning down once. The starting point is what we’ve just seen: the `content` is saved raw in the database, and the sanitization lives only in the React render. But the React render isn’t the only place that content leaves for the world. The same `articles.content` is reread and re-emitted by at least four different “emitters,” and each has to defend itself on its own. The defense that lives in a single component doesn’t cover the other three.

Emitter	Where	What it does with the content	Anti-XSS defense	Outcome
React render	public page (this chapter)	it via dangerouslySetInnerHTML	DOMPurify (SPW, SR) / nothing (DIS)	real choke point; DIS exposed
SEO prerender	index.php for bots (Ch. 11)	re-emits its HTML body	strip_tags with an allowlist (SPW, SR) / doesn't emit the body (DIS)	attribute hole alive in SPW and SR; DIS immune by subtraction
RSS feed	rss.php/feed_* (Ch. 12)	emits the article	doesn't emit the content (SPW uses the excerpt; DIS does podcasts only) / escapes it (SR: strip_tags + htmlspecialchars)	closed
Newsletter	email send (Ch. 13)	sends the article	no one emits the content	closed

Here’s the read. The feed and the newsletter close the problem, because they either don’t emit the body or escape it entirely. The render closes it where there’s DOMPurify

and leaves it open in DIS. The only flaw still alive in the flagships is the SEO prerender: to serve bots an indexable body, SPW and SR re-emit the `content` passing it through `strip_tags` with a tag allowlist, which isn't `DOMPurify`. `strip_tags` removes the disallowed tags but doesn't touch the attributes: an `onerror` or a `javascript:` inside an allowed tag survives. And since SR copied SPW's SEO engine to the letter, it copied the flaw too.

IMPORTANT

The lesson of the picture: a shared server-side sanitization When you save the content raw and entrust the cleanup to whoever prints it, you're betting that *every* print point remembers to clean up. Four emitters are enough, and it only takes one to forget: in DIS the render forgets, in the flagships the prerender forgets. The conclusion that runs through the next three chapters is that the sanitization should live once, on the server, where the content is produced or reread, instead of being reinvented by every consumer. The thread reopens in Ch. 11 (where the prerender hole is alive), closes again in Ch. 12 (the feed that escapes), and closes fully in Ch. 13 (the newsletter that doesn't emit).

*
**

5. Inserting Media into the Content

Managing the media library (upload, optimization, formats: that's Ch. 7) is one thing; embedding an image *inside* the article's text is another. Here it's the second that matters, and it's another point where the three sites diverge.

SPW opens a modal gallery, `MediaSelectorModal`, which reuses the same upload bridge from Ch. 7: the author picks an already-uploaded image or uploads a new one with a progress bar, and the URL comes back to the editor.

```
// src/components/admin/MediaSelectorModal.tsx:65-71 (excerpt)
const result = await api.uploadMediaWithProgress(file, (p) => setUploadProgr
↵ ess(p));
if (result.url) onSelect(result.url);    // the URL becomes an <img src> in
↵ the editor
// gallery: onClick={() => onSelect(item.file_path)}
```

SR skips the gallery: an `<input type="file">` created on the fly, a direct upload, and done. Simpler, but no reuse of images already present. DIS doesn't allow images in the article body at all: the editor covers only formatted text and links, and the cover is handled elsewhere.

On the portability side, all three save **only relative paths** in the database (for example `/api/uploads/file.jpg`), so the content isn't tied to a domain. When an absolute URL is needed, at the moment of copying or sharing a link, it's the client that rebuilds it from `window.location.origin`. The thing saved stays portable; the thing shown adapts to the context.

*
**

6. The Editorial Micro-Touches

Under the differences, the writing experience shares a few small attentions worth gathering, because they're the kind of detail that tells a usable editor apart from one that frustrates.

All three show the word count and a reading-time estimate in real time. All have a toolbar that stays visible while you scroll a long article (sticky), with buttons that don't lose the text selection (onMouseDown with `preventDefault`). And all have to solve the same subtle problem: when the content changes "from outside" (because you open an existing article), updating the editor without making the cursor jump or marking the form as modified. Tiptap does it with `setContent(value, { emitUpdate: false })`, and the comment in SPW's code explains why:

```
// SPW RichTextEditor.tsx:157-165 – the Tiptap v3 gotcha
// In v3 setContent emits onUpdate by default, marking the form "dirty"
// and creating phantom drafts in localStorage. emitUpdate:false avoids it.
editor.commands.setContent(value, { emitUpdate: false });
```

DIS gets the same result by hand, updating the `innerHTML` only when the editor isn't in focus.

Where the sites really diverge is in the safety net against losing work. SPW saves a draft to `localStorage` every couple of seconds and, on reopening, offers a banner to restore the interrupted session; the "modified" state also feeds a block on navigation. SR stops at `beforeunload`, the browser's warning when you close the tab with unsaved changes. DIS has neither.

TIP

Not losing the editor's work without a backend In the thin stack there's no autosave endpoint: the draft lives in the browser. SPW's solution (`localStorage` plus a restore banner) is the most robust at zero backend cost, and it turns a browser crash into an explicit choice ("restore" or "ignore") instead of lost work. The limit remains: if you switch devices or clear the cache, the unsaved draft is gone anyway. But between DIS's nothing and SR's warning alone, a local draft is the difference between losing half an hour of writing and getting it back.

IMPORTANT The Canon

- A Tiptap editor, HTML as the source of truth saved raw.
- The defense against XSS is sanitization **at render** (DOMPurify), a single choke point; the paste cleanup is cosmetic, not security.
- Guards on link insertion (`isSafeLinkUrl`): no `javascript:` and no unvalidated URLs.
- The four `content` emitters (render, SEO prerender, feed, newsletter) share the same server-side sanitization: if one forgets it, the XSS hole reopens.

Next Chapter: Content Lifecycle. The lifecycle of content, from draft to scheduled publication, and the three visibility rules that decide what the public really sees.

PART IV

The Operational Flow

How content lives. From the lifecycle to distribution, by way of security and SEO.

CHAPTER 9: Content Lifecycle

This chapter turns a database into an editorial system: how a piece of content is born as a draft, gets scheduled, goes public, and how the API that serves it decides shape, count, and visibility. It's also the chapter that closes a thread opened in Chapter 6: the payload contract the client reads defensively is born here, on the server side.

1. Dynamic States vs. Persistent States

The database stores a fixed state (`status`), but the application computes a dynamic one that depends on time. The clean matrix is this:

status (DB)	published_at	Real state (UI)	Description
draft	any	draft	never visible to the public
published	in the future	scheduled	visible to admins only, waiting
published	in the past	published	visible to everyone

This is the matrix of **SimonePizziWebSite**, and it's the tidiest one. The other two sites carry the scars of their migrations: **SitoRuntime** filters the public list with `status = 'published' OR status IS NULL`, where the `IS NULL` is the residue of the `status` column having been added outside the base schema; **DISINTELLIGENZA** still queries `status = 'scheduled'`, a state a migration declared obsolete but that the code keeps looking for. The tidy matrix is a destination, not the natural state of things (Chapter 15).

2. Scheduling without Cron: the Three Strategies of the Present

The idea is elegant and shared: no scheduled job. An article with `published_at` in the future is “published but not yet visible,” and it shows up on its own when the visibility query finds `published_at <= now`. On the React side, the dashboard computes the same state on the fly to give the admin immediate feedback:

```
const isDraft    = item.status === 'draft';
const isScheduled = item.status === 'published' && new Date(item.published_at) > new Date();
const isPublished = item.status === 'published' && new Date(item.published_at) <= new Date();
```

The delicate part is what “now” means. There's no single right answer: the three sites compute it in three ways, and each has its own way of getting it wrong.

- **SimonePizziWebSite** forces `date_default_timezone_set('Europe/Rome')` and compares in PHP. Correct, but only as long as the timezone is forced in *every* endpoint (Chapter 5).

- **SitoRuntime** compares `date('Y-m-d H:i:s')` strings with a space separator. The query is right, but if the client sends a date in ISO format with the τ , the string comparison breaks: it's the incident documented in `debug_time.php`.
- **DISINTELLIGENZA** delegates to SQLite's `CURRENT_TIMESTAMP`, which is in **UTC**, while `published_at` is stored in the server's timezone. An article appears or disappears off by an hour or two.

WARNING

Who computes the present: PHP or the database? Converting `datetime-local` (the browser's τ) to the DB format (the space) isn't "the standard": it's the sore point for anyone who compares dates as strings, namely **SitoRuntime**. The other two strategies avoid that problem but introduce others (the timezone to force everywhere in SPW, the UTC offset in DIS). The practical rule: pick a single source of the present and use it consistently. If you compare in PHP, force the timezone in the shared prelude; if you compare in the database, make sure `published_at` and `NOW/CURRENT_TIMESTAMP` live in the same timezone. Mixing them is the recipe for a post that shows up an hour ahead of schedule.

```
// the T <-> space conversion is SitoRuntime's, not a universal rule
value={published_at.replace(' ', 'T').slice(0, 16)} // DB -> UI
onChange={e => setPublishedAt(e.target.value.replace('T', ' ') + ':00')} /
↵ / UI -> DB
```

3. The Response Contract: Where the Double Read Closes

In Chapter 6 the client read the payload defensively, because the shape of the responses isn't uniform. The why is here, on the server side: the contract was never versioned, it was **extended whenever needed** (adding pagination, a `success` wrapper), and so the envelope differs by endpoint and by site.

- **SimonePizziWebSite**: *only* the article list returns `{ data, total, page, limit }`; everything else (projects, categories, tags) is a bare array. The client does the "Double Read" not because an endpoint changes shape, but because it mixes the two families in its loaders.
- **SitoRuntime**: three different envelopes, `{ success, data, meta }` for news, `{ success, articles, total }` for the admin, a bare array for speakers and podcasts. A per-endpoint mosaic.
- **DISINTELLIGENZA**: always a bare array or object, the "zero envelope."

The pagination count lives in different places too. SPW returns the raw `total` and leaves the `hasMore` calculation to the client (with a separate `COUNT(*)` over the same conditions and, a mandatory MySQL detail, the `LIMIT/OFFSET` bound with `PARAM_INT`, otherwise PDO treats them as strings and the query fails). **SitoRuntime** pre-computes `total_pages` on the server and puts it in `meta`, so the client's load-more is trivial. **DISINTELLIGENZA** gives no metadata at all: the client asks for the next page blind, until it comes back empty.

NOTE

Extending a contract instead of versioning it: what it costs Adding `{ data, total }` to an endpoint that used to return a bare array is the quickest thing in the world, and it doesn't break anything right away. The cost comes later, and the client pays it: it has to guess which shape it will receive, and when it guesses wrong (a bare array read as if it carried the count) you get the broken `hasMore` from Chapter 6. Versioning an API costs discipline; extending it in place costs widespread fragility. For a small CMS the second is a legitimate choice, but it should be made knowing that the price moves to the frontend.

4. The Content Cache without Redis

SitoRuntime is the only one of the three to add a read cache, and it does it in the most thin-stack way possible: a JSON file on disk, no Redis, no Memcached. The news list is served from a file in `.cache/`, regenerated only when it's older than the TTL.

```
// SR news.php - file cache with a TTL and a diagnostic header
$cacheFile = __DIR__ . "/.cache/news_p{$page}_l{$limit}.json";
if (file_exists($cacheFile) && (time() - filemtime($cacheFile) < 300)) { //
↳ TTL 300s
    header('X-Cache: HIT');
    echo file_get_contents($cacheFile);
    exit;
}
// ...otherwise query the DB, save the JSON, and:
header('X-Cache: MISS');
```

Two details keep it sound. Invalidation is explicit: on every `save` or `delete` the `.cache/` folder is wiped, so the public never sees stale data after a change. And the `X-Cache: HIT/MISS` header makes it possible to check from the outside whether the cache is doing its job. It's the same mechanism that, applied to SEO, becomes the anti-bot shield of Chapter 11; here it just keeps the database from being queried on every visit to the same list page.

5. Taxonomies: Categories and Tags (and When You Don't Need Them)

Here the Model shows its scale again, and a common assumption needs correcting: relational multi-tagging isn't everyone's standard, it's the choice of **one** site when it's genuinely needed.

SimonePizziWebSite is the full taxonomic blog: hierarchical categories with `parent_id` (a container category filters its subcategories too, via `IN`), and tags in a many-to-many relation on `article_tags`. But this isn't an "exclusive switch" to the relational model: the function that saves the tags also writes, **in parallel**, the old CSV field `articles.tags`, kept as a backward-compatibility cache. It's the coexistence of the old model and the new one during a migration that was never closed, not a clean cut.

```
// SPW - double write: new M:N relation + legacy CSV field in parallel
syncArticleTags($pdo, $articleId, $tagIds); // article_tags
↳ table (new)
```

```
$pdo->prepare("UPDATE articles SET tags = ? WHERE id = ?") // CSV field (le
↳ gacy, backward-compat cache)
    ->execute([implode(',', $tagNames), $articleId]);
```

SitoRuntime and **DISINTELLIGENZA** have no relational tags at all: the `category` is a free string (in DIS with a default of `'generale'`), and the “tags,” where they exist, are a `TEXT` or `JSON` field. For a site that isn’t a taxonomic archive, a categories table is weight you don’t need: a string is enough.

TIP

When You Don’t Need a Categories Table The `parent_id` hierarchy with M:N tags is right for a blog with hundreds of articles to filter by topic. For a radio with a few fixed categories, or a festival, a free string does the same job without joins, without a relation table, without a taxonomy editor. Adding relational taxonomy “because it’s more correct” is the kind of complexity the Model invites you to defer until the content asks for it.

6. Public vs. Admin: the Bypass and the Deliberate 404

The same resource serves two audiences, and the difference is a visibility condition added only for whoever isn’t logged in. `SimonePizziWebSite` does it with a conditional `AND` in the same endpoint; `SitoRuntime` with two queries in two different files (reading in `news.php`, `management` in `admin.php`); `DISINTELLIGENZA` with an `if (! $isAdmin)` that adds the `WHERE`. Same idea, three structures.

On a single unpublished article, though, there’s a security detail worth a rule of its own: you respond **404**, not **403**.

```
// SPW articles.php?slug=... - the deliberate 404: don't confirm a draft exi
↳ sts
$article = $stmt->fetch();
if ($article) {
    $is_admin    = isset($_SESSION['user_id']);
    $is_published = $article['status'] === 'published' &&
        (empty($article['published_at']) || strtotime($article['
↳ published_at']) <= $ita_now_time);
    if (!$is_admin && !$is_published) {
        http_response_code(404); // not 403: a 403 wou
↳ ld confirm the draft exists
        echo json_encode(['error' => 'Articolo non trovato']);
        exit;
    }
    echo json_encode($article); // the admin sees eve
↳ rything
}
```

For the dashboard list, which has to show drafts too, you need a double check: the session **and** an explicit parameter.

```
// the ?admin=true parameter alone would be bypassable: always tie it to the
↳ session
$is_admin_dashboard = isset($_SESSION['user_id']) && ($_GET['admin'] ?? '')
↳ === 'true';
if (!$is_admin_dashboard) {
    $conditions[] = "status = 'published'";
    $conditions[] = "(published_at IS NULL OR published_at = '' OR published
```

```
↵ _at <= ?)";  
  $params[] = $ita_now_str;  
}
```

The fetch by `id` (not by `slug`) serves only the dashboard editor, and requires a mandatory `Auth::check()`, because it has to load even never-published drafts into the form. It's the third way to reach the same content, each with its own level of gate.

7. Editorial Workflow and Integrity

Three precautions hold the editorial experience together:

- **Auto-slug only on creation:** the slug is generated when the article is born, not on every title change, so as not to break links that are already published and indexed.
- **An editor that cleans itself between one piece of content and the next:** moving from editing one article to another, the editor component has to be unmounted and remounted (`key={item.id}`), so its internal buffers don't drag text from one piece of content into the next.
- **Cover preview:** the form shows a thumbnail of the selected image, with immediate removal before saving.

On the dashboard side, the management table minds its proportions (the title no wider than 45% of the grid, a category column with colored badges for quick scanning, dates and dynamic state clearly visible, the actions condensed into icons at the end of the row): SimonePizziWebSite details, useful as a UX reference more than as a prescription.

IMPORTANT The Canon

- Draft/scheduled/published states; the `published_at <= NOW` comparison must be done in the same format and timezone (or delegated to `NOW()`).
- For non-public content, respond 404, not 403: don't confirm that a draft exists.
- Extend a contract (e.g., `{data, total}`) instead of versioning it, but keep it consistent across endpoints.
- JSON cache with a TTL for heavy listings, invalidated on `save/delete`.

✱
✱
✱

Next Chapter: Security & Auth. Session management, CSRF, roles, and anti-abuse protection.

CHAPTER 10: Security & Auth

Security is the only lens in this book where the three sites don't scale in step with how engineered their backend is. In every other chapter an intuitive rule holds: the more a site has grown, the richer its infrastructure. Not here. The same thin-stack skeleton (native PHP, cookie-based sessions, Apache rules in the `.htaccess` files) holds up three descending rungs of defense, but the top rung doesn't belong to the site with the most sophisticated backend.

SimonePizziWebSite (SPW) has the most mature perimeter: `Secure` cookies, anti session-fixation, global session invalidation, full password recovery, IP anti-spoofing. SitoRuntime (SR), the most engineered site of the trio, the one with the scalability scars, is only halfway there: it has roles and CSRF tokens, but the cookie travels without `Secure`, it doesn't regenerate the session at login, and its rate limit can be sidestepped with a header. DISINTELLIGENZA (DIS) is base-rung authentication: no CSRF, no rate limit, cookies at PHP's default values. And yet it's the only one of the three to bring two security ideas of its own that are genuinely worth something: anti-fraud defense for a public action (the festival vote) and an automatic backup before a destructive operation.

It's the sharpest demonstration of the thesis that runs through the whole book: more engineered doesn't mean more secure, and more secure doesn't mean more complete on every single point. The right way to read each defense is as a scale of subtraction: what's left, and what breaks, when you remove a flag from the cookie, a token from a request, a counter from a login endpoint.

NOTE

A note on method. Every code excerpt in this chapter comes from the real state of the three sites, cited as `file:line`. When the chapter says "the Model recommends," it's prescribing; when it says "SPW does" or "SR does" or "DIS does," it's photographing the code in production. The two don't always line up, and it's precisely in that gap that you learn.

*
**

1. The Common Perimeter: Security by Hand

Before the divergences, it's worth pinning down what the three sites share. Under the three implementations there's the same object, with five recurring traits.

1) No authentication library. No Passport, no JWT package, no session framework. Everything rests on native PHP primitives: `session_*`, `password_hash/password_verify`, `random_bytes`, `hash_equals`, and on the Apache rules in the `.htaccess` files. Auth is built by

hand, and it's the founding choice that makes every difference between the sites a visible choice, not a configuration buried inside a vendor.

2) Cookie session plus `password_verify`: the identical base. All three open a native PHP session, look the user up by `username`, verify the password against a `PASSWORD_DEFAULT` hash, and populate `$_SESSION`. The system never knows the password in cleartext:

```
// The scheme common to the three sites: no secret in cleartext
$hash = password_hash($password, PASSWORD_DEFAULT); // at creation
// ...
if (password_verify($input, $user['password_hash'])) { /* login ok */ }
```

3) The gate is a handful of lines at the top of the endpoint. There's no middleware and no router: protecting an endpoint is a few lines at the start of the mutating branch, demanding a valid session before touching the data. It's the thin-stack version of middleware. But how much that handful of lines does (session only? plus CSRF? plus role? plus invalidation?) is the first major axis of divergence, and we'll see it in §2.

4) "JSON-first" defense even on security errors. A denied access doesn't produce an Apache error page: you set the semantically correct HTTP code (401/403/429) and respond with a `{status|success, message|error}` object. The frontend reacts by code.

5) Server-level hardening via `.htaccess`. All of them have at least one `.htaccess` that sets security headers and denies access to sensitive files. It's the second layer, outside the application, and here too the coverage ranges from complete (forced HTTPS, HSTS, CSP, PHP off) to minimal (a deny on two extensions).

To these is added one shared negative trait: none of the three keeps an access audit log, and in two cases out of three the login's brute-force counter doesn't even live in the database. The perimeter is thin by construction. The chapter's question is how thin you can go before something breaks.

**

2. The Gate: Single Middleware, Composable Gate, Inline Gate

Protecting an endpoint means deciding, at the top of the code, whether the caller has the right to proceed. The three sites solve the same problem with three different architectures, and it's the first point where the scale becomes visible.

2.1 SPW: the Single Gate `Auth::check()`

SPW concentrates everything in a class included by every protected endpoint. A single line, `Auth::check()`, does three things in sequence: it demands a valid session, verifies anti-CSRF on the mutating methods, and checks invalidation via `session_version`.

```
// public/api/articles.php:238-239 - the typical consumer
elseif ($method === 'POST') {
    Auth::check(); // session + CSRF + session_version, all here
    // ... from here on you can write
```

The advantage is that coverage is automatic: every new endpoint that includes `auth_helper.php` and calls `Auth::check()` inherits all the defenses together. There's no way to remember the session and forget the CSRF.

2.2 SR: the Composable Function Gate

SR breaks the same protection into separate bricks, authorization on one side and anti-CSRF on the other, which each endpoint composes by hand in the right order:

```
// public/api/auth_utils.php – the bricks
function isLoggedIn() { return isset($_SESSION['user_id']); }
function isAdmin()    { return isLoggedIn() && ($_SESSION['role'] ?? '') ===
↳ 'admin'; }

// public/api/upload.php:8-13 – the consumer composes them
if (!isLoggedIn()) { http_response_code(401); echo json_encode([...]); exit;
↳ }
// ...
validateCsrf();
```

It's more flexible, and the role gate (`isAdmin()`) comes for free: you get that admin/editor distinction SPW doesn't have. But it's also easier to forget, because protection depends on the discipline of each individual endpoint.

2.3 DIS: the Raw Inline Gate

DIS takes the same scheme to the extreme. No file of brick functions: the gate is rebuilt by hand in every branch as a bare `isset()`.

```
// public/api/reset_votes.php:11 – the whole gate, inline
if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    http_response_code(401); die(...);
}
```

WARNING

Middleware or discipline? The gate you forget The composable gate and the inline gate carry a structural risk the single gate doesn't: security depends on the order of the branches and on the memory of whoever writes them. In DIS the `participants.php?update_status` and `update_round` branches are protected by `isset($_SESSION['user_id'])` alone, not by `isAdmin()`. The result: an editor, not an administrator, can approve or reject participants, send the emails, and move the festival rounds. The same crack shows up in SR, where `list_users` and `create_user` sit before the 401 block and protect themselves only with the local role check. It works as long as every branch is written with discipline. But a single gate (`Auth::check()`) can't be forgotten on an endpoint: either you include it, or the endpoint doesn't start. That's the real difference between middleware and convention. The consequences for the festival show up in Chapter 18.

*
**

3. CSRF: Three Rungs of Defense

Cross-Site Request Forgery is the problem of preventing a third-party site from sending mutating requests by exploiting the session cookie that the user's browser attaches automatically. It's the load-bearing defense of this chapter, and the three sites solve it at three distinct levels.

3.1 SPW: origin/Referer Check (Server-Side, Zero Handshake)

SPW doesn't use tokens: inside `Auth::check()`, on the non-safe methods, it compares the host of `Origin/Referer` with that of `SITE_URL`.

```
// public/api/auth_helper.php:21-37 (excerpt)
$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
if (!in_array($method, ['GET', 'HEAD', 'OPTIONS'], true)) {
    $source = $_SERVER['HTTP_ORIGIN'] ?? $_SERVER['HTTP_REFERER'] ?? '';
    if ($source !== '') {
        $sourceHost = parse_url($source, PHP_URL_HOST);
        $allowedHost = parse_url(SITE_URL, PHP_URL_HOST);
        $isLocalDev = in_array($sourceHost, ['localhost', '127.0.0.1'], true);

        if ($sourceHost !== $allowedHost && !$isLocalDev) {
            http_response_code(403);
            echo json_encode(['status' => 'error', 'message' => 'Origine del
            la richiesta non valida']);
            exit;
        }
    }
}
```

It requires no handshake with the client, because the browser sends `Origin/Referer` on its own. And living inside `Auth::check()`, it automatically covers every endpoint that includes the guard.

3.2 SR: the Synchronized X-CSRF-Token

SR adopts the by-the-book pattern: a random token per session, returned to the client at login, sent back by the client in a header and validated with `hash_equals` (a timing-safe comparison).

```
// public/api/auth_utils.php:20-35
function generateCsrfToken(): string {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function validateCsrf(): void {
    $incoming = $_SERVER['HTTP_X_CSRF_TOKEN'] ?? '';
    $stored = $_SESSION['csrf_token'] ?? '';
    if (!$stored || !hash_equals($stored, $incoming)) {
        http_response_code(403);
        echo json_encode(['success' => false, 'error' => 'Token di sicurezza
        non valido. Ricarica la pagina.']);
        exit;
    }
}
```

3.3 DIS: Nothing

In DIS the mutations are protected by the session cookie alone. A grep across all of `public/api/` finds no CSRF token and no `Origin/Referer` check: the defense is absent.

WARNING

CSRF in the thin stack: Origin/Referer, synchronized token, or nothing There's a counterintuitive subtext in these three rungs. The most "by-the-book" solution, SR's synchronized token, is also the easiest to weaken, because it depends on per-branch discipline: one mutating endpoint that forgets `validateCsrf()` and the hole

is open. The simplest solution, SPW's `Origin/Referer` check, covers more precisely because it's centralized inside the single gate. More code doesn't mean more coverage. It's the book's underlying thesis applied to CSRF.

*
**

4. The Session Cookie and Anti Session-Fixation

NOTE

A common mistake about session cookies. You'd be tempted to prescribe a cookie with `cookie_secure = 1` and `cookie_samesite = 'Lax'`. Looking at the real code, two things need correcting: the right `SameSite`, and the one the sites actually use, is `strict`, not `Lax` (both SPW and SR); and only SPW actually sets all three flags.

The flags have to be set before `session_start()`, otherwise they have no effect on the current cookie. This is where the scale is clearest.

SPW sets the three flags in a shared file:

```
// public/api/auth_helper.php:7-11
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 1);
ini_set('session.cookie_samesite', 'Strict');
session_start();
```

SR omits `Secure`:

```
// public/api/auth_utils.php:4-9
ini_set('session.cookie_httponly', '1');
ini_set('session.cookie_samesite', 'Strict');
session_start();           // NB: cookie_secure NOT set
```

DIS sets no flags at all: the cookie's behavior depends entirely on the hosting's `php.ini`. An implicit, silent dependency.

On top of this there's anti session-fixation, that is, regenerating the session ID right after login to invalidate an ID an attacker may have planted earlier. Only SPW does it:

```
// public/api/auth.php:186-188 (excerpt of the success branch)
if ($user && password_verify($password, $user['password_hash'])) {
    session_regenerate_id(true);    // anti session-fixation: one line, oft
    ↪ en missing
        $_SESSION['user_id'] = $user['id'];
        // ...
}
```

SR and DIS don't regenerate: a session ID fixed before login stays valid afterward.

WARNING

The cookie flags, and why HSTS isn't the HTTPS redirect Dropping `Secure` isn't a cosmetic detail. Without `Secure`, the session cookie can also travel over plain HTTP. You might think HSTS protects you anyway, but HSTS only protects after the first successful HTTPS visit. SR applies HSTS but doesn't force the 301 redirect to HTTPS: there's a real window of exposure over HTTP on the first access. SPW

closes the window on both sides, with `cookie_secure=1` and with `RewriteCond %{HTTPS} !on → 301`. The lesson: Secure on the cookie and a forced HTTPS redirect are two distinct defenses; HSTS doesn't replace either of them. A historical aside (SPW): until v1.18 the flags were only in `auth.php`, so a cookie issued by another endpoint was born weak. v1.19.0 moved the `ini_set` calls into the shared file `auth_helper.php`. The session's security config belongs in the file everyone includes, not in the login endpoint.

**

5. check_auth and the Session State

The frontend must never store the password or the login state in `localStorage`: it asks the server whether the session is still valid.

```
// check_auth scheme (SPW/DIS)
if (isset($_SESSION['user_id'])) {
    echo json_encode([
        'status' => 'success',
        'user'    => ['username' => $_SESSION['username'], 'role' => $_SESSION[
↵ N['role']]
    ]);
}
```

NOTE

username isn't guaranteed in the session. You'd be tempted to read `$_SESSION['username']` as if it were always present. In SR it isn't: login saves `user_id` and `role` but not `username` (`admin.php:123-124`). The consequence is concrete: `check_auth` returns `username: null`, and the article save falls back to `author = $_SESSION['username'] ?? 'Admin'`. In SPW and DIS the `username` is in the session and the example holds. It isn't a security hole, but a real state inconsistency: the same field isn't guaranteed across all three sites.

**

6. Passwords and Brute-Force Defense

Hashing is the only point where the three sites are identical and all correct: `password_hash($pass, PASSWORD_DEFAULT)` at creation, `password_verify` at verification. The system never knows the password in cleartext, and the algorithm can evolve (from `bcrypt` to `argon2`) without touching a line of code.

NOTE

Brute-force defense isn't a `sleep(1)`. You'd be tempted to reduce brute-force defense to a `sleep(1)`, but that's an incomplete solution calibrated on SR alone. The `sleep(1)` is barely a stopgap; the real defense is lockout, and above all what matters is which IP you measure it from.

The question isn't "how do I slow down the attempts" but where the counter lives. Three answers. SPW keeps it in a DB table `login_attempts`: after 5 failed attempts from an IP within 15 minutes the 429 kicks in, and on a successful login the counter resets.

```
// public/api/auth.php:177-211 (excerpt)
if ($attempts >= 5) {
    http_response_code(429);
    echo json_encode(['status' => 'error', 'message' => 'Too many failed log
↳ in attempts. Try again in 15 minutes.']);
    exit;
}
// ... on failure:
$pdo->prepare("INSERT INTO login_attempts (ip_address) VALUES (?)")->execute
↳ ([ $ip_address]);
```

SR keeps it on file: one JSON per IP in `.cache/ratelimit/<md5(ip)>.json`, a 900-second window, a threshold of 5, plus a `sleep(1)` on failure. No DB table, consistent with the MySQL schema that doesn't contain `login_attempts`. DIS doesn't keep it anywhere: the login can be hammered as much as you like.

TIP

The brute-force counter: file, DB, or absence There's no absolutely right home. SPW's DB table is transactional and lends itself to reuse (the same `login_attempts` also serves recovery). SR's file avoids burdening the database but lives outside the schema, so it doesn't show up in a dump or a migration: it's invisible until you go looking for it in the filesystem. DIS's absence is consistent with a festival site where there's a single admin and the friction on an internal login is low, but it remains an absence, not a documented choice.

The subtlest point, though, is which variable you read the IP from. It's the anchor box of this chapter.

WARNING

Trusting the IP: the rate limit that doesn't limit (*anchor box, referenced in Chapter 18 and Chapter 20*) A "5 attempts per IP" lockout is worth exactly as much as your ability to know what the IP is. And here the three sites diverge in an instructive way. SR trusts the wrong header. It takes the IP from `X-Forwarded-For`, first and without validation:

```
// public/api/admin.php:106-107
$ip = $_SERVER['HTTP_X_FORWARDED_FOR'] ?? $_SERVER['REMOTE_ADDR'] ?? 'unknown
↳ n';
$ip = trim(explode(',', $ip)[0]); // ← the attacker controls this header
```

That header is written by the client. An attacker varies it on every request and the counter never increments: the 5/15min lockout is sidestepped.

SPW trusts the TCP IP, and the header only behind an internal proxy. The `getClientIp()` function uses `REMOTE_ADDR` if it's already public, and accepts `X-Forwarded-For` (validated `NO_PRIV_RANGE|NO_RES_RANGE`) only when `REMOTE_ADDR` is private, that is, only behind a trusted proxy. No spoofing.

DIS uses `raw REMOTE_ADDR`, and here it's a virtue. For the anti-double-vote barrier (§12), `REMOTE_ADDR` can't be forged at the TCP level, so the barrier holds. The downside is NAT: behind a proxy or a shared network, many users collapse onto the same IP. The lesson isn't "always use `REMOTE_ADDR`." It's that the right IP depends on the abuse model. A login to be brute-forced (you want the attacker unable to change their own identity, so you distrust `X-Forwarded-For`) is the opposite problem from a public vote not to be duplicated; and yet the conclusion is the same: the client-controlled header isn't trustworthy. That same `raw REMOTE_ADDR` is a hole in one context and a defense in the other.

**

7. session_version: Invalidating Sessions at Zero Cost

There's a classic problem with session-based auth: how do you log out all of a user's sessions, for example after a password reset, if you don't keep a server-side store of the active sessions? Only SPW solves it, with a zero-cost trick: a `session_version` integer in `users`, copied into `$_SESSION` at login and compared on every protected request.

```
// public/api/auth_helper.php:51-57 - inside Auth::check()
try {
    $stmt = $pdo->prepare("SELECT session_version FROM users WHERE id = ?");
    $stmt->execute([$_SESSION['user_id']]);
    $row = $stmt->fetch();
    if (!$row || (int)$row['session_version'] !== (int)$_SESSION['session_v
↵ ersion'] ?? -1)) {
        session_destroy();
        http_response_code(401);
        echo json_encode(['status' => 'error', 'message' => 'Sessione scadut
↵ a. Effettua nuovamente il login.']);
        exit;
    }
} catch (PDOException $e) {
    error_log('session_version check failed: ' . $e->getMessage());
    http_response_code(401); // ← fail-closed: on a DB error, DENY
    echo json_encode(['status' => 'error', 'message' => 'Sessione non verifi
↵ cabile. Riprova.']);
    exit;
}
```

You just increment `session_version` by one (the password reset does it, §8) and all the sessions with the old number become invalid on the first check. Logout-everywhere without any session store.

TIP

Fail-closed, not fail-open The detail that sets a well-built defense apart is the `catch` branch. If the `session_version` check raises a `PDOException` (DB unreachable), SPW denies access (401), it doesn't grant it. The rule: when a security check can't be completed, the default outcome must be "denied." SR and DIS don't have this mechanism, and in SR a password change doesn't log out the other sessions; in DIS, not even that.

**

8. Recovery and Password Reset Done Right

This is a whole section the earlier chapter didn't have, because only SPW implements self-service password recovery. SR and DIS only have authenticated `change_password`, where you have to be inside already: a forgotten password means no way back in.

SPW's flow has four precautions worth looking at one by one.

The first is the single-use random token, with an expiry.

```
// public/api/auth.php:98-106
$token = bin2hex(random_bytes(32)); // 32 random bytes
$expires = date('Y-m-d H:i:s', strtotime('+1 hour'));
$pdo->prepare("DELETE FROM password_resets WHERE user_id = ?")->execute([$us
↵ er['id']]); // invalidate the previous ones
$pdo->prepare("INSERT INTO password_resets (user_id, token, expires_at) VALU
↵ ES (?, ?, ?)")
->execute([$user['id'], $token, $expires]);
```

The second is the link built from a canonical URL, not from `HTTP_HOST`. It's the defense against *password-reset poisoning*: if the link in the email were built from `HTTP_HOST` (controllable with a forged `Host` header), the attacker could have the victim delivered a link pointing to their own domain and intercept the token.

```
// public/api/auth.php:227-230
$link = SITE_URL . "/admin/reset-password/{$token}"; // SITE_URL hardcoded
↵ , never HTTP_HOST
```

The third is being enumeration-safe. The recovery request always responds with the same generic message ("if the account exists, you'll receive an email"), whether the user exists or not, so the form can't be used to discover which emails are registered. The same `login_attempts` counter is reused here with key namespacing (`'rec:' + sha256(IP)`), to limit recovery abuse too.

The fourth is that the reset invalidates the sessions. Applying the new password, `session_version` is incremented, and by what we saw in §7 every open session falls:

```
// public/api/auth.php:127-149 (excerpt)
if (strlen($newPassword) < 12) { /* 400: "almeno 12 caratteri" */ }
// ... verify the token hasn't expired ...
$hash = password_hash($newPassword, PASSWORD_DEFAULT);
$pdo->prepare("UPDATE users SET password_hash = ?, session_version = session
↵ _version + 1 WHERE id = ?")
->execute([$hash, $reset['user_id']]);
$pdo->prepare("DELETE FROM password_resets WHERE token = ?")->execute([$toke
↵ n]); // single-use token
```

NOTE

The schema that isn't there. The `password_resets` table is used everywhere in `auth.php`, but no file creates it: the migration script that generated it was deleted after running in production. The same goes for the `session_version` column, added on the fly with an idempotent `ALTER TABLE` inside `auth.php`. It's a traceability debt: the real schema can't be reconstructed from a single file. The theme of throwaway migrations returns in Chapter 15 (Database Evolution).

**

9. Default Credentials: Random, Hardcoded, Omitted

The first administrator is an often-underestimated security problem. The three sites solve it in three ways, and here a point anticipated in Chapter 5 (Backend Logic) closes.

WARNING

Seeding the admin: what NOT to do SPW generates it randomly and prints it once. The first admin's password is random and shown only once during setup. It's the correct choice. SR hardcodes it to `runtime2026`. The maintenance file `fix_users_table.php` recreates the admin user with `password_hash('runtime2026', ...)` if the table is empty, and that password is committed in the repo. The real mitigation is the `.htaccess`, which denies HTTP access to `fix_users_table.php` (§10); but there's no flow that forces changing the default, and if no one uses `change_password` the admin account stays guessable. DIS omits it entirely. There's no seeding: `init_db.php` elided the admin creation, and `users.php` can create users only if you're already admin. The result: no guessable default (the upside), but the first admin lives only in the `.sqlite` file and can't be reconstructed from the repo (the downside, with the classic chicken-and-egg problem on the first clean deploy). Three positions on the same scale: the random one is the most secure, the omitted one is the most spartan but not insecure, the hardcoded one is the only one that's truly dangerous. And not because the password exists, but because nothing forces you to change it.

**

10. Protecting the File-Based Database and the Maintenance Scripts

When the database is a file (`.sqlite`), or when powerful scripts live in the docroot (migrations, fixes, resets), the `.htaccess` becomes part of the security perimeter.

The first task is denying direct access to the file-based DB. DIS keeps the `.sqlite` in a `.data/` folder generated at runtime and protected:

```
# DIS: deny of the data and backup files
<FilesMatch "\.(sqlite|bak)$">
    Require all denied
</FilesMatch>
```

The base rule is the one the chapter already taught (files outside the public root, or `Require all denied` plus unpredictable names). What's new is that DIS pairs it with a `.data/` folder created at runtime by the bootstrap, which we discuss in Chapter 5.

The second task is denying the maintenance scripts, and this is where SR has the most interesting `.htaccess` of the three, because it blocks a whole family of files by prefix, before PHP even sees them:

```
# public/.htaccess:80-81 (SR)
<FilesMatch "^(debug_|test_|emergency_|migrate_|fix_|init_|rebuild_|setup_|o
  ptimize_)">
    Order allow,deny
    Deny from all
```



```
</FilesMatch>
# also blocks the typical scanner URLs, before PHP
RewriteRule ^(wp-|wordpress|xmlrpc|\.env|\.git|cgi-bin|phpmyadmin) - [R=404,
↳ L]
```

It's this deny that makes the `fix_users_table.php` with the `runtime2026` password from §9 non-executable via the browser.

The third task concerns uploads. If an attacker manages to upload a `.php`, execution has to be turned off at the folder level, as SPW does:

```
# public/uploads/.htaccess (SPW)
<IfModule mod_php.c>
  php_flag engine off
</IfModule>
<FilesMatch "\.(php|phtml|php[0-9]|phps|cgi|pl|py|sh)$">
  Require all denied
</FilesMatch>
```

WARNING

The deny that's missing where it's needed most Be careful not to read these rules as a uniform checklist: the coverage is uneven. SR has the most sophisticated by-prefix deny of the three, but it doesn't have an `uploads/.htaccess` that turns PHP off in the upload folder. DIS protects the `.sqlite` and the `.bak` files, but its `update_db_*` scripts don't fall under any deny pattern and stay reachable. Each site bolted the door it had seen, leaving another one open. Protecting the public upload, and the abuse chain that grows out of it in DIS, is the heart of Chapter 7.

*
**

11. Errors That Speak to the User, Opaque to the Attacker

An unhandled PHP error that ends up in the output can reveal paths, queries, database structure (the so-called *path/information disclosure*). The Model's standard: semantically correct HTTP codes, generic messages to the client, technical detail only in the log.

```
// SPW: the detail goes in the log, not to the client
} catch (PDOException $e) {
    error_log('auth: ' . $e->getMessage());    // only here
    http_response_code(500);
    echo json_encode(['status' => 'error', 'message' => 'Errore interno. Rip
↳ rova.']);
}
```

WARNING

Don't send the exception back to the client DIS does the opposite: `auth.php`, `users.php`, and `participants.php` send `$e->getMessage()` straight back to the client.

```
// DIS auth.php:48 (anti-pattern)
} catch (PDOException $e) {
    echo json_encode(['status' => 'error', 'message' => $e->getMessage()]);
↳ // leaks DB details
}
```

It's information disclosure for free: an attacker reads table and column names from the error messages. The rule “speak to the user, opaque to the attacker” costs nothing; you just have to not skip it.

*
**

12. Destructive and Public Actions

The last front is the slipperiest: what happens when an action is protected by login but also destructive, or is public (no login) yet still has to defend itself from abuse.

12.1 The One-Click Reset: Why “Gated” Isn’t Enough

In DIS the `reset_system.php` and `reset_votes.php` endpoints require admin login but have no CSRF. A cross-site POST to `reset_system.php`, triggered while the admin is logged in on another tab, deletes all the participants, the votes, and the audio. The only mitigation is the cookie's default `SameSite` (not set, as we saw in §4), which depends on the PHP version and doesn't cover every case.

WARNING

Why a gated action still needs CSRF “It's protected by login” and “it's protected by CSRF” are different guarantees. Login says who you are; CSRF says whether it was actually you who wanted it. A destructive action needs both: without CSRF, it's the admin's legitimate session being used against him. And a JavaScript `confirm` (“are you sure?”) doesn't replace CSRF, because it runs on the client and the attacker skips it.

12.2 The Just-in-Time Backup: the Defense DIS Has and SR Doesn't

And here comes the surprise that breaks the scale. The same DIS that has no CSRF on the reset does something the incident flagship, SR, doesn't: it copies the database before touching it.

```
// public/api/reset_votes.php:18-21 - backup before the destructive op
$dbPath = __DIR__ . '/.data/database.sqlite';
if (file_exists($dbPath)) {
    copy($dbPath, __DIR__ . '/.data/backup_votes_' . date('Ymd_His') . '.sql
↳ ite.bak');
}
$pdo->exec("DELETE FROM votes");
```

This is exactly the prevention SR lacked (in Chapter 15 we call it “cure without prevention”): the site weakest on identity is the only one to do the just-in-time backup right. More secure doesn't mean more complete on every point.

12.3 Anti-Fraud for a Public Action, and Privacy

The festival vote is an action that isn't authenticated: it's the public that votes. DIS defends it in layers, with a single real barrier, the `REMOTE_ADDR/24h` one seen in the IP box of §6. The full treatment of vote anti-fraud (master switch, cosmetic cookie, denormal-

ized `vote_count`) lives in Chapter 18; here only one cross-cutting point matters: how you preserve the identity of whoever performs a public action.

TIP

Voting anonymously: a hash instead of a cleartext IP (*bridge to Chapter 20*) DIS saves `ip_address` and `user_agent` in cleartext in the `votes` table. It works for anti-double-vote, but it stores personal data without need. SPW, for the anonymous reactions (Chapter 20), gets the same anti-fraud guarantee by storing only `voter_hash = SHA256(IP + UA)`: the comparison holds, but the personal data is never written. Two opposite privacy postures on the same functional need. Note, though, that SPW's hash isn't salted and uses low-entropy input (IP and UA): it's anti-collision, not strong cryptographic anonymity, because a candidate IP can still be verified by brute force. The full comparison between the two philosophies (write-time vs. render-time sanitization, and anonymous identity) is in Chapter 20.

On the anti-abuse side, there's one last detail: the same SPW `login_attempts` is reused as a two-layer rate limit for the reactions (per-hash 20/min plus IP-only 30/min): the first layer is sidestepped by rotating the User-Agent, the second is the real bulwark. Here too the detail is in Chapter 20.

*
**

13. The Client Side and a Lesson About Bots

The Admin area in React is protected by an `AdminLayout` that, in its main `useEffect`, queries `check_auth`: if the server responds 401, the client destroys the local state and redirects to login, so it doesn't flash sensitive content. The sidebar and the routes are generated from the `role` received from the server (admin vs. editor in SR and DIS).

NOTE

The client isn't the defense. Client-side logout and the 401 check are user experience, not security: they hide what you shouldn't see, but it's the server-side gate (§2) that keeps you from touching it. The difference between "logout on the client" and real invalidation via `session_version` (§7) is exactly this. The full architecture of the admin panel (the three dashboards, the three guard architectures, the placement of the backups) is in Chapter 14.

There's finally a lesson that comes from a real incident and stays relevant here, even though the full case has migrated elsewhere. In February 2026 Runtime Radio was swamped by bots that mimicked the social crawlers (Telegram, Facebook, X) to hit the PHP SEO entry point, which queried the database on every request. The bots' User-Agent is forgeable.

WARNING

The User-Agent isn't a gatekeeper You never make security decisions based on the User-Agent, because it's forged in a header field. You can use it to optimize (serving

a cache to recognized bots), never as an access barrier. The full account of the bot-DDoS attack and the solution (a precompiled static cache, a bot path separate from the human one) is in Chapter 11, because the vector is precisely the SEO entry point. Here only the maxim remains.

**

In Summary

Security is the lens that disproves the intuition that “bigger equals sturdier.” SPW, which isn’t the most engineered site, has the most mature perimeter. SR, the richest, leaves three real holes: a cookie without `Secure`, no anti-fixation, a rate limit that can be sidestepped. DIS, the base rung, still brings two ideas the others lack: robust public anti-fraud and the pre-destructive backup. Read as a scale of subtraction, every defense teaches two things at once: what it does, and what breaks when you remove it.

IMPORTANT The Canon

- Sessions with `HttpOnly` + `SameSite=Strict` + `Secure` cookies over HTTPS; passwords with `password_hash()`.
- A CSRF token on all mutations: a `confirm()` isn’t a CSRF defense.
- Authorization by **role** (`isAdmin`), not just by login (`isLoggedIn`).
- Brute-force lockout measured on a trustworthy IP, not on client-controlled headers (`X-Forwarded-For`).
- `session_version` to invalidate sessions on a password change; no default credentials in the code.

**

Next Chapter: SEO Pre-rendering with a PHP Entry Point. The invisible SEO engine that turns a SPA into an indexable site, and the vector of the February 2026 bot-DDoS attack.

CHAPTER 11: SEO Pre-rendering with a PHP Entry Point

A Single Page Application has a birth defect when it comes to search engines. Google’s bot, or the Telegram crawler that generates a link preview, receives from the server an almost-empty `index.html`: a `<div id="root"></div>` and a JavaScript bundle. The real content only arrives after that JavaScript runs, and many crawlers don’t execute it. The result is a page that, to the bot, has no sensible title, no description, no preview image, and no text to index.

The Model solves this without an SSR framework, without Next.js, without Node. It puts a PHP file in front of the SPA, has it query the database, and has it inject the correct meta tags in the right places, before the HTML reaches the bot. It’s the thin-stack version of server-side rendering: a few hundred lines of PHP and zero new infrastructure.

But this chapter has three threads to hold together. The first is a story of attempts: the winning pattern isn’t the first one that was tried, and along the way the two flagships discarded a solution that, at first glance, looks the most natural. The second is a three-rung scale: the two flagships do full Dynamic Rendering, DIS stops at a social-preview proxy, and the most capable rung carries three debts with it. The third is the security thread opened in Chapter 8: the prerender is the `content` emitter where the XSS hole of the “four emitters” is **still live**.

NOTE

A temptation to avoid: Static Prerendering (SSG). For deep indexing you might be tempted to add, on top of meta injection alone, a **Static Prerendering** step at build time (something like `vite-plugin-prerender`). But that’s **exactly the Puppeteer-based SSG that SimonePizziWebSite tried and then abandoned** (§1): today in its repo it’s nothing but dead code, complete with a post-mortem. The solution the flagships actually adopted is **Dynamic Rendering**. Watch out, too, for a detail that misleads: a setup with a `.sqlite` file and a direct connection isn’t SPW’s engine, which runs on MySQL, but DIS’s proxy.

*
**

1. The Problem, and Three Attempts to Solve It

The story of prerendering in SPW is a three-stage path, and telling it explains how the final pattern came to be.

The first stage (v1.7.3) injected only title, description, and Open Graph into the `<head>`. That was enough for social previews, but not for Google: the crawler found the correct meta tags and a still-empty `<body>`, and indexed very few pages.

The second stage was **SSG with Puppeteer**: a Node script that, at build time, prerendered every route into a static HTML file on disk. Technically it worked. But it was conceptually wrong for a CMS, and the post-mortem says so plainly: it broke the “edit online, publish” flow (you needed a local build and an FTP upload for every article), it froze the data at build time, and it dragged a chain of complications behind it (an anti-CORS bridge, a `dist-static` buffer, the render events). It was abandoned.

The third stage, the current one, is **Dynamic Rendering**: no build, no static files. A single `index.php`, in real time, decides what to serve depending on who’s asking.

TIP

The code that’s left after the strategy changes Of the abandoned SSG, what remains in SPW is the fossil imprint: `prerender.php` is deprecated (it only returns a warning), `prerender.js` and `prerender-routes.js` are dead code (the `postbuild` runs `clean-dist.js`, not them), and an `IS_PRERENDERING` constant guards branches that no one defines anymore. SR, which came later, went straight to Dynamic Rendering and has no fossils. It’s a small archaeological reminder: when you change strategy, the code of the old strategy rarely disappears: it sits there, inert, until someone trusts the change enough to delete it.

*
**

2. Dynamic Rendering: Everything in a Single `index.php`

The winning pattern concentrates everything in `public/index.php`, which `.htaccess` places ahead of `index.html` and to which it routes every virtual React Router URL. For each request, in real time, the file does four things: it deduces from the path what type of page it is, queries the database for the real data, and then chooses between two paths depending on who’s asking.

The choice runs through `isCrawler()`, which sniffs the User-Agent:

```
// public/index.php:46-86 (SPW) – the crawler-vs-human fork
function isCrawler(): bool {
    $ua = $_SERVER['HTTP_USER_AGENT'] ?? '';
    if (empty($ua)) return false;
    $crawlers = ['Googlebot', 'Bingbot', 'facebookexternalhit', 'Twitterbot'
    ↵ ,
        'TelegramBot', 'WhatsApp', 'Discordbot', /* ... */];
    foreach ($crawlers as $bot) { if (stripos($ua, $bot) !== false) return t
    ↵ rue; }
    return false;
}
$isCrawler = isCrawler();
if ($isCrawler && $pageType !== 'admin') { /* full server-side HTML for the
    ↵ bot */ }
else { /* Vite's index.html + meta injected into the head; React builds the
    ↵ body */ }
```

The real user gets Vite's `index.html` with the meta tags and JSON-LD injected into the `<head>`, and React builds the `<body>` as always. The crawler, instead, gets already-complete HTML, with the title, the meta tags, and the **article body** written directly into the `<body>`, so the bot indexes without executing a single line of JavaScript. The file explicitly claims this isn't cloaking: the content served to the bot is the same one the user sees after React hydrates.

For this to work, the PHP has to know the routes as well as React Router knows them, and here lies the method's first cost:

```
// public/index.php:133-154 (SPW) - server-side routing that mirrors React R
↪ outer
if (count($uri_parts) === 0)           $pageType = 'homepage';
elseif ($uri_parts[0] === 'admin')     $pageType = 'admin';      // no SEO
↪ for the admin area
elseif ($uri_parts[0] === 'tutti-i-progetti') $pageType = 'projects';
elseif (count($uri_parts) === 2) { $pageType = 'article'; $catSlug = $uri_p
↪ arts[0]; $slug = $uri_parts[1]; }
elseif (count($uri_parts) === 1) { $pageType = 'category'; $catSlug = $uri_p
↪ arts[0]; }
```

WARNING

The price of not having an SSR framework: the double truth of the routes The routes now live in two places: in `App.tsx` for React, and in `index.php` for the bots. Adding a public page means touching both. A new route that the PHP doesn't know about falls into the default branch, and the bot receives the wrong meta tags with no one noticing. It's the compromise of hand-rolled Dynamic Rendering: no infrastructure, but a map you have to keep aligned by hand in two different languages.

The injection itself is a surgical operation on the HTML compiled by Vite: you remove the `<title>` generated by the build and insert the SEO block before `</head>`.

```
// public/index.php:576-614 (SPW) - meta injection + removal of Vite's title
$seoInjection = '<title>' . $metaTitle . '</title>'
    . '<meta name="description" content="' . esc($metaDesc) . '" />'
    . '<link rel="canonical" href="' . esc($canonicalUrl) . '" />';    // + 0
↪ G/Twitter
if ($jsonLd) { $seoInjection .= '<script type="application/ld+json">' . json
↪ _encode($jsonLd) . '</script>'; }
$htmlContent = preg_replace('</title>.*?</title>/s', '', $htmlContent, 1);
↪ // avoid the double title
$htmlContent = str_replace('</head>', $seoInjection . '</head>', $htmlConten
↪ t);
```

Every value coming from the database passes through an `esc()` (that is, `htmlspecialchars`) before it lands in an attribute. Every value, with one exception that is the heart of §4: the article body.

✱
✱✱

3. The Three-Rung Scale: Dynamic Rendering or OG-Proxy

The two flagships do the same thing. SR copied SPW's engine almost to the letter: same `isCrawler()`, same helpers (`esc`, `truncateText`, `absImageUrl`), same injection. The most visible difference is cosmetic (SR derives `baseUrl` from `$_SERVER['HTTP_HOST']`, SPW uses its

canonical `SITE_URL`) and the type of structured data: SPW emits `Article` and `CollectionPage`, SR adds a `RadioStation` suited to its domain. But the setup is the same.

DIS sits on a different rung. Its `index.php` isn't a Dynamic Rendering engine: it's a social-preview proxy, and it says so. It doesn't sniff the `User-Agent`, so it serves everyone the same HTML (no cloaking risk) and injects only the meta tags, leaving the body to React. And it writes those meta tags by passing every value through `htmlspecialchars`:

```
// public/index.php:93-109 (DIS) - escape-safe meta injection, no prerendere
↳ d body
function injectTag($html, $tag, $content, $property = null) {
    if (!$content) return $html;
    if ($tag === 'title')
        return preg_replace('/<title>.*?</title>/s', "<title>".htmlspecialchars
↳ hars($content)."</title>", $html);
    $attr = $property ? "property" : "name";
    return /* inserts */ "<meta '$attr.'='". $tag.'" content='".htmlspecialchars
↳ hars($content)."' />";
}
```

TIP

Full Dynamic Rendering or a lightweight OG-proxy They're two different answers to the same question. Dynamic Rendering (SPW, SR) indexes better on crawlers that don't execute JavaScript, because it serves them the article body too; in exchange it's more complex, maintains a double route map, and (as we'll see) reopens a security hole. The OG-proxy (DIS) is simple and honest: no cloaking, no body to re-emit, only escaped meta tags; in exchange its textual SEO for no-JS bots is weak. There's no absolute winner: a festival site that lives on Telegram social previews does just fine with the lightweight proxy; a content site that wants to rank on Google needs the prerendered body. The choice follows what you have to show, and to whom.

*
**

4. The Prerender Reopens the XSS Hole: the Live Flaw of the Four Emitters

In Chapter 8 we pinned down the picture of the four `content` emitters: the same HTML, saved raw in the database, exits toward the world from four different points, and the sanitization that lives in the React render doesn't cover the other three. The SEO prerender is the point where that hole is **still open**.

To serve the bot an indexable body, the crawler branch re-emits the article's `content`. But it doesn't pass it through `DOMPurify` (which lives only in the React render): it passes it through `strip_tags` with a tag allowlist.

```
// public/index.php:403-405 (SPW) - the body for the crawler: strip_tags all
↳ owlist, NOT DOMPurify
<div>' . strip_tags($article['content'],
    ' <p><br><h2><h3><h4><ul><ol><li><strong><em><a><blockquote><pre><cod
↳ e>' ) . '</div>
```


The difference isn't theoretical. `strip_tags` with an allowlist removes dangerous tags at the element level: `<script>`, `<iframe>`, `<svg>` aren't on the list, so they vanish. But `strip_tags` **doesn't look at the attributes** of the tags it lets through. An `<a href="javascript:...">` or a `<p onmouseover="...">` survives intact. DOMPurify would have removed those two; `strip_tags` doesn't.

And the crawler branch is reachable by anyone, because `isCrawler()` trusts only the User-Agent. All it takes is showing up with User-Agent: Googlebot to receive the body passed through `strip_tags` alone. The surface is narrow (because, as a rule, the victim has to be the one carrying a crawler UA, and bots don't execute JavaScript), but it's a second content-render path that doesn't share the first one's defense.

WARNING

When you copy a pattern, you copy its flaw too SR-C7 is SPW-C7 almost verbatim: same `isCrawler`, same helpers, **the exact same** `strip_tags` with the same allowlist. In copying the signature Dynamic Rendering pattern, SR copied its hole as well. And in copying it, it lost a piece (the visibility rule, §5) while introducing a new bug. DIS, which doesn't prerender the body, is the only one immune: not because of a better defense, but because it emits less. It's the exact flip side of its missing DOMPurify seen in Chapter 8: there, "not defending" hurt; here, "not emitting" saves. The lesson closes the thread opened in Chapter 8: when the XSS defense lives in a single render, every other emitter has to re-sanitize on its own, and sooner or later one of them forgets. The right fix isn't to add `strip_tags` here and DOMPurify there, but **a single server-side sanitization function**, used by every PHP emitter of the content. The thread will close in Chapter 12 (the feed, which escapes) and Chapter 13 (the newsletter, which doesn't emit).

*
**

5. The SEO That Indexes Drafts

There's a second rule the prerender ought to share with the rest of the system, and that SR and DIS forget. In Chapter 9 the content lifecycle established that a piece of content is public only if `status = 'published'` **and** its publication date is in the past. SPW reuses this rule in the SEO queries and even in the sitemap. SR and DIS filter on the date alone:

```
// public/index.php:130-135 (SR) – the status filter is missing
$stmt = $pdo->prepare(
    "SELECT id, title, slug, summary, content, cover_image, published_at
    FROM news WHERE slug = ? AND published_at <= ? LIMIT 1"); // ← no "AND
    status = 'published'"
```

The consequence is a crack between two ideas of "public." A draft article with a past date is invisible in the API and in the public list, but it leaks into the meta tags, into the HTML served to the crawler, into the "latest news" block on the homepage and, in SR, even into the sitemap, which hands it to Google.

WARNING

Two ideas of “public” in the same site “Public for the user” and “public for the bot” ought to be the same thing, but when the visibility rule is written by hand in every query, it’s easy for one path to apply it and another not to. SPW keeps it aligned across three files (API, prerender, sitemap); SR loses it in the very file that feeds it to the search engines. Here too the root is the same as the XSS hole: a domain rule that doesn’t live in one single place ends up diverging among its consumers.

✱
✱✱

6. The SEO Cache That Outlives Its Reader

SR, unlike SPW, has an SEO cache: on every save it writes a `.cache/seo_news_<md5(slug)>.json` file, regenerates it in bulk with a dedicated script, and deletes it when you delete the content. All perfectly maintained, except for one detail: **no one reads it**.

A search across the entire codebase doesn’t turn up a single place that opens those files. The v3.0 engine queries the database directly. And the smoking gun is written in the code itself: the banner in `index.php` declares it has replaced “the v2 cache-file proxy,” and a regeneration script contains a comment that gives away the vanished reader:

```
// public/api/rebuild_seo_cache.php:67-79 (SR)
$seoData = [
    'title' => $speaker['name'],
    'description' => $speaker['role'] . ' - ' . substr($speaker['bio'] ?? ''
    ↪ , 0, 150) . '...',
    'name' => $speaker['name'], // Injected for consistency with index.php r
    ↪ eader ← the reader no longer exists
];
file_put_contents($cacheDir . '/seo_speaker_' . md5($speaker['id']) . '.json'
    ↪ , json_encode($seoData));
```

Version 2 was a proxy that read those JSON files; v3.0 rewrote the engine with the direct query and removed the reader without shutting down the writers. Every save still pays the cost of writing a file no one will open. It’s the opposite of SPW, which never had an SEO cache, by choice.

TIP

The cache that outlives its reader It’s a specific way code rots: you rewrite the consumer of something and forget to shut down its producers. The cache stays, gets updated, invalidated, regenerated, and serves no purpose. It also stands as a correction to a point in Chapter 7, which cited the `rebuild_seo_cache` script as useful for migrations: today it regenerates a dead cache. The next section gives this cache a less innocent twist than that.

✱
✱✱

7. When the Entry Point Becomes a Target

There's a reason a cache that serves bots without touching the database isn't just an optimization. Runtime Radio learned it, and the lesson is important enough to have a home in this chapter: the attack vector is exactly the SEO entry point we've just described.

Between February 23 and 27, 2026, the site went through two overlapping crises. The first was the collapse of the SQLite database under a load that had been growing for months, resolved with an emergency migration to MySQL in under a day (Chapter 15 tells it). The second arrived while the infrastructure was still unstable: a wave of requests returning 503 and 500 errors, with near-vertical traffic spikes that organic growth couldn't explain.

The vector was elegant. Thousands of hostile bots **impersonated the social crawlers**, sending a Telegram or Facebook `User-Agent`. And every request with that UA hit `index.php`, which by design queried the database to extract the meta tags. The SEO engine, designed to make shared links look good, had become a lever: a near-zero-cost request for the attacker forced a query on the just-migrated, still-fragile database.

```
Hostile bot → User-Agent: TelegramBot → request to /news/a-slug  
→ index.php → DB query for the meta tags → response  
→ the bot discards it and repeats, a thousand times a second  
→ the DB gives out → 503/500
```

The response separated the bots' path from the users'. The HTML for social crawlers was **generated from precompiled static JSON files**, written to disk at the moment an article is published: the bot gets its page with the meta tags in a few milliseconds, read from the cache instead of from the database, which is therefore never queried. Only real users, with a browser that runs JavaScript, take the normal path. And the core service, audio streaming, stayed up behind a static maintenance page even while the rest was offline.

Those precompiled JSON files have a familiar name: `.cache/seo_*.json`. They're the same SEO cache from the previous section. It's hard not to read the two facts together: the cache was born as an anti-DDoS shield, a path for the bots that didn't touch the database; then the v3.0 rewrite of the engine returned to the direct query and removed the reader, leaving the cache written-but-never-read. The shield is still there, it's still polished on every save, but no one carries it anymore.

WARNING

Every public endpoint that queries the DB is a target; the User-Agent is not a gatekeeper Three things to take home. First: any unauthenticated endpoint that, to

respond, queries the database is a lever for a volumetric attack. If the response to a repetitive request can come from a static cache, that cache isn't just performance, it's a security layer. Second, already met in Chapter 10: social bots are recognized by their `User-Agent`, and the `User-Agent` can be spoofed. You can use it to *optimize* (serving a cache to recognized bots), never as an access barrier. Third, the most uncomfortable: a defense that works can be dismantled without meaning to, in a

rewrite that looks only at features. Runtime Radio’s SEO cache wasn’t “removed”: it was orphaned, and with it the protection it gave.

8. Sitemap, robots, and Structured Data

What remains is the surrounding SEO infrastructure, which is easy to dismiss as “already handled by the .htaccess.” In reality there’s a precise pattern: `sitemap.xml` and `robots.txt` aren’t physical files, but rewrites toward `sitemap.php` and `robots.php`, generated in real time.

```
# public/.htaccess (SPW) — no physical files: sitemap and robots are PHP
DirectoryIndex index.php index.html          # the SEO Engine before
↪ index.html
RewriteRule ^sitemap\.xml$ sitemap.php [L,NC]
RewriteRule ^robots\.txt$ robots.php [L,NC]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_URI} !^/api/ [NC]
RewriteRule ^(.*)$ /index.php [L,QSA]      # every virtual React U
↪ RL → SEO Engine
```

This way the sitemap is always fresh without prerendering, and its `baseUrl` is derived from the request host, so the same file works on production and staging. SR’s `robots.php` adds a small editorial personality: it blocks commercial SEO crawlers (Ahrefs, Semrush, DotBot) and imposes a `Crawl-delay`, so as not to waste bandwidth on tools that bring no readers.

On structured data, both flagships build the JSON-LD as a PHP array, choosing the type based on the page: `Article` or `NewsArticle` for articles, `CollectionPage` for listings, and on the homepage an `@graph` that describes the site and the author (or, for Runtime Radio, a `RadioStation`). DIS, faithful to its rung, doesn’t emit JSON-LD: it stops at the meta tags.

In Summary

Indexing a SPA without an SSR framework is possible, with a PHP that acts as a front controller, sniffs the bot, and serves it complete HTML. But Dynamic Rendering isn’t free: it duplicates the route map, and above all it reopens the XSS hole in the content, because it re-emits the `content` with a tool (`strip_tags`) weaker than the render’s `DOMPurify`. SPW does it well, SR copied its defects too and added some of its own (the indexed drafts, the orphaned cache), DIS sidesteps the problems by doing less. And the story of February’s DDoS closes the circle: the entry point that makes the site visible is the same one that, under strain, brings it down, and the cache that saved it is today a tool forgotten in a corner.

IMPORTANT
The Canon

- Index the SPA with Dynamic Rendering from a PHP entry point (UA-sniffing), not with a fragile SSG.
- The prerender re-emits the `content`: sanitize it with the **same** defense as the render, because `strip_tags` with an allowlist lets attributes through (`onerror, href="javascript:"`).
- Respect the `status` in the crawler branch too: don't index drafts.
- Dynamic `sitemap/robots`; the User-Agent is not a security gatekeeper.

*
**

Next Chapter: RSS Feed & Syndication. The feed is the safest emitter of the content, the one that closes the thread of the four emitters, and at the same time the place where a CORS proxy and a bit of security theater lurk.

CHAPTER 12: RSS Feed & Syndication

The RSS feed is the channel a site uses to deliver its content to people who don't come read it on the site: aggregators, news readers, podcast apps, Telegram bots. In the Model it's always the same thin-stack gesture: a PHP endpoint that, on every request, queries the database and prints the XML on the fly. No cron, no `.xml` file materialized on disk, no serialization library. A published article shows up in the feed on the first refresh.

But this chapter has a precise lens, and it's the closing of a thread. In Chapter 8 we opened the picture of the four `content` emitters: the same HTML, saved raw in the database, exits toward the world from four points, each with its own defense. In Chapter 11 we saw the prerender leave the hole open. The feed is the emitter that **closes** it: in all three sites, either it doesn't emit the `content` at all, or it escapes it in full. It's the safest point in the chain.

And precisely because the three sites converge on security, it's interesting how much they diverge on everything else. The geography runs from a single file to a trio of files with opposite roles, all the way to a feed that isn't even a news feed but a podcast one. And on GUID discipline you see a small story of regression: a good idea solved by one site and forgotten by the other two.

NOTE

Three common misconceptions about these feeds. Three points, often taken for granted, are actually wrong or misleading. The **podcast feed isn't SitoRuntime's but DISINTELLIGENZA's**: SR doesn't generate a podcast feed, it *consumes* external ones with a proxy (§4). The **empty catch** is sometimes taught as a virtuous “silent fallback”: it's an anti-pattern instead, because it serves a truncated feed with HTTP 200 (§7). And **feed.php is not an “alias”** of `rss.php`: it's DIS's podcast feed, an endpoint with an entirely different purpose (§3).

✱
✱ ✱

1. The Common Anatomy of a Feed

Beneath the differences, the three sites' feeds share the same skeleton. A single file, a `GET`, the XML printed live from the database. The example is SimonePizziWebSite's `rss.php`, the most straightforward of the three:

```
// public/api/rss.php (SPW) — the news feed, real-time from the DB
header('Content-Type: application/rss+xml; charset=utf-8');
$pdo = Database::connect();
date_default_timezone_set('Europe/Rome'); // timezone forced
↳ in the individual file
```

```
$ita_now_str = date('Y-m-d H:i:s');
define('RSS_FEED_LIMIT', 50);

echo '<?xml version="1.0" encoding="UTF-8" ?>' . "\n<rss version=\"2.0\">\n<
↳ channel>\n";
echo '  <title>' . htmlspecialchars($site_title) . "</title>\n";
echo '  <link>' . htmlspecialchars($base_url) . "</link>\n";
echo '  <language>it-IT</language>' . "\n";
```

Three precautions recur in all three sites. Every dynamic field that enters the XML passes through `htmlspecialchars`: title, link, image URL, none of them ends up raw in the feed. The Europe/Rome timezone is re-forced inside the file itself, without trusting the timezone set elsewhere, because every file that compares dates against the database has to defend itself against a server in the wrong timezone (the lesson of Chapter 9). And the publication date always comes out in RFC-822 format, the one RSS 2.0 demands, via the `DATE_RSS` constant: never ISO 8601, which many readers reject.

There’s also a small shared asymmetry. While `sitemap.xml` and `robots.txt` are served from clean URLs via a rewrite (we saw it in Chapter 11), the feed is always reached at the raw URL of the PHP file: `/api/rss.php`, `/api/feed_news_rss.php`, `/api/feed.php`. None of the three sites gives it a clean `/feed.xml`. SEO care on the other assets, none here.

**

2. The Feed Closes the Thread of the Four Emitters

Let’s pick the table back up from Chapter 8. The same `content`, saved raw, exits from four emitters, and the only defense that really counts is the one each puts up on its own. The render uses `DOMPurify` (except DIS). The prerender uses `strip_tags` with an allowlist, which lets attributes through: it’s the live flaw of Chapter 11. The feed, the third emitter, closes it.

#	Emitter	What it emits of the content	Defense	Outcome
1	React render (Ch. 8)	full content	DOMPurify (SPW, SR) / none (DIS)	real choke point; DIS exposed
2	SEO prerender (Ch. 11)	full content	<code>strip_tags</code> allowlist (tags only)	attribute hole (SPW, SR)
3	RSS feed (this chapter)	nothing (SPW, DIS) / escaped preview (SR)	<code>htmlspecialchars</code> / <code>strip_tags+htmlspecialchars</code>	safe
4	Newsletter (Ch. 13)	(the next chapter closes it)	—	—

The feed is safe, but by two different roads. SPW is safe **by subtraction**: its `rss.php` loads the `content` column in the query but never prints it; the item’s `<description>` uses only the excerpt, escaped.

```
// public/api/rss.php (SPW) – content SELECTed but NEVER emitted; descriptio
↳ n = escaped excerpt only
```

```
echo '    <description>' . htmlspecialchars($article['excerpt']) . "</descri
↳ ption>\n"; // ← excerpt, not content
// (the content column is in the SELECT but there isn't a single echo of $ar
↳ ticle['content'])
```

SR, instead, **breaks the subtraction**: when the summary is missing, it falls back to the first 500 characters of the `content`. But it neutralizes them with a `strip_tags` without an allowlist, which removes *all* tags, followed by `htmlspecialchars`:

```
// public/api/feed_news_rss.php (SR) - emits a preview of the content, but f
↳ ully escaped
$descriptionText = $article['summary'] ?: $article['content_preview'] . '...
↳ '; // uses content if summary is missing
$description = htmlspecialchars(strip_tags($descriptionText));
↳ // strip_tags WITHOUT allowlist + escape
```

The difference from Chapter 11's prerender is all here: there, `strip_tags` had an allowlist and left the tags (and their dangerous attributes); here it has no allowlist, so no tag survives, and that bit of text also gets turned into entities. DIS, for its part, doesn't emit a news feed: its feed is a podcast one, and the `news content` is touched by no one.

IMPORTANT

Two ways to make a feed safe: don't emit, or escape The feed proves, in the positive, the thesis that runs through Chapters 8, 11, and 13: when the XSS defense lives in a single render, every other emitter has to fend for itself, and the feed manages it. SPW doesn't emit the dangerous field (safe by subtraction: robust, but it loses information, no full article in the feed). SR emits it but escapes it in full (safe by escaping: more informative, but more fragile, because it would only take adding an allowlist back to `strip_tags` to reopen the hole, as happened in the prerender). Same outcomes, opposite philosophies. The conclusion is still the one from the thread: sanitization should live once, server-side, not be reinvented by every emitter. The picture closes completely in Chapter 13, where no one emits the `content`.

*
**

3. The Geography: One File, a Trio, a Podcast Feed

On form, the three sites couldn't be more different.

SPW is a single file, `rss.php`: a news feed, minimal and rigorous. SR is a trio, and here lies its complexity: `feed_news_rss.php` generates the site's news feed, `rss.php` (same name as SPW's, opposite role) is a proxy that *consumes* external feeds, and `feed_config.php` is a dispenser that serves the admin the feed address. DIS is a case of its own: no news feed, only `feed.php`, a **podcast** feed in RSS 2.0 with the iTunes namespace, subscribable from an app like Apple Podcasts.

NOTE

Producing a podcast feed, or consuming one: not the same thing It's an easy point to confuse. The site that *generates* a podcast feed is DIS, with `feed.php`: it reads the `podcasts` table and emits the episodes with an `audio <enclosure>` and the `<itunes:*`

tags. SR does the opposite: it generates no podcast feed, but with `rss.php` it *downloads* other people’s podcast feeds (Spreaker, its own AzuraCast) to show them on the site. Producing and consuming a feed are two mirror operations, and it’s precisely the contrast between these two sites that makes it plain. We see it in the next section.

*
**

4. The Inbound CORS Proxy: Consuming Other People’s Feeds

A browser can’t read a feed hosted on another domain if that domain doesn’t send CORS headers, and external podcast feeds usually don’t. SR solves this with a server-side proxy: `rss.php` downloads the other feed and returns it same-origin. The interesting part is the defense, because a proxy that downloads whatever URL you hand it is an *open proxy*, a door flung wide to SSRF attacks.

```
// public/api/rss.php (SR) - inbound proxy: allowlist + https-only + stale f
↳ allback
$allowedHosts = ['www.spreaker.com', 'spreaker.com', 'player.runtimeradio.co
↳ m'];
if ($scheme !== 'https' || !in_array($host, $allowedHosts, true)) {
    http_response_code(400);
    echo json_encode(['success' => false, 'error' => 'Feed URL not allowed'])
↳ ); exit; // no open proxy
}
// ... disk cache rss_<md5>.xml, TTL 30 min ...
if ($xml === false || $statusCode !== 200) {
    if (file_exists($cacheFile)) { header('X-Cache: STALE'); readfile($cache
↳ File); exit; } // a stale cache beats nothing
}
```

Two sound choices: an allowlist of hosts (the proxy downloads only from those domains) and the HTTPS requirement close the SSRF door; a disk cache with a “stale” fallback means that, if the upstream is down, the last good copy is served instead of failing. There is, though, a less sound detail on the client side: if the site’s proxy doesn’t respond, the frontend code falls back to public third-party proxies (CodeTabs, AllOrigins), which don’t know the allowlist. Resilience is bought with an undeclared dependency on third parties.

WARNING

A proxy that downloads URLs is an SSRF target Any endpoint that, on request, downloads a URL supplied from outside has to be treated as a potential lever to reach internal resources (cloud metadata, services on `localhost`, private IPs). The two defenses in `rss.php` are the correct minimum: an explicit allowlist of trusted hosts and the rejection of anything that isn’t HTTPS. The rule is the same as Chapter 10’s on the IP: don’t trust a value that arrives from the client, in this case the URL to download. And watch the fallbacks: the client’s fall-through to public proxies sidesteps the allowlist and is worth knowing about.

*
**

5. feed_config.php: The Lock on the Door Next to It

In SR's trio there's a third file that's worth a box of its own, because it's a textbook case of apparent security. `feed_config.php` is protected by `isAdmin()` and its comment promises a lot: "Returns the private RSS feed URL only to authenticated admins. The token is never exposed in the public JavaScript bundle."

```
// public/api/feed_config.php (SR) - gates the URL, but the endpoint is publ
↳ ic
require_once 'auth_utils.php';
if (!isAdmin()) { http_response_code(403); echo json_encode(['success'=>false]);
↳ e, 'error'=>'Forbidden']; exit; }
$origin = (/* https */ . '://' . $_SERVER['HTTP_HOST']);
echo json_encode(['success' => true, 'feed_url' => $origin . '/api/feed_news
↳ _rss.php']);
```

Two things don't add up. There is no token: the returned URL is bare. And above all the feed that URL reaches, `feed_news_rss.php`, is **completely public**: it doesn't include `auth_utils.php`, doesn't call `isAdmin()`, anyone can read it without logging in. The `isAdmin()` gate protects the act of *discovering* an address that's public and guessable anyway.

WARNING

The lock on the door next to it Putting a gate on the dispenser of a URL doesn't make the endpoint that URL reaches private. It's security through obscurity, and the obscurity here isn't even there, because the file name is predictable. Almost certainly it's the fossil of an intention never realized: an authenticated news feed with a token for the Telegram bot (consistent with the `TELEGRAM_BOT_TOKEN` that lives in the secrets, Chapter 10), designed and never built. The lesson: if an endpoint has to be private, the authentication goes on *that endpoint*, not on the slip of paper that carries its address.

**

6. The GUID That Republishes

Every `<item>` in a feed has a `<guid>`, the identifier consumers use to recognize whether a piece of content is new or already seen. Getting it wrong has a concrete, annoying consequence: if an article's GUID changes, aggregators and bots treat it as a new article and republish it. On a Telegram integration, that means spamming every subscriber all over again.

SPW solved the problem cleanly: the GUID is a URN built on the database ID, decoupled from the URL.

```
// public/api/rss.php (SPW) - stable GUID, independent of the URL
$stable_guid = 'urn:simonepizzi:article:' . (int)$article['id'];
echo '    <guid isPermaLink="false">' . $stable_guid . "</guid>\n";
```

This way a change of slug or category doesn't touch the article's identity: to consumers it stays the same content. It's the same concern that, on the URL side, justifies 301 redirects: keeping a page's identity stable even when its address changes.

The other two sites lost this idea. SR uses the permalink as the GUID, with `isPermaLink="true"`: at the first slug change, the article republishes (and SR doesn't even have 301s to act as a safety net). DIS goes lower still and uses the audio file's URL: all it takes is for `migrate_media` to move the audio and every episode is "new" again in the podcast apps.

TIP

An article's stable identity: a URN, not the permalink The URN GUID is the correct recommendation. What needs adding is that it's a best practice two sites out of three **don't** follow: SPW applies it, SR regressed to the permalink, DIS to the audio URL (the least stable of all). The rule, in one line: a feed item's identity has to depend on something that never changes (the database ID), never on the address, which changes plenty.

*
**

7. The Empty Catch That Hides a Database Down

There's a point often mistaken for a virtue, and wrongly so. Both SPW and DIS wrap the query in a `try/catch` with an **empty** catch:

```
// public/api/rss.php (SPW) - the empty catch: the header and <channel> have
↳ already gone out with HTTP 200
} catch (Exception $e) {
    // silent fallback: no log, no 5xx
}
echo "</channel></rss>";
```

The problem is the sequence. By the time the exception fires, the `Content-Type` header and the opening of the `<channel>` have already been printed, with response code 200. The empty catch swallows the error, and the closing `</channel></rss>` comes out anyway: the client receives a perfectly well-formed and **empty** feed, indistinguishable from "no news." No log, no 500, no signal. A database down becomes invisible.

SR does better, though not perfectly: it intercepts the exception and responds with a real HTTP error.

```
// public/api/feed_news_rss.php (SR) - at least it signals the failure
} catch (PDOException $e) {
    http_response_code(500);
    echo "<error>Database Error</error>"; // (covers only PDOException: a
↳ non-PDO error would slip through)
}
```

WARNING

The silent fallback that hides a failure An error disguised as a valid response is worse than a visible error: no one notices it until a user complains that the feed "hasn't updated in a week." The rule is not to open the response (header and first output) before running the part that can fail, or to handle the failure with an honest HTTP code, as SR tries to do. The empty catch isn't a fallback: it's a silenced failure.

*
**

8. Minor Scars

Some smaller details, but real ones. The visibility filter from Chapter 9 (`status = 'published'`) is forgotten here too: SR's news feed filters on the date alone, so a draft with a past date leaks into the feed, the third file after `index.php` and the sitemap to forget the same rule. The image's `<enclosure>` declares `type="image/jpeg"` hardcoded, but the upload converts covers to WebP (Chapter 7): the pickier readers discard a thumbnail whose MIME doesn't match. And in both flagships the channel configuration (title, description) is hardcoded, with a comment that promises to read it "from the settings": an announced configurability never wired up, of which DIS is the extreme version (it reads `podcast_*` keys that no one populates, so it always runs on the defaults).

On DIS there's also a detail that's almost an artifact. The `feed.php` contains comments where the code reasons out loud with itself:

```
// public/api/feed.php (DIS) — the code debating its own doubts, in producti
↪ on
// During init_db.php step I created a 'podcasts' table? Let's check init_db
↪ .php if I can.
// Fallback: ... create it via SQL here? No, bad practice on GET.
// I'll assume it exists or use NEWS with a category.
```

NOTE

When the code narrates its doubts It isn't a bug, but it's a process scar: the portrait of a codebase generated in conversation with an assistant, and never reread before deploy. The same tone shows up in other DIS files (the `init_db.php`, the `api.ts`). In production the code should assert, not question itself: these monologues should be removed in the rewrite, because they reveal, to anyone who opens the source, exactly what the author hadn't verified.

✱
✱✱

9. Announcing and Delivering the Feed

A feed is of little use if no one finds it. It has to be announced in the HTML's `<head>`, so browsers and RSS readers discover it on their own:

```
<link rel="alternate" type="application/rss+xml" title="Feed Notizie" href="
↪ /api/rss.php" />
```

And it has to be delivered to the distributors. SPW has a "Copy RSS" button in the admin area, which puts the address on the clipboard with a click; SR shows the URL in the dashboard (that's what `feed_config.php` is really for, once the pretense of privacy is stripped away). A small editorial UX touch: giving whoever publishes a way to hand the feed to an aggregator or a bot without transcribing it by hand.

✱
✱✱

In Summary

The feed is the quietest part of the content's journey: the emitter that closes the XSS hole, by subtraction or by escaping. But around this quiet the three sites tell three

different stories: SPW is minimal and disciplined (stable GUID, visibility rule respected, errors aside); SR is richer and more fragmented, with a well-defended inbound proxy, a dispenser that feigns a privacy that isn't there, and a few regressions (the drafts in the feed, the GUID that republishes); DIS does only podcasts, on the defaults, with the author's doubts still written in the code. The thread of the four emitters, meanwhile, has one last box to fill: the newsletter.

IMPORTANT **The Canon**

- A valid RSS 2.0 feed: correct header, RFC-822 dates, absolute URLs, `content` escaped (or not emitted at all).
- The feed is a `content` emitter: either you escape it fully or you don't put it in.
- A stable GUID with `isPermalink="false"` (a URN), not the mutable permalink.
- No empty `catch` that masks a failure: better an explicit 5xx. An inbound proxy has to be protected with a host allowlist + https-only (anti-SSRF).

✧
✧ ✧

Next Chapter: Newsletter & Email System. The last emitter of the content, the one that closes the thread completely, and a scale of how far you can simplify a mail system before it becomes dangerous.

CHAPTER 13: Newsletter & Email System

An email list is one of the few assets a site truly owns: it doesn't pass through an algorithm, it doesn't risk a shadowban, it doesn't vanish if a platform shuts down. The three sites in the Model all have a homegrown newsletter, with no external services, and they all build it on the same thin-stack skeleton: a PHP endpoint that handles subscription and sending, and the email composed as an HTML string on the fly.

This chapter closes a thread that began five chapters ago. The newsletter is the **fourth and last emitter** of the `content`: after the render (Chapter 8), the prerender (Chapter 11), and the feed (Chapter 12), it's the last place from which the same raw-saved HTML could exit toward a browser, the mail client's. The good news, which we'll see at the end, is that none of the three sites emits the `content` in the email: the thread closes without reopening the XSS hole.

And precisely because the three converge on the XSS risk, the chapter's real lens becomes another one: **how far you can simplify a mail system** before the simplification becomes dangerous. It runs from a complete system, with double opt-in and anti-abuse protection, all the way to a bare `mail()` that anyone can use to subscribe or unsubscribe anyone. And, as in Chapter 10, the site with the richest backend isn't the most secure.

NOTE

Three points it's easy to get wrong. Three aspects of this system are often misunderstood. The first is **double opt-in**, the cornerstone feature of two sites out of three: omitting it, and showing in its place a subscription that's active immediately with unsubscribe-by-email-only, means adopting DIS's model, the weakest. That unsubscribe-by-email-only **isn't "GDPR-compliant"**: it's the insecure version (anyone unsubscribes anyone). And a `usleep` **isn't "rate limiting"**: it's a different thing (§4).

✱
✱ ✱

1. The Life of a Subscriber

Beneath the differences, the anatomy is shared. The endpoint dispatches by `?action=`, with the public actions served before a central gate and the admin ones after. The email is always validated server-side, without trusting the form, and re-subscription is handled reactively: you attempt the `INSERT` and catch the `UNIQUE` constraint violation, returning a neutral success that doesn't reveal who's already on the list.

```
// public/api/newsletter.php (DIS) - server-side validation + idempotence by
↳ catching the duplicate
$email = filter_var($input['email'] ?? '', FILTER_VALIDATE_EMAIL);
if (!$email) { echo json_encode(['status'=>'error','message'=>'Email non val
↳ ida']); exit; }
try {
    getDB()->prepare("INSERT INTO subscribers (email) VALUES (?)")->execute(
↳ [$email]);
    echo json_encode(['status'=>'success','message'=>'Iscrizione completata'
↳ ]);
} catch (PDOException $e) {
    if ($e->getCode() == 23000) { // UNIQUE violation → already subscribed
↳ , but we don't reveal it as an error
        echo json_encode(['status'=>'success','message'=>'Sei già iscritto!'
↳ ]);
    } else { echo json_encode(['status'=>'error','message'=>'Errore database
↳ ']); }
}
```

The unsubscribe is “soft”: no one deletes the record, the subscriber is flagged `is_active = 0`, which preserves the history and avoids accidental re-subscriptions. And the final email, whatever the site, is built as an HTML string with a table-based layout and inline CSS (mail clients don’t read external stylesheets), with the images made absolute and a placeholder in the unsubscribe link, substituted for each recipient.

NOTE

The minimal schema isn’t the real one. The four-column `subscribers` table (`id`, `email`, `is_active`, `created_at`) is DIS’s model. The real schema of SPW and SR instead has the double opt-in fields: the status (`pending/confirmed/unsubscribed`), one or more tokens, the confirmation date. In SR that extended schema coexists with two older versions created by other scripts, one of which is an SQLite fossil that would break on MySQL: the runtime assumes the extended schema and one query fails until the right migration has been run. It’s the same “one table, several truths” story from Chapter 15.

✱
✱✱

2. Double Opt-In and the Secret of the Unsubscribe Link

Double opt-in is the guarantee that whoever subscribes an email **owns** that mailbox: instead of activating the address right away, you create a pending record and send an email with a confirmation link; only after the click does the subscription become active. It’s the most important feature of the whole system, and the three sites implement it on three rungs.

SPW does it the textbook way, with **two distinct tokens**: a confirmation one, single-use, cleared after use; an unsubscribe one, random and stable, separate from the first.

```
// public/api/subscribers.php (SPW) - two tokens with different purposes
$confirmToken    = $forceConfirm ? null : bin2hex(random_bytes(32)); // s
↳ single-use
$unsubscribeToken = bin2hex(random_bytes(32)); // s
↳ table, separate
$status          = $forceConfirm ? 'confirmed' : 'pending';
```

```
if (!$forceConfirm) { sendConfirmEmail($email, $name ?: 'Amico', $confirmTok
↵ en); }
```

SR does it with a **single token** that serves both purposes, confirmation and unsubscribe, and that's never cleared or expired. Two small consequences follow. The confirmation email promises that “the link expires after first use,” but that's false: the token survives use. And since the same token ends up in the unsubscribe URL of *every* newsletter, anyone who forwards a message hands someone else the power to unsubscribe that user.

```
// public/api/newsletter.php (SR) – one token, two purposes
// confirm: activates the subscription BUT doesn't clear the token
"UPDATE subscribers SET is_active = 1, confirmed_at = NOW() WHERE confirmati
↵ on_token = ?"
// send: the unsubscribe URL exposes the same confirmation_token
$unsubscribeUrl = 'https://runtimeradio.com/unsubscribe?token=' . urlencode($sub['
↵ confirmation_token']);
```

DIS has neither. The subscription is active immediately, and unsubscribe happens by email alone, with no token:

```
// public/api/newsletter.php (DIS) – unsubscribe by email alone, via GET, wi
↵ th no token
if ($action === 'unsubscribe') {
    $email = filter_var($_GET['email'] ?? '', FILTER_VALIDATE_EMAIL);
    getDB()->prepare("UPDATE subscribers SET is_active = 0 WHERE email = ?")
↵ ->execute([$email]);
    echo "<h1>Disiscrizione completata</h1>"; // anyone who knows the email
↵ can unsubscribe it
}
```

WARNING

The unsubscribe link needs a secret This unsubscribe-by-email-alone is often passed off as “GDPR-compliant.” It's the opposite: without a secret token, anyone who knows or guesses a subscriber's address can unsubscribe them. And since it's a GET, it's also *prefetchable*: a mail client that preloads links can unsubscribe the user just by hovering over it. The correct version is SPW's, with a random, stable unsubscribe_token (and, ideally, a POST confirmation from the landing page). Double opt-in protects the entrance; an unsubscribe token protects the exit. You need both.

✱
✱✱

3. Sending: Native mail() or Authenticated SMTP

Transport is the point where SR breaks away from the other two. SPW and DIS use PHP's native mail(), which leans on the system sendmail: zero configuration, but fragile deliverability, because without SPF and DKIM the emails easily land in spam (DIS even has a “Fake domain?” comment next to the sender). SR, instead, uses PHPMailer with authenticated SMTP and STARTTLS, reading the credentials from the environment secrets.

```
// public/api/newsletter.php (SR) – authenticated SMTP via PHPMailer
$mail = new \PHPMailer\PHPMailer\PHPMailer(true);
```



```
$mail->isSMTP();
$mail->Host = $cfg['SMTP_HOST']; $mail->SMTPAuth = true;
$mail->Username = $cfg['SMTP_USER']; $mail->Password = $cfg['SMTP_PASS'];
$mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS; $mail->Port = $cfg['SMTP
↳ _PORT'];
$mail->setFrom($cfg['SMTP_USER'], $cfg['SMTP_FROM_NAME']);
```

There’s an internal contradiction, though: SR uses authenticated SMTP only for the newsletter. The contact form, `contact.php`, stayed on native `mail()`. The same site thus has two different mail mechanics: it’s easy to think of SMTP as a choice “for the future, at higher volumes,” when in reality in SR it’s already in production, alongside the `mail()` it never retired.

*
**

4. The Form That Fires Emails in Your Name

Here comes the chapter’s most instructive flaw, and it’s born of a common confusion between two defenses that look similar and aren’t.

A **throttle** slows the outgoing send, so as not to overload the mail server and get greylisted: it’s a pause every so many emails. A **rate limit** caps incoming requests, to keep a stranger from abusing an endpoint: it’s a ceiling of attempts per IP. They’re orthogonal defenses, on opposite sides of the system. What often gets called “rate limiting” is really a throttle:

```
// public/api/newsletter.php (SR) – this is an OUTBOUND THROTTLE, not an inb
↳ ound rate limit
if ($count % 10 === 0) { usleep(500000); } // half a second every 10 email
↳ s: protects the mail server
```

The throttle protects *your* mail server. It doesn’t protect against someone hammering the subscription form. And here’s SR’s hole: its `subscribe` has no rate limit at all. The requester’s IP is even recorded, but only stored, never used as a limit, and on top of that read from raw `X-Forwarded-For` (forgeable, as in Chapter 10). Anyone can therefore send subscription requests with arbitrary emails, and each one fires a real confirmation email via SMTP toward a third party who asked for nothing. It’s a mail-bombing vector that burns the domain’s reputation and consumes the SMTP quota.

SPW, instead, closes the vector, reusing for the newsletter the same `login_attempts` table as the login (with a different key prefix, so as not to mix the counters):

```
// public/api/subscribers.php (SPW) – anti-mail-bombing rate limit that recy
↳ cles login_attempts
$rl_key = 'sub:' . substr(hash('sha256', $_SERVER['REMOTE_ADDR']) ?? 'unknown
↳ ', 0, 40);
$stmtRl = $pdo->prepare("SELECT COUNT(*) FROM login_attempts WHERE ip_addres
↳ s = ?");
$stmtRl->execute([$rl_key]);
if ((int)$stmtRl->fetchColumn() >= 3) { http_response_code(429); exit; } /
↳ / max 3 subscriptions / 15 min
```

WARNING

Inbound rate limit ≠ outbound throttle SR has the throttle but not the rate limit; SPW has the rate limit but not the throttle. They have opposite defenses, and the one SR lacks is the one that matters most for security: without an inbound ceiling, the subscription form becomes a weapon that fires emails in your name toward anyone. It's the reversal already seen in Chapter 10: SR, the most engineered site, leaves open exactly the hole that SPW, simpler, had closed. And the irony is that the infrastructure to limit (the same `.cache/ratelimit/` as the login) already exists in SR: it simply wasn't reused here.

There's then a problem common to all three, on a descending scale of crudeness: sending is a blocking `foreach` inside the HTTP request. On a large list, the request hits `max_execution_time` and delivery cuts off halfway, without anyone knowing. SR is the least bad (it has the throttle and a per-recipient `try/catch` that counts errors); SPW counts only `mail()`'s return value; DIS ignores it entirely, and the campaign counter records the *attempts*, not the successes. None of the three has a queue or a cron: it's the same "heavy work inside the request" anti-pattern seen with image conversion in Chapter 7.

**

5. Header Injection from the Name Field

One last trap, and it's in DIS, in the contact form. The name entered by the user is cleaned with `strip_tags` before landing in the database, and that's fine for the database. But that same name is also placed in the subject of the notification email to the admin, and there `strip_tags` isn't enough:

```
// public/api/contact.php (DIS) - strip_tags removes tags, NOT line breaks
$name = strip_tags($input['name'] ?? ''); // sanitization for the DB
$subject = "Nuovo Messaggio da $name - Disintelligenza"; // ...but
// $name ends up in the Subject header
$headers = "From: no-reply@...\r\nReply-To: $email\r\n";
mail('runtime@radio@gmail.com', $subject, $body, $headers);
```

`strip_tags` removes HTML tags, but not the line-break characters `\r\n`. A name that contains them can inject additional headers into the email, for example a `Cc` or a `Bcc` toward addresses chosen by the attacker. The sender's email, which ends up in the `Reply-To`, is instead safe because it's passed through `FILTER_VALIDATE_EMAIL`: the vector is the name, the only free text field that enters a header.

WARNING

Sanitizing for the database isn't sanitizing for email headers Sanitization always has a context. `strip_tags` neutralizes HTML, which is the danger when the data will be shown on a page; but when the same data enters a mail header, the danger changes shape, and it's the `\r\n` that count. The rule: for an email header, remove or reject control characters, and never put user input in the headers if you can avoid it. SR, not by chance, builds the `From` from a fixed value, not from input.

**

6. The Thread of the Four Emitters Closes

Let’s go back one last time to the table from Chapter 8. The newsletter is the fourth box, and in all three sites the send query selects title, summary, image, and link, **never the content**. The email links back to the article with a “Read more,” and the raw HTML field defended only at render-time is never touched.

#	Emitter	What it emits of the content	Defense	Outcome
1	React render (Ch. 8)	full content	DOMPurify (SPW, SR) / none (DIS)	real choke point; DIS exposed
2	SEO prerender (Ch. 11)	full content	strip_tags allowlist (tags only)	attribute hole (SPW, SR)
3	RSS feed (Ch. 12)	nothing / escaped preview	htmlspecialchars / strip_tags+escape	safe
4	Newsletter (this chapter)	nothing (title, summary, intro only)	htmlspecialchars (SR, DIS) / raw behind Auth (SPW)	safe

You could read the query-without-content as a simple “payload optimization,” to keep the emails light. That’s true, but above all it’s the closing of the thread: not emitting the raw field is what keeps the email from becoming a fifth XSS vector.

There’s a curious symmetry between the two flagships. In SR the newsletter is the *most* secure of the four emitters: it doesn’t touch the content and on top of that escapes every other field. In SPW it’s instead the *least* sanitized, because the introductory text written by the admin is emitted raw, with no htmlspecialchars: it’s safe only because whoever writes it is authenticated, not because there’s a defense. Two opposite positions on the same scale.

IMPORTANT

The full picture: one sanitization, four render paths With the newsletter, the thread of the four emitters closes. Feed and newsletter don’t reopen the hole, because they either don’t emit the content or they escape it. The only flaw still live in the whole picture is the prerender of Chapter 11, with its allowlist strip_tags that lets attributes through. The conclusion, repeated for five chapters, is always the same: content sanitization should live once, server-side, shared by every emitter, instead of being reinvented (or forgotten) by each one. That the hole stayed open in only one point out of four isn’t to the architecture’s credit: it’s luck, plus the discipline of whoever wrote the other three.

✱
✱ ✱

7. Consent, and Where It Disappears

What remains is the GDPR side of subscription, which isn’t only a matter of tokens but of consent. On the form, SPW asks for double explicit agreement (data processing and a

declaration of legal age); SR asks for only one, and has a “minimal” variant of the form, meant for the footer, that has no checkbox at all. DIS, on the form, just validates and that’s it.

But the most interesting case is in DIS, and it doesn’t even go through the form. An email can enter the list from a second door too: when a festival participant is approved, their address is added to the subscribers with an `INSERT OR IGNORE`, **with no explicit consent to the newsletter**. The comments in the code show the same developer wondering whether it’s correct.

WARNING

Consent as a side effect Subscribing someone to a list because they did *something else* (applying to a festival) is a collection of consent that the GDPR doesn’t consider valid: consent has to be specific to that purpose. It’s an easy trap, because the code that does it looks harmless: a line of `INSERT` tacked onto the approval. The rule: every entry door to an email list must have its own consent, explicit and separate. The full treatment of the festival workflow is in the chapters that cover it; here the lesson is enough.

*
**

In Summary

The newsletter shows a scale of simplification that’s also a scale of risk. SPW is the complete rung: double opt-in with two distinct tokens, a rate limit against mail-bombing, an unsubscribe link with a secret, double consent. SR adds the more serious transport, authenticated SMTP, but removes the rate limit (and opens the mail-bombing vector) and merges the two tokens into one. DIS removes the double opt-in and every token too, and leaves a header injection in the contact form’s name field, while keeping two good hygiene habits (email validation everywhere and cleanup on write). On the `content`, though, all three do the right thing: they don’t emit it, and the thread of the four emitters closes. The defense that’s missing most often isn’t the one against XSS, which here is solved by discipline: it’s the one against abuse of your own form.

IMPORTANT The Canon

- Double opt-in with two distinct tokens (single-use confirmation + stable unsubscribe): the unsubscribe link needs a secret, the cleartext email isn’t enough.
- A rate limit (per-IP ceiling) on subscription against mail-bombing; don’t confuse it with the throttle, which only regulates send cadence.
- Sanitize for the email headers (the `\r\n`), not only for the DB: the name is a header-injection vector.
- Explicit GDPR consent; mass sending, better if asynchronous or queued.

*
**

Next Chapter: Admin Dashboard & Panels. The control panel that ties together the systems seen so far, and the three very different ways the sites decide what an administrator can see and do.

CHAPTER 14: Admin Dashboard & Panels

Every system seen so far has a back office. The content of Chapter 9, the media of Chapter 7, the newsletter of Chapter 13, the reactions of Chapter 20, the festival of the chapters that follow: all of them open, sooner or later, onto a panel from which an administrator watches and governs them. The admin area is the fabric that ties the other clusters together, the point where they become numbers to read and actions to take. It's a surface the festival makes easy to underestimate: the only "dashboard" people usually think of is the contest's, which is instead a special case (we meet it again in Chapter 19). Here we deal with the general administrative area, of which the festival's is only a specialization.

The three sites build their console on two independent questions. The first is *how it's built*: a route structure with a declarative guard, a single mega-component, or something in between. The second is *how much it measures*: an analytical dashboard with charts, a menu that counts nothing, or real numbers written in text. The two questions don't go hand in hand, and their combination yields three very different admins.

There's a reversal worth flagging up front, because it's the through-line. SitoRuntime is the site of the scalability scars, the one that lived through the nighttime crash and the emergency migration of Chapter 15. And yet it's the one with the least-equipped admin: it measures nothing and has no backup. That is, it has neither the eye to notice a problem nor the safety net to survive it. It's confirmation, once again, that more engineered doesn't mean more protected.

*
**

1. The Admin Is an Aggregator, Not an Application

Before the differences, four traits the three sites share.

The restricted area isn't a separate application: it's a frame that mounts the other clusters' panels inside itself. The editor of Chapter 8, the media library of Chapter 7, the newsletter composer of Chapter 13, the festival management live as pages of a shared console. What's the admin's own are the frame, the guard, and a few surfaces of its own: the dashboard, the settings, user management.

The guard, precisely, covers the whole area in a single place, not page by page. It's the reverse of the backend, where every endpoint calls its own `Auth::check()` (Chapter 10):

here the frontend protects the entire restricted subtree with a single barrier, and all the child pages inherit its protection. It’s the “one guard, N pages” principle.

Each panel degrades gracefully on its own: if a data source goes down (the metrics absent, a table not yet migrated), that panel hides itself or shows zero, but the rest of the console stays usable. The dashboard is never a blank page. And in all three the admin password change lives inside the settings, often the only truly “configuration” item present.

*
**

2. How It’s Built: Three Guard Architectures

On this axis, the scale runs from declarative to imperative.

SPW uses a declarative *route guard*: the route that mounts the admin layout has a loader, and all the children inherit its redirect. The session is verified before the page mounts; if it’s missing, the user doesn’t even see a flash of restricted content.

```
// SPW App.tsx:239-258 + loaders.ts:10-20 – one guard, N pages
{ element: <AdminLayout />, loader: adminAuthLoader, children: [ /* dashboar
  ↪ d, settings, ... */ ] }

export const adminAuthLoader = async () => {
  const session = await api.checkSession();
  if (!session || !session.user) return redirect('/admin/login'); // nev
  ↪ er an admin datum served without a session
  return session;
};
```

SR sits at the opposite extreme: a single component, `Admin.tsx`, nearly six hundred lines, that switches “section” in memory instead of navigating between routes. The guard is a check inside the component, run on mount. DIS takes a middle road that’s almost a curious hybrid: it has SPW’s structure (an `AdminLayout` with a sidebar and child routes via `Outlet`), it even runs on the data-router infrastructure that would allow loaders, but then it protects the area with a check inside the component, like SR.

```
// DIS AdminLayout.tsx:13-25 – structure like SPW, guard like SR
useEffect(() => {
  api.checkAuth().then(u => {
    if (!u) navigate('/admin/login'); // client-side protection,
  ↪ no loader
    else setUser(u);
  });
}, [navigate]);
if (!user) return null; // NB: no role === 'admin'
↪ check (see below)
```

The full comparison between loader guard and component guard, with its trade-offs, is in Chapter 6. Here what counts is a security consequence that shows up precisely in DIS.

WARNING

Protecting the area isn’t enough: you need the role DIS’s guard verifies that there’s a logged-in user, not that they’re an administrator. An *editor*, therefore, sees the entire console: subscriptions, voting, reset, user management. It’s the backend

that has to reject the actions the editor can't perform, but it does so inconsistently (Chapter 10): some endpoints are reserved for admins, others accept any logged-in user. The result is that an editor can really approve participants and change the rounds from the UI. It's the whole-area version of the "the gate hides the content but not the button" problem that also shows up in SR. The rule: hiding a page is user experience; preventing an action is security, and it has to be done on the role, both on the client and (above all) on the server.

**

3. How Much It Measures: Analytical, Console, Textual

The other axis is orthogonal to the first, and it separates the three sites cleanly.

SPW has a real analytical dashboard: seven total cards, a dozen mini-stats with a percentage trend, six charts (with a period selector at 7, 30, or 90 days). SR measures nothing: its dashboard is made of cards that are navigation buttons, not counters. It's a menu disguised as a dashboard. DIS is in the middle, and it's the most useful lesson of the three: it really measures, but in text. Counters of subscribers and votes, disk space broken down by folder, the provisional ranking, the latest arrivals. No charts, no tracking engine, just the right COUNTS.

TIP

How much dashboard you really need DIS's dashboard proves you can give informational value to an administrator without Chart.js and without an analytics engine: the right queries and a page that writes them out plainly are enough. SPW's analytical dashboard is richer, but it costs a whole tracking system to maintain; SR's "menu" is the rung below the minimum, because it doesn't even answer the simplest question ("how many subscribers do I have today?"). Between too much and nothing, the text-based dashboard is often the right point: it measures what you need to decide, and nothing else.

**

4. Measuring Without Third Parties

When a site wants to know how many visits it gets without handing its readers' data to Google Analytics, it has to build its own in-house analytics. SPW does it with `analytics.php`, a dual-personality file: the branch that *records* an event is public (the live site calls it on every visit), the branch that *reads* the reports is reserved for the admin.

```
// SPW analytics.php:16-77 – one file, two audiences
if ($method === 'POST') {
    // 'view'/'click' tracking from the live site, anonymous: no Auth
    // 'view' deduplicated by IP-hash + article + day; 'click' rate-limited
}
elseif ($method === 'GET') {
    Auth::check(); // only the reporting is private
```



```
// ~20 aggregations for the dashboard
}
```

Two precautions make this tracking sound. Views are deduplicated by IP and day, so a reader who reloads ten times counts only once: the numbers don't inflate. And clicks, beyond the frequency threshold, get a neutral response instead of an error:

```
// SPW analytics.php:62 – beyond the limit, a neutral response: the client c
↪ an't tell "recorded" from "discarded"
echo json_encode(['status' => 'ok']); // not a 429
```

It's the same `analytics.php` that consumes the reactions of Chapter 20, turning them into “most-loved articles” and “reactions by type.” A site that counts its own readers keeps the data in-house (a concrete privacy advantage) and decides for itself what counts as a real visit. The price is one more public endpoint to defend from abuse, and it's the reason for the deduplication and the rate limit.

*
**

5. The Safety Net: the Backup, and SitoRuntime's Paradox

Here the three sites part ways in the most instructive manner. SPW has the gold pattern: the automatic backup is written **outside the document root**, where no web request can reach it. And there's a detail born of a real scar.

```
// SPW backup.php:192-213 – the backup goes outside the docroot; the fallback
↪ k defends itself at runtime
$outside = dirname(realpath(__DIR__ . '/../')) . '/db_backups_simonepizzi';
if (!is_dir($outside)) @mkdir($outside, 0700, true);
if (is_dir($outside) && is_writable($outside)) {
    $backup_dir = $outside;
} else {
    // Fallback inside the docroot: but clean-dist.js (postbuild) removes .d
    ↪ ata/ from the dist,
    // so the deny .htaccess committed in the repo does NOT reach the server
    ↪ → recreate it at runtime
    $backup_dir = __DIR__ . '/.data/backups';
    if (!is_dir($backup_dir)) mkdir($backup_dir, 0700, true);
    @file_put_contents($backup_dir . '/.htaccess', "Require all denied\n");
}
$filename = "auto_backup_" . date('Y-m-d_H-i-s') . '_' . bin2hex(random_byte
↪ s(8)) . ".sql"; // unguessable name
@chmod($backup_dir . '/' . $filename, 0600);
```

WARNING

The build can betray your static defense: defend yourself at runtime

The `.htaccess` with `Require all denied` that protects the backups folder is committed in the repo. But the build script (`clean-dist.js`) removes the `.data/` folder from the distribution, so as not to ship development databases to production. Side effect: the deny file never reaches the server. SPW recreates it at runtime, the first time it writes a backup. The lesson is general: a defense that lives in a static file is worth something only if that file gets to where it's needed; if your build pipeline touches it, the defense has to be reestablished at runtime. It always pays to ask what the build *removes*, not just what it adds.

SPW’s backup is also schedulable from an external cron without login, but protected by a secret compared in a timing-safe way, and *fail-closed*: if the secret isn’t configured, the cron branch is reachable only by a logged-in admin.

```
// SPW backup.php:156-166 – cron without login, but protected; denies if the
↪ secret isn't defined
if (!$is_admin && (empty($configured_secret) || !hash_equals($configured_sec
↪ ret, $secret))) {
    http_response_code(403); die("Accesso negato.");
}
```

SR has none of this. No backup, no export, no cron, and, as we saw in §3, no metrics.

WARNING

Cure without prevention: the site that doesn’t back itself up It’s the paradox at the heart of this chapter. SitoRuntime lived through a nighttime database crash and an emergency migration (Chapter 15), and carries around a WAL “emergency revert” script: it has the *cure*. But it doesn’t have the *prevention*: no automatic backup, no metric to warn it before things get worse. The site mapped precisely for its incidents is the one least equipped to see them coming and to recover. Having the emergency script but not the backup is like keeping the fire extinguisher and not the smoke alarm.

**
**

6. The Powerful Actions, and How They’re (Not) Protected

An admin area contains the site’s most dangerous levers, and the three sites protect them unequally.

SR has what you could call a **hidden console**: inside its `admin.php` live powerful actions, like an `ALTER TABLE` or the reconversion of all the images, reachable via `GET` by typing the URL. They’re protected by login alone, not by role, they have no CSRF protection (they’re GETs), and above all no button in the UI invokes them: whoever doesn’t know the action’s name doesn’t even know they exist. It’s maintenance done by hand, with no interface and no role boundary (the mechanics of those migrations are in Chapter 15).

DIS exposes the destructive resets in a panel, and surrounds them with confirmations. But the confirmations are only user experience:

```
// DIS Settings.tsx:155-163 – double CLIENT confirm, POST fetch WITHOUT a CS
↪ RF token
if (confirm('⚠ RESET EDIZIONE: cancellerà TUTTI i partecipanti/voti/audio...'))
↪ ) {
    if (confirm('ULTIMA CONFERMA: i dati saranno persi per sempre. Procedere?'))
↪ )) {
    await fetch('/api/reset_system.php', { method: 'POST',
        body: new URLSearchParams({ action: 'confirm_reset' }) }); // no CSR
↪ F
    }
}
```

WARNING

confirm() is not a security defense A double `window.confirm` reduces human error: it's useful against the careless click. But it doesn't stop a request forged by another site while the admin is logged in, because that request doesn't go through the browser's dialog. The defense against that vector is the CSRF token (Chapter 10), which is missing here. Confusing the confirmation with the protection is a common mistake: the first speaks to the honest user, the second to the attacker. You need both, and they do different things.

SPW is the most disciplined here too, but not free of blemishes. Its password change increments the `session_version` server-side, invalidating the other open sessions (Chapter 10); but the settings save accepts any key, without a list of the allowed ones. It's a low risk, because everything is behind the admin guard, but it's the kind of door worth closing before someone leans something sensitive against it.

*
**

7. Data Without a Consumer: the Table No One Reads

A subtler flaw, and all DIS's, concerns the contact form messages. They're saved in the `contacts` table, and then no one reads them: there's no panel that shows them, nor an endpoint that retrieves them. The only way the admin "sees" a message is the notification email that goes out at the moment of sending; the copy in the database stays there, written and never consulted.

WARNING

Collect and forget Persisting a piece of data without having a place to read it is a hidden cost, and for personal data it's also a risk. That table accumulates names, emails, and messages (with the IP address) with no application purpose and, above all, with no place to delete them when they ought to be deleted. The rule is simple and often ignored: if you save a piece of data, you must have a consumer that uses it and a way to dispose of it. A write-only table isn't an archive, it's a debt.

*
**

8. The Festival Is a Special Case of This Chapter

Everything we've seen (the frame with the guard, the dashboard, the settings as switches, the evaluation of applications) has a specialization in the festival module, which has panels of its own (listening to the participants' tracks, the master switches for registration and voting, the ranking). That dashboard is Chapter 19, and it should be read as the festival instance of the general pattern described here: the structure, the guard, and the backup belong to this chapter; the master switches and the contest KPIs stay there.

*
**

In Summary

The admin area is measured on two independent axes, and the three sites occupy them revealingly. SPW is high on both: an analytical dashboard and a declarative architecture, with the out-of-docroot backup acting as a safety net. SR is low on both: a mega-component that measures nothing and saves nothing, the paradox of the site that suffered the most without having equipped itself to see it coming. DIS is in the middle in an instructive way: the first one's structure, the second one's guard (and holes), and a text-based dashboard that proves how much you can measure with little. The chapter's lesson isn't "add more charts": it's that a good admin lets you *see* the state of the system and gives you the *safety net* for when something goes wrong. The apparatus matters less than the question it answers.

IMPORTANT The Canon

- A single guard for the entire restricted area (route guard or check on mount), with a **role** verification, not just login.
- An automatic backup outside the docroot, with a cron protected by a timing-safe secret and fail-closed; recreate at runtime the defenses the build strips.
- Powerful actions go through POST + a CSRF token, not hidden GETs; no write-only table (saved data has a consumer and a way to delete it).
- Measure what you need to decide: even a text-based dashboard is enough.

*
**

Next Chapter: Database Evolution, from SQLite to MySQL. The February night a database went down, and the emergency migration told hour by hour.

PART V

The Real-World Cases

Where theory meets production. Patterns extracted from real projects, with their scars.

CHAPTER 15: Database Evolution — From SQLite to MySQL

On February 23, 2026, at 2:25 a.m., a SitoRuntime commit carries a terse message: “Risolto crash server e ottimizzate performance con switch SQLite WAL->DELETE” (Fixed server crash and optimized performance by switching SQLite WAL→DELETE). The next day, the 24th, the site abandons SQLite and moves to MySQL. The two dates, one day apart, tell the story of this chapter on their own: the migration wasn’t a planned upgrade, it was an escape.

It’s the difference that changes everything. You could describe the move from SQLite to MySQL as a growth threshold: more traffic, more writes, more contention on the file, and at some point you change engines. It’s a reasonable guideline, and further on we give it too. But it isn’t SitoRuntime’s real story, and telling it that way would lose the most valuable lesson. SitoRuntime migrated because it had hurt itself, and this chapter isn’t about “how you migrate” but about **what a migration leaves behind** and who’s equipped to survive the next bad night.

The three sites in the Model have three different relationships with the same technique. DISINTELLIGENZA runs **on SQLite to this day, in production, without ever having gone down**: it’s living proof that the problem wasn’t the engine. SimonePizziWebSite migrated to MySQL quietly, with no incident in git, and with a safety net underneath. SitoRuntime migrated on the run, at night, and left six fossils and no backup behind it. It’s the same reversal that runs through the whole book: the site that suffered the most is the one least prepared to suffer again.

*
**

1. The WAL Night: When the Optimization Is the Disaster

Before the migration, SitoRuntime was on SQLite and wanted to go faster. SQLite, on its own, can use a journaling mode called **WAL** (Write-Ahead Logging): faster on writes, because it queues the changes in a side file instead of rewriting the database right away. On paper it’s an optimization. In practice, on shared Apache/PHP hosting, WAL creates two service files (`.sqlite-wal` and `.sqlite-shm`) and needs a kind of locking that many shared hosts don’t handle well. The locks don’t release, the requests time out, the site goes down.

The treatment lives inside the code that explains it, and it’s twofold. One script forces the return to classic journaling:

```
// SitoRuntime optimize_db.php:17-22 – the "optimization" that triggered the
↳ crash, this is SQLite code
// WAL is performant but often causes locks on shared hosting.
// We set DELETE for maximum stability.
$pdo->exec("PRAGMA journal_mode=DELETE;"); // ← PRAGMA: a SQLite-only dire
↳ ctive
```

And a second script is the red button, to be pressed from the browser if the site is already timing out:

```
// SitoRuntime emergency_revert_wal.php:19-43 – the first-aid kit
$result = $pdo->query("PRAGMA journal_mode=DELETE"); // removes the .wal/.
↳ shm files
$mode = $result->fetchColumn();
if (strtoupper($mode) === 'DELETE') echo "SUCCESSO: ...ripristinato...";
$pdo->exec("VACUUM;"); // rebuild of the DB
↳ file
// in the catch: "potresti dover eliminare manualmente i file .wal e .shm vi
↳ a FTP"
```

WARNING

When the optimization is the cause of the disaster WAL isn't a mistake: in many contexts it's the right choice. But on shared hosting, where the filesystem and the locking are outside your control, touching a file-based database's `journal_mode` is a high-risk change disguised as an improvement. It's the reason Chapter 3 prescribes `journal_mode=DELETE` and not WAL: it isn't an aesthetic preference, it's a lesson paid for in production, at night. The general rule: an optimization that changes how the engine writes to disk has to be treated as a risky change, with a backup first and a rollback plan ready, not as a switch to flip and forget.

The correct diagnosis isn't "SQLite can't take it." It's "touching WAL on shared hosting can't take it." The proof comes from the site next door.

*
**

2. DISINTELLIGENZA: SQLite Alive, in Production, Without Scars

While SitoRuntime fled, DISINTELLIGENZA stayed. It runs today on SQLite, with the database in a file inside `.data/`, on the same kind of shared hosting, and it has never gone down. Everything that in SitoRuntime is an inert fossil (`PRAGMA`, `sqlite_master`, `AUTOINCREMENT`, the file-based database) is here the real, current engine. The difference isn't the engine: it's that DISINTELLIGENZA never touched the journal mode to squeeze out performance it didn't need, and it makes a `.bak` backup before every destructive operation (Chapter 10).

This qualifies the practical rule rather than refuting it. SQLite stays perfect for light-weight or in-development projects: zero configuration, a single file, a deploy in seconds. The threshold beyond which it's worth considering MySQL does exist, and it's recognized by concrete signals more than by a number:

- **frequent concurrent writes:** more users writing at the same instant means more contention on the file lock;
- **hosting that limits or blocks SQLite** for internal policy reasons;
- **a need for external access to the database**, for example from phpMyAdmin or visual management tools, which talk to a server, not to a file;
- **complex queries on large tables**, where MySQL handles joins and aggregations better.

As a rough line, as long as a site stays under a few dozen writes an hour and shows no recurring locks, SQLite is enough for it. DISINTELLIGENZA is in that band and sits there comfortably. SitoRuntime left it not because it had crossed the traffic threshold, but because a botched optimization had made it want to no longer have a file-based database. They're two different roads to MySQL, and only one of them is the textbook one.

*
**

3. Evolving a Schema Without Migration Tools

There's a trait the three sites share, and it's the real context of the whole chapter: **none of them has a migration tool**. No migrations/ folder, no schema_version table, no runner that keeps track of what's already been applied. The schema evolves with a single tool, the same everywhere.

The tool is ALTER TABLE ... ADD COLUMN inside a try/catch, made idempotent. You add the column, and if the error says it already exists you treat it as a success. On MySQL the signal is the Duplicate column code:

```
// SitoRuntime migrate_status.php:6-16 - the micro-migration that heals itse
↪ if
$pdo->exec("ALTER TABLE news ADD COLUMN status VARCHAR(20) NOT NULL DEFAULT
↪ 'published'");
// ...
} catch (PDOException $e) {
    if ($e->getCode() == '42S21' || strpos($e->getMessage(), 'Duplicate colu
↪ mn') !== false) {
        echo "OK: colonna 'status' già presente.\n"; // re-running = no ha
↪ rm
    } else { echo "ERRORE: " . $e->getMessage() . "\n"; }
}
```

On SQLite the same philosophy changes dialect: you query PRAGMA table_info and add the column only if it's missing.

```
// DISINTELLIGENZA update_db_v0.4.2.php:10-17 - same pattern, SQLite dialect
$columns = $pdo->query("PRAGMA table_info(users)")->fetchAll(PDO::FETCH_COLU
↪ MN, 1);
if (!in_array('role', $columns)) {
    $pdo->exec("ALTER TABLE users ADD COLUMN role TEXT DEFAULT 'editor'");
    $logs[] = "Added 'role' column to users table.";
} else {
    $logs[] = "'role' column already exists in users table.";
}
```

The price of this simplicity is that **no file represents the current schema**. The version is implicit, written in the file names: DISINTELLIGENZA has a chain update_db_0_1_3 →

0_1_4 → v0.4.2 → v0.5.4, SitoRuntime has `apply_v291` and `apply_v293`. To know what a table really looks like today, reading a file isn't enough: you have to query the database, or reconstruct the history from the names and the order in which they were run.

WARNING

Without a registry, the schema's truth lives only in the database Versioning the schema with file names works as long as there's a single person who remembers the order. But there's no clean path to recreate the database from scratch: in SitoRuntime it would take `init_mysql.php` plus the `apply_v29x` chain applied in the right order, and the starting `init_db.php` is a fossil stuck at an old version. It's the thin stack's "schema-as-code" debt: the price to pay for not having adopted a migration system. If the project grows, a `schema_version` table with a list of applied migrations costs little and pays off at the first disaster recovery.

To this is added a delivery duality. The same migration often exists in two forms: as a throwaway file to upload and delete, and as a persistent action inside the admin console. SitoRuntime's `status` column, for example, lives both in `migrate_status.php` and as an `?action=apply_v291_status` action inside `admin.php`:

```
// SitoRuntime admin.php:459-469 - the same migration, but self-healing behi
↪ nd the login
if ($action === 'apply_v291_status' && $_SERVER['REQUEST_METHOD'] === 'GET')
↪ {
    try {
        getDB()->exec("ALTER TABLE news ADD COLUMN status VARCHAR(20) NOT NU
↪ LL DEFAULT 'published'");
        sendSuccess(['message' => "OK: colonna 'status' aggiunta..."]);
    } catch (PDOException $e) {
        if (strpos($e->getMessage(), 'Duplicate column') !== false) sendSucc
↪ ess(['/* ... */]);
        else sendError('Errore DB: ' . $e->getMessage(), 500);
    }
}
```

Two philosophies coexist: the script to FTP and then remove, and the maintenance console behind a login. They're both legitimate, but without a registry neither one tells you, just by looking at it, whether that migration has already run on a given database. They're GETs protected by `isLoggedIn` alone, not by role or a CSRF token: their harmlessness depends on being idempotent, not on a real barrier (it's the same gate hole seen in Chapter 14).

*
**

4. Switching Engines: the Three-Script Pattern

When the engine migration was decided, SitoRuntime carried it out with three dedicated scripts, each with a precise role. It's a hand-rolled ETL, with no tools, and it's a positive, citable pattern.

The first is the secrets file, kept out of version control:

```
// db_credentials.php - to be added to .gitignore RIGHT AWAY, never commit r
↪ eal credentials
<?php
```

```

return [
    'DB_HOST' => 'mysql.tuohoster.com',
    'DB_NAME' => 'nome_database',
    'DB_USER' => 'utente_mysql',
    'DB_PASS' => 'password_sicura',
    'DB_PORT' => 3306,
];

```

The second is the MySQL connector, which replaces the SQLite version of `db.php`:

```

// db.php (MySQL version) – the connection singleton
$config = require __DIR__ . '/db_credentials.php';
$dsn = sprintf("mysql:host=%s;dbname=%s;port=%d;charset=utf8mb4",
    $config['DB_HOST'], $config['DB_NAME'], $config['DB_PORT']);
self::$pdo = new PDO($dsn, $config['DB_USER'], $config['DB_PASS'], [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false, // native prepared statements:
    ↪ on SQLite this was the default, here it must be forced
    PDO::ATTR_TIMEOUT => 5, // SQLite read a local file; M
    ↪ ySQL is over the network, the timeout matters
    PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8mb4 COLLATE utf8mb4_unico
    ↪ de_ci"
]);

```

Three details of the move deserve attention. `EMULATE_PREPARES => false` enables native prepared statements, which on SQLite were implicit. The `charset=utf8mb4` in the DSN plus the `SET NAMES` ensure that emoji and accents survive. And `PDO::ATTR_TIMEOUT` appears for the first time: with a local file it wasn't needed, with a server over the network it's essential. What disappears, instead, are all the `PRAGMA` directives: `journal_mode`, `busy_timeout`, `foreign_keys` were SQLite language and on MySQL they make no sense.

The third script, `init_mysql.php`, recreates the schema from scratch in MySQL dialect; it's run once, protected by authentication and then removed. And the fourth, the real move, is the ETL: it opens two connections, copies table by table idempotently, and closes with an explicit check.

```

// SitoRuntime migrate_to_mysql.php:37-72,199-209 – two PDOs, idempotent cop
↪ y, count verification
$sqlite = new PDO("sqlite:" . $sqlitePath); // source: the file
$mysql = Database::connect(); // destination: MySQL
$rows = $sqlite->query("SELECT * FROM news ORDER BY id ASC")->fetchAll();
$stmt = $mysql->prepare("INSERT INTO news (id, slug, title, summary, content
↪ , ...)
    VALUES (?, ?, ?, ?, ?, ...)
    ON DUPLICATE KEY UPDATE title=VALUES(title), ...");
↪ // re-runnable
foreach ($rows as $r) { $stmt->execute([$r['id'], $r['slug'], ...]); $count++;
↪ }
// at the end, the cross-check table by table:
foreach (['news', 'users', 'subscribers', 'speakers', 'podcasts'] as $t) {
    $c = $mysql->query("SELECT COUNT(*) FROM $t")->fetchColumn();
    echo " $t: $c record in MySQL\n";
}

```

TIP

The thin stack's ETL: two PDOs and a COUNT To move data between two engines you don't need a dedicated tool: two PDO connections, a loop that copies with `ON DUPLICATE KEY UPDATE` (so the copy can be repeated without creating duplicates), and a

final `COUNT(*)` that confirms how many records reached the destination are enough. It's the minimal, readable, verifiable version of a data migration. The script should then be uploaded to the server only for as long as needed and deleted together with the `.sqlite` file: leaving it lying around is a risk, not a convenience.

★
★★

5. What Changes, Line by Line, Between the Two Engines

The move touches dozens of micro-decisions. The table sums them up.

Aspect	SQLite	MySQL
Connection	local file	network (host:port)
Engine directives	<code>PRAGMA journal_mode, busy_timeout, foreign_keys</code>	not applicable
Charset	configurable per file	<code>utf8mb4</code> for emoji and accents
Auto-increment	<code>INTEGER PRIMARY KEY AUTOINCREMENT</code>	<code>INT AUTO_INCREMENT PRIMARY KEY</code>
Boolean	<code>INTEGER (0/1)</code>	<code>TINYINT(1)</code>
JSON	stored as <code>TEXT</code>	native JSON type
Prepared statements	native by default	native with <code>EMULATE_PREPARES=false</code>
Concurrent writes	file lock (problematic)	row-level locking, handled by the daemon
Backup	copy of the <code>.sqlite</code> file	<code>mysqldump</code> or dedicated tools

The last row is the one that hurts SitoRuntime the most, and we'll get to it shortly.

★
★★

6. The Six Fossils: Repo Hygiene After a Migration

After the switch, the SQLite layer wasn't removed. In a MySQL repository there remain **six files written for an engine that no longer exists**: inert at best, broken at worst.

Fossil	SQLite mechanism	State on MySQL
init_db.php	AUTOINCREMENT, seed of the 24 speakers	creates the wrong types, partly broken
fix_users_table.php	sqlite_master, PRAGMA, datetime('now')	broken (PRAGMA gives a syntax error)
fix_newsletter_table.php	sqlite_master, PRAGMA, 4-column schema	broken and obsolete
setup_podcasts.php	AUTOINCREMENT	partly harmless
optimize_db.php	PRAGMA journal_mode, CREATE INDEX IF NOT EXISTS	broken
emergency_revert_wal.php	PRAGMA, VACUUM	inert (WAL doesn't exist on MySQL)

The most toxic fossil is the first. `fix_users_table.php` queries the schema with SQLite dialect, on a database that is now MySQL:

```
// SitoRuntime fix_users_table.php:12,20 - code that speaks the wrong engine
↳ 's language
$stmt = $pdo->query("SELECT name FROM sqlite_master WHERE type='table' AND n
↳ ame='users'"); // SQLite!
$stmt = $pdo->query("PRAGMA table_info(users)"); // on MySQL → syntax
↳ error
$pdo->exec("UPDATE users SET created_at = datetime('now') WHERE created_at I
↳ S NULL"); // datetime() = SQLite
```

The only safety net keeping them in check is the by-prefix `.htaccess` deny, which blocks HTTP execution of every script with a maintenance name:

```
# SitoRuntime public/.htaccess:32-35 - the only barrier against the fossils
RewriteRule ^(debug_|test_|emergency_|migrate_|fix_|init_|rebuild_|setup_|op
↳ timize_) - [F,L]
```

WARNING

An engine migration has to be finished by deleting, too Those scripts are still on the server and still in the repo. They produce three harms: noise (whoever opens the project doesn't know which files count), confusion about the schema's truth, and a latent attack surface, because if one day the `.htaccess` deny dropped, they'd be powerful, reachable endpoints. Migrating engines isn't done when the data has moved: it's done when the old engine's code has been removed. Keeping the fossils "to be safe" is the opposite of safe.

✱
✱✱

7. One Table, Three Different CREATES

The “no file represents the schema” debt has a concrete face in SitoRuntime's `subscribers` table, which in the code exists in **three divergent definitions**:

1. `init_mysql.php` creates it with four columns, the base schema, predating double opt-in.

2. `fix_newsletter_table.php`, the SQLite fossil, recreates the same four columns, and it's broken on MySQL and insufficient anyway.
3. `apply_v293_newsletter`, inside `admin.php`, is the **real** migration: it adds the double opt-in columns (`confirmation_token`, `confirmed_at`, `subscribed_at`, `subscribed_ip`) and retroactively confirms the historical subscribers.

Only the third is aligned with the runtime. The newsletter code (Chapter 13) **assumes** the extended schema, so on a database that has only the base schema every query fails until `apply_v293` runs. It's an undeclared ordering dependency: the right schema exists, but only if you've run the right migration, which no file tells you to run.

WARNING

Where a table's truth lives When the same table has three `CREATE`s scattered across three files, and only one is the good one, the source of truth isn't the code but the running database. It's a symptom of a schema versioned by file names: the correct definition exists, but it's hidden in the chain of migrations instead of in a single place. You pay for it the day someone recreates the database from the wrong file and everything looks fine until the runtime starts failing.

*
**

8. The Date-as-String Bug

Migrations leave scars in the data too, not only in the schema. SitoRuntime keeps `debug_time.php`, the diagnostics of a timezone incident. The article visibility rule compares `published_at <= now` **as strings**: if the server's timezone and the date format don't match, published articles vanish from the site.

```
// SitoRuntime debug_time.php:23-24 – the "fix" that never reached productio
↳ n
// FIX: Use T separator to match DB
$now = date('Y-m-d\TH:i:s'); // 'T' separator
```

The revealing detail is that this fix was never applied to the runtime. The production code (`news.php`) uses the space, not the `T`, because the space is the correct format for MySQL's `DATETIME`. The “solution” with the `T` stayed only in the diagnostic file, a record of a past bug and of a correction the real site, in the end, didn't adopt.

TIP

Comparing dates as strings is a bug in waiting Two dates compared as text match only if they have exactly the same format and the same timezone. It takes just a `T` in place of a space, or a server set to UTC while the content is written in local time, for a “published” article to stay invisible. The defense is to force the timezone explicitly (`date_default_timezone_set('Europe/Rome')`) and compare values in the same format as the database's date type, or to delegate the comparison to the database with `NOW()`.

*
**

9. The Cure Without the Prevention

What remains is the most painful row of the §5 table: the backup. SitoRuntime has the defibrillator, `emergency_revert_wal.php`, the script that revives the site after the incident. But it doesn't have the check-up: no automatic backup, no cron, no scheduled `mysqldump`. The only safety net is a `_BACKUP_BEFORE_OPTIMIZATION/` folder, a manual snapshot of the project committed to the repo before the risky operation. It's a gesture, not a system, and on top of that it's versioned noise.

The contrast with the other two is sharp. SimonePizziWebSite has the automatic backup written outside the document root, with rotation and a protected cron (Chapter 14). DISINTELLIGENZA makes a `.bak` before every destructive reset (Chapter 10). If SitoRuntime's MySQL corrupts, there's no recent dump to start over from.

WARNING

Having the defibrillator but not the fire alarm It's the paradox at the heart of the chapter. The site mapped precisely for its incidents is the one without the most basic safety net: a backup. It learned to *cure* the disaster, the emergency script is proof that the pain was real, but it didn't learn to *prevent* it. Curing an emergency is reactive: it keeps you alive this time. An automatic backup is preventive: it keeps you alive for the next one, the one you didn't see coming. The checklist below prescribes a backup, and it's the ought-to-be; SitoRuntime tells you what happens when you skip it.

**

10. Migration Checklist

The sequence, in order, for a SQLite → MySQL move done right. The first point is also the one SitoRuntime didn't have: it isn't a detail, it's the safety net.

- ☐ Make a full backup of the production `.sqlite` file, and archive it outside the repo.
- ☐ Create the MySQL database on the hosting provider.
- ☐ Upload and run `init_mysql.php` to create the schema, then remove it.
- ☐ Upload the `.sqlite` and `migrate_to_mysql.php` to the server.
- ☐ Run `migrate_to_mysql.php` and verify the counts in the summary.
- ☐ Replace `db.php` with the MySQL version and upload `db_credentials.php` with the real credentials.
- ☐ Add `db_credentials.php` to the `.gitignore` (before the project's first commit).
- ☐ Test all the APIs in production, including those that assume migrations beyond the base schema.
- ☐ Delete `migrate_to_mysql.php` and the `.sqlite` file from the server.
- ☐ **Remove the old engine's fossils from the repo** (`init_db.php` SQLite, `fix_*`, `optimize_db.php`, `emergency_revert_wal.php`): the migration isn't finished while they remain.
- ☐ Configure an automatic MySQL backup, outside the document root: the cure doesn't replace the prevention.

*
**

In Summary

The engine migration isn't a scaling rung everyone climbs sooner or later: it's an event, and the *why* matters as much as the *how*. SitoRuntime fled SQLite after a bad night, and brought MySQL along without cleaning house, leaving behind six fossils, three versions of the same table, and no backup. DISINTELLIGENZA proves that SQLite in production holds up just fine, until you ask it for an optimization it can't deliver on its own ground. SimonePizziWebSite made the same journey quietly, with a safety net underneath. The lesson isn't "migrate to MySQL" and it isn't "stay on SQLite" either: it's that every schema evolution, and even more every engine change, has to be carried all the way through, with a registry of what you did and a backup of what you had before. What a migration leaves behind weighs more than what it carries forward.

IMPORTANT The Canon

- Migrate engines only for a concrete constraint (recurring locks, hosting, remote access), not out of superstition; make the backup **first**, outside the repo.
- An idempotent ETL (ON DUPLICATE KEY UPDATE) with count verification at the end of the move.
- Once the migration is done, remove the old engine's fossils from the repo.
- A single definition for each table and a schema registry (`schema_version`); the emergency cure doesn't replace prevention, that is, the automatic backup.

*
**

Next Chapter: Portfolio & Projects Module. The universal module for portfolios and showcases, with drag-and-drop reordering and switch-based visibility.

CHAPTER 16: Portfolio & Projects Module

The Portfolio module is an entity distinct from News/Article, meant for a personal site, an agency, a showcase. Mapped onto **SimonePizziWebSite**, it brings patterns of its own: granular visibility, manual ordering, multiple action buttons, category-based management. It's the reference for any site that has to display a catalog of works, products, or projects.

1. The Difference from the News/Articles Module

Feature	News/Articles	Projects/Portfolio
URL identifier	slug (human-readable text)	id (numeric)
Visibility	status (draft/published)	is_visible (boolean)
Time scheduling	published_at	not provided
Rich-text body	yes (HTML)	optional (short description)
Ordering	by date (automatic)	sort_order (manual)
CTA	none	button_a + button_b (external URLs)
Categorization	category + tag	category only

The row on the identifier matters more than it seems: articles live on human-readable URLs (slug), projects on a numeric id. Projects, then, generate no slug, and the advanced slug logic (with its map of Italian accents) lives only once, in Chapter 5, where the articles need it.

2. Database Schema

```
CREATE TABLE IF NOT EXISTS projects (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  name        TEXT NOT NULL DEFAULT 'Nuovo Progetto',  
  description TEXT DEFAULT '',  
  category    TEXT NOT NULL DEFAULT 'progetti-software',  
  cover_image TEXT DEFAULT '',  
  button_a_label TEXT DEFAULT 'Scopri',  
  button_a_url  TEXT DEFAULT '',  
  button_b_label TEXT DEFAULT '',  
  button_b_url  TEXT DEFAULT '',  
  is_visible   INTEGER NOT NULL DEFAULT 1,    -- 1=visible to the public,  
  ↪ 0=hidden  
  sort_order  INTEGER NOT NULL DEFAULT 0,    -- manual ordering within a  
  ↪ category  
  created_at  DATETIME DEFAULT (datetime('now'))  
);
```



```
CREATE INDEX IF NOT EXISTS idx_projects_category ON projects(category);
CREATE INDEX IF NOT EXISTS idx_projects_sort_order ON projects(sort_order AS
↵ C);
```

NOTE

A note on dialect. The schema above is in the SQLite dialect, the book's base language (Chapter 3). SimonePizziWebSite today runs on MySQL (Chapter 15), and there the same constructs change shape: `INTEGER PRIMARY KEY AUTOINCREMENT` becomes `INT AUTO_INCREMENT PRIMARY KEY, DEFAULT (datetime('now'))` becomes `DEFAULT CURRENT_TIMESTAMP`, and the `INTEGER` boolean becomes `TINYINT(1)`, exactly as in the reactions schema in Chapter 20. Translated badly, `datetime('now')` doesn't run at all on MySQL: it's one of the fossils Chapter 15 collects.

3. The projects.php API: All Five Verbs

The module uses the full range of HTTP methods, and `PATCH` is what sets it apart: it's the right verb for the visibility and reordering operations, which change a single field.

GET: list with admin bypass. The public sees only the visible ones; the admin sees everything. Same pattern as Chapter 9, here on `is_visible` instead of `status`.

```
$is_admin = isset($_SESSION['user_id']);
if (!$is_admin) $conditions[] = "is_visible = 1";           // the public see
↵ s only the visible ones
if ($category) { $conditions[] = "category = ?"; $params[] = $category; }
$query .= " ORDER BY category ASC, sort_order ASC, created_at ASC";
```

POST: creation with auto-sort. On creation, `sort_order` becomes `MAX(sort_order) + 1` within the same category, so the new project shows up at the bottom of its list.

```
$stmtMax = $pdo->prepare("SELECT COALESCE(MAX(sort_order), 0) FROM projects
↵ WHERE category = ?");
$stmtMax->execute([$category]);
$sort_order = (int)$stmtMax->fetchColumn() + 1;
```

PATCH: partial updates. It doesn't send the whole object, only the field that changes: the visibility toggle, or the new `sort_order` arriving from the frontend's drag-to-sort.

```
if (isset($data['is_visible'])) {
    $pdo->prepare("UPDATE projects SET is_visible=? WHERE id=?")->execute([(
↵ int)$data['is_visible'], $id]);
}
if (isset($data['sort_order'])) {
    $pdo->prepare("UPDATE projects SET sort_order=? WHERE id=?")->execute([(
↵ int)$data['sort_order'], $id]);
}
```

The HTTP semantics are clear: `POST` creates, `PUT` replaces the whole object, `PATCH` modifies a piece. Using `PATCH` for the toggle and the reordering communicates intent better than a generic `POST` would.

4. The CTA Buttons (button_a / button_b)

Each project can have up to two buttons pointing to external resources: one primary, whose default label is “Scopri” (Discover) but which an author can rename to “Visit the Site” or “Play Now,” and one optional secondary (“GitHub,” “Case Study,” “App Store”).

```
{project.button_a_url && (
  <a href={project.button_a_url} target="_blank" rel="noopener noreferrer" c
↵ lassName="btn-primary">
    {project.button_a_label || 'Scopri'}
  </a>
)}
{project.button_b_url && (
  <a href={project.button_b_url} target="_blank" rel="noopener noreferrer" c
↵ lassName="btn-secondary">
    {project.button_b_label}
  </a>
)}
```

The `rel="noopener noreferrer"` on `target="_blank"` links is mandatory: it stops the destination page from reaching the `window.opener` of the originating one (tabnapping).

4.1 The Web / Email Switch

A refinement introduced in the CTA handling (SimonePizziWebSite v1.7.x) is a “link type” toggle in the editor: a button often doesn’t point to a website but has to open the mail client. If the author picks “Email,” the editor prepends `mailto:` to the saved string on its own, ignoring the `https://`. It’s a small, distraction-proof piece of UX: the content editor doesn’t have to remember the right protocol.

5. Unified Search: One Endpoint for Articles and Projects

Articles and projects are different entities, but to someone searching the site they’re the same thing: content. SimonePizziWebSite recognizes this with a single search endpoint, `search.php`, which queries both tables with a `LIKE` and tags every result with a `type` field so the frontend knows how to render it.

```
// SPW search.php – one query per family, results merged and tagged with `ty
↵ pe`
$like = '%' . $q . '%';
$articles = $pdo->prepare("SELECT id, title AS name, slug, 'article' AS type
↵ FROM articles
                                WHERE status='published' AND (title LIKE ? OR con
↵ tent LIKE ?)");
$projects = $pdo->prepare("SELECT id, name, NULL AS slug, 'project' AS type
↵ FROM projects
                                WHERE is_visible=1 AND (name LIKE ? OR descriptio
↵ n LIKE ?)");
// ...both run, the results are concatenated, and the client sorts by `type`
```

It’s a search that’s honest about its limits: `LIKE '%q%'`, not a full-text engine. For a portfolio or a personal blog it’s more than enough, and the `type` field saves you from building two separate searches in the frontend. This endpoint lives halfway between the content lifecycle (Chapter 9) and this module: it’s the point where the two entities go back to speaking the same language.

6. React Frontend: The Key Components

- **PortfolioGrid.tsx**: the public grid, which filters by category on the client, shows the cover images with lazy loading, renders the two CTAs conditionally, and the category badges.
- **ProjectEditor.tsx** (admin): image upload via `MediaPicker` (Chapter 8), the two label+URL button pairs, the `is_visible` toggle, the category from a dropdown.
- **ProjectsList.tsx** (admin): drag-to-sort that sends a `PATCH` on every repositioning, a visibility toggle with an instant `PATCH` (the open/closed eye icon), a category filter.

7. Category Strategies: From Static to DB-Driven

Up to v1.6, the categories were fixed strings in the React code (`PROJECT_CATEGORIES`). A growing portfolio needs more freedom, and from v1.7.10 the categories become DB-driven: a `categories` table the frontend queries on startup (`GET /api/categories.php`), so the admin can rename them or add new ones from the panel without a fresh Vite build.

Many-to-many multi-tagging, on the other hand, stays a feature of the **articles**, not the projects (which have a single `category`): its treatment, with the dual track toward the legacy CSV field, is in Chapter 9. Here the lesson on availability is enough: moving the categories out of the code and into the database makes them editable on the fly, and for an editorial catalog that’s what counts.

NOTE

The `auth_helper.php` pattern Like every protected endpoint, `projects.php` too leans on the `auth_helper.php` that wraps `session_start()`, the JSON headers, and the `Auth` class (the detail is in Chapter 5). Concentrating those calls in a single include, instead of repeating them in every file, cuts down on “headers already sent” errors.

IMPORTANT The Canon

- A `projects` table with `sort_order`, `is_visible`, and the CTA fields; an endpoint with `PATCH` for the visibility toggle and reordering.
- Project URLs on a numeric `id`, not a slug; a single category (M:N multi-tagging belongs to the articles, Chapter 9).
- Unified search with a `type` field that sorts the results on the client.

*

**

Next Chapter: Festival Logic — Submissions and the Approval Workflow. The competitor-management cycle for DISINTELLIGENZA and FDCA.

CHAPTER 17: Festival Logic – Submissions & Approval Workflow

A word of honesty before the module. The “festival” isn’t part of the miniCMS core: it’s an **optional module**, present in **one site out of four** (DISINTELLIGENZA), which FDCA inherits unchanged because it’s a fork with a byte-identical backend. The three chapters that follow (this one, the voting, the dashboard) describe a real public-vote contest, with its strengths and its cracks, not a standard component to take for granted. This first chapter covers the front door: how an entrant signs up, how the admin evaluates them, and the two points where the idealized module diverges from the code (the newsletter sign-up and the public upload of the tracks).

1. The Participant Workflow

A submission follows a three-state pipeline, run not by a state machine but by a `status` column that the admin moves forward.

- **pending**: the initial state. The entrant has sent their data and their track, but isn’t visible on the site.
- **approved**: validated. They get the confirmation email, enter the contest, and (see §4) get signed up to the newsletter.
- **rejected**: discarded, with a courtesy notice.

There’s a security detail the “clean” workflow hides, though: the action that changes the state (`update_status`) is protected only by a session check, not a role check. It means that **an editor too**, not just an administrator, can approve or reject entrants.

WARNING

The gate that confuses “logged in” with “admin” Approving an entrant is a weighty decision: it lets them into the contest, sends them an email in your name, signs them up to the newsletter. And yet the backend grants it to anyone who’s logged in, without checking that they’re an administrator. It’s the same “role-blind gate” that runs through several points of the site (Chapter 10): hiding a menu item from an editor is user experience, but stopping them from taking an action is security, and that has to be done on the role, server-side. Here it isn’t.

2. The Transactional Emails

On every state change the backend sends an email with an HTML template consistent with the festival’s branding: a technical confirmation on receipt, and a formal message

(positive or negative) on the outcome. The transport is native `mail()`, *fire-and-forget*: the record in the database is the source of truth, the email is a best-effort channel (the full mechanics, with its limits, are in Chapter 13).

3. Participant Assets: The Public Upload That Opens a Door

The audio tracks uploaded by the entrants land in an isolated folder (`uploads/audio/participants/`) with a unique name, and the admin pre-listens to them in the Media Center before deciding. So much for the convenient version. The real version is that this upload, to lower the friction of signing up, **requires no login**, and it's the front from which the RCE chain described in Chapter 7 starts.

WARNING

The public upload of the tracks changes all the rules A submission form that accepts files without authentication is an attack surface open to the internet. In DISINTELLIGENZA four weaknesses add up: the upload is public, the validation trusts the Content-Type the browser declares, the file name keeps its extension, and the folder doesn't turn off PHP. The result, verified, is remote code execution (the full chain, link by link, is in Chapter 7). The lesson for anyone building a frictionless sign-up: opening to the public is a legitimate product choice, but it has to be paid for with the defenses that opening demands (validate the real bytes, neutralize the name, turn off PHP in the folder), not with their absence. FDCA, being a fork, inherited the same open door intact.

4. Newsletter Sign-Up on Approval

On approval, the entrant's address is inserted into the newsletter table with an `INSERT OR IGNORE`. The idealized module presents it as a "growth strategy" for the marketing database, one that would guarantee a list of only real, validated users. That's the framing to flip.

WARNING

Signing up for a contest isn't consenting to marketing Whoever submits their track consents to taking part in the festival, not to receiving a newsletter: these are two different legal bases (Chapter 13). Signing approved entrants up to the mailing list by default, without explicit and separate consent, is a GDPR compliance problem, not a product strength. The comments in the source give away the developer's own doubt. The fix is simple and points in the opposite direction from the "convenient" one: a distinct marketing consent, opt-in, collected at the moment of sign-up and recorded; approval for the contest must not drag the newsletter sign-up along with it as a side effect.

IMPORTANT The Canon

- A pending → approved/rejected workflow with a `registration_active` master switch.

- The public upload of the tracks is the front of the RCE chain: gate it and validate it like every upload (Chapter 7).
- Approval is an admin action: gate by **role**, not just by login (no role-blind gate).
- Signing up for a contest isn't consent to marketing: no newsletter sync without explicit, separate opt-in.

*
**

Next Chapter: Festival Logic — Voting & Anti-Fraud Protection. The public vote, the defenses that truly count, and the merely cosmetic ones.

CHAPTER 18: Festival Logic — Voting & Anti-Fraud Protection

The vote is the heart of the contest, and it's also the point where a seemingly robust module reveals which of its defenses truly count and which are only decoration. This chapter tells them apart, and makes plain two fragilities the idealized text kept quiet about: the ranking can drift in silence, and the reset isn't protected.

1. The Voting Session

A visitor can cast one to three preferences in a single call. The backend validates that every participant voted for is really in the running (`status = 'approved'` and `in_current_round = 1`), then records the vote in a transaction that does two things at once: it inserts the row in `votes` and increments the counter on the participant.

```
// every vote is a transaction: INSERT + increment of the denormalized count
< er
$pdo->beginTransaction();
$pdo->prepare("INSERT INTO votes (participant_id, ip, user_agent) VALUES (?,
<  ?, ?)")
    ->execute([$pid, $ip, $ua]);
$pdo->prepare("UPDATE participants SET vote_count = vote_count + 1 WHERE id
<  = ?")->execute([$pid]);
$pdo->commit();
```

That `vote_count` isn't a convenience: it's the **source of truth for the ranking**. The participants are ordered by `vote_count`, not by counting the rows in `votes`. It's a reasonable performance choice (the ranking becomes a `SELECT ... ORDER BY vote_count`), but it comes at a price that §5 brings into focus.

2. The Anti-Abuse Defenses, in Order of Real Effectiveness

The module presents three defenses against multiple voting. The point, though, is that they're not equivalent at all: one holds, one is cosmetic, one defends nothing. Telling them apart matters, because confusing them gives a false sense of security.

- **The real barrier: IP for twenty-four hours.** The backend records the voter's IP address and refuses new votes from the same IP for the next twenty-four hours. It's the only server-side defense an attacker can't bypass from their own browser.
- **The cosmetic defense: the cookie.** After the vote, a cookie (`dis_voted`) is set with a thirty-day expiry. It lives on the client, so anyone can delete it, open a private window or switch browsers, and vote again. It improves the honest user's experience (it reminds them they've already voted); it doesn't stop someone bent on cheating.

- **No defense, just a record: the User-Agent.** Every vote stores the `User-Agent`, but it isn't used to block anything: it serves only for possible after-the-fact analysis.

WARNING

Three defenses, only one counts Listing cookie, IP, and User-Agent as if they were three equivalent locks is a common and dangerous mistake, because the reader believes they have a three-layer defense when they have only one. The cookie and the User-Agent are under the client's control: the first gets deleted, the second gets spoofed. The only barrier that lives on the server, and that can therefore really limit abuse, is the IP-based one. Knowing which defense actually holds is what keeps you from leaving a contest defenseless while believing it's armored.

On the IP there's a counterintuitive observation (Chapter 10): the module uses the raw `REMOTE_ADDR`, not a helper that reads the forwarding headers. For authentication behind a proxy that would be a flaw, but for a public vote it's a **strength**: the `X-Forwarded-For` header is written by the client and gets spoofed, while `REMOTE_ADDR` doesn't. The flip side is NAT collision: behind a single corporate or university network, many legitimate voters share an IP and block one another.

NOTE

The privacy counterpoint: the hashed identity of the reactions The festival vote saves the IP and User-Agent **in cleartext** in the `votes` table: personal data persisted without obfuscation. The article reactions (Chapter 20) face the same problem more carefully, deriving a `SHA256(IP+UA)` pseudonym instead of saving the bare IP. Neither one is perfect anonymity (that hash is reversible, see Chapter 20), but it's the difference between "I obfuscated the data" and "I kept every voter's address in cleartext." For a module that collects votes from the public, saving the raw IP is a choice to weigh on the GDPR plane too, not just the anti-fraud one.

3. The On/Off Rounds

A participant shows up on the voting page only if they're approved **and** `in_current_round = 1`. The phases of the contest (heats, semifinals, final) aren't entities with a history: they're this flag, switched on or off by the admin on groups of participants. It's the module's simplicity, and also its limit.

WARNING

reset_votes erases the round's history Advancing the contest means zeroing out the round's votes and switching the flag back on for the next group. But `reset_votes` **deletes** the previous round's votes and resets the counter to zero: there's no per-round archiving, so the results of the heats vanish, except for the `.bak` copy the system makes before destructive operations (Chapter 10). Managing the phases with a single boolean flag is convenient, but if you want to keep each round's results you have to archive them before the reset: the module, on its own, doesn't.

4. The Voting Master Switch

The ability to vote is governed by a global switch (`voting_active`) in the `settings` table: if it's off, the backend refuses every vote with a 403. The `settings` table is publicly readable (the frontend consults it to show or hide the form) and writable only by the admin.

One detail reveals a small internal inconsistency: the reading of the flag accepts both `'1'` and `'true'`, because one spot in the code saves booleans as `'1'` and another as the string `'true'`. The defensive read (`=== '1' || === 'true'`) makes up for the fact that the write never settled on a convention. It works, but it's the symptom to recognize: when the reader has to guess how the writer wrote, there's an agreement missing upstream.

5. The Ranking That Can Drift

The subtlest fragility remains, and the most important one for the fairness of the contest. Because the ranking is ordered by `vote_count` (the denormalized counter from §1) and not by the real count of the rows in `votes`, the two values can diverge. The transaction keeps them aligned in normal operation, and the reset zeroes them together, but there's no **reconciliation**: no periodic check verifies that `vote_count` really matches the number of votes recorded.

WARNING

Denormalizing a counter: speed against truth Keeping the vote total in a column makes the ranking instant, but it also makes it fragile: a transaction interrupted halfway, a manual import, a correction made by hand on the database, and the counter no longer matches the real votes. The trouble is that the divergence is **silent**: the ranking shows an order that looks right, and no one notices it's wrong until someone counts the votes by hand. When a denormalized number decides an outcome (a ranking, a prize), you need a reconciliation query that periodically compares the counter against `COUNT(votes)` and flags the gaps. The module doesn't have one, and it's the first thing to add before entrusting it with a real contest.

There's the reset itself, finally: the actions that zero out votes and counters (`reset_votes`, `reset_system`) are powerful and destructive, but they have **no CSRF protection** (Chapter 10). All that surrounds them is a browser confirmation, which stops the careless click but not a request forged by another site while the admin is logged in. The pre-destruction `.bak` copy is the safety net that softens the damage; the defense that's missing is the token.

IMPORTANT The Canon

- The real anti-fraud is the IP constraint plus a time window (24h); the cookie is cosmetic, the User-Agent only a hint.
- For a public vote the raw IP is a strength against spoofing (unlike auth behind a proxy, Chapter 10).
- If you denormalize a counter (`vote_count`), plan for periodic reconciliation against `COUNT(votes)`, or you accept the silent drift.

- Destructive resets go through a CSRF token plus the .bak copy; the browser confirmation isn't enough.

**

Next Chapter: Festival Logic — Admin Dashboard, Settings & Reporting. The contest's control panel, and the final report that was never switched on.

CHAPTER 19: Festival Logic – Admin Dashboard, Settings & Reporting

This dashboard is the contest’s command board, but it isn’t a console of its own: it’s the **festival specialization** of the general admin area of Chapter 14. The structure that holds it up (the guard that protects the area, the layout, the backup outside the docroot) belongs to that chapter; what stays here is what’s specific to the festival, the phase switches, the contest KPIs, the approval, the ranking, and the report. Reading this chapter as the festival instance of a general pattern, and not as an isolated panel, keeps you from duplicating what Chapter 14 has already explained.

And like the other two chapters of the module, this one too lines the idealized text up against the real code, where two promised functions aren’t what they seem: the final report and the finalists.

1. The Master Switches

The `settings` table, a key/value pair, is the festival’s fuse box: `registration_active` opens or closes the sign-up form, `voting_active` the voting session, `current_round` indicates the phase. One switch here opens or closes a whole phase for everyone.

The table is publicly readable (the frontend consults it to know what to show) and writable only by the admin. A detail already met in Chapter 18 comes back here: the reading of the flags accepts both `'1'` and `'true'`, because the write never settled on a single convention. It’s a defensive patch that makes up for an inconsistency upstream, not a design.

2. The Contest KPIs

The area shows the indicators in real time: the participants broken down by state (pending, approved), the total volume of votes, an estimate of unique voters. On that last one a caveat is needed, because the module calculates it “by IP and cookie.” The cookie, though, is a cosmetic defense (Chapter 18): it gets deleted, so it counts for little as a measure of uniqueness. The reliable signal is only one, the IP within the twenty-four-hour window; the “unique voter” should be read as “unique IP,” with all the imprecision that entails (the corporate network that collapses many voters onto a single address).

3. Approval and Ranking

The admin sees the participants in a table, listens to the track, and decides the outcome; approval is the only action that fires off the official confirmation email. That same

action, though, is granted to anyone who's logged in, not just to administrators: the role-blind gate of Chapter 17 applies from here too.

The ranking is ordered by `vote_count`, and it's fast precisely because it reads a ready-made counter instead of counting the votes. But it pays for that with fragility: without a periodic reconciliation, that counter can diverge in silence from the real votes (the box in Chapter 18 explains why, and what to add). Then there's a promise the text makes that the code doesn't keep.

WARNING

The `finalist` state tells of a plan never carried out The idealized module talks about “selecting the finalists to move into the next round,” as if there were a `finalist` state the admin assigns. In the participants' enum that state exists, but **no line of code ever sets it**: the contest's phases are managed by switching on the boolean flag `in_current_round` on groups of participants, not by promoting them to a state. That `finalist` is a schema documenting an abandoned intention, not a function. It's worth knowing how to spot it: a state that exists in the database but that no one writes is a fossil, and describing it as active confuses the reader about how the contest really runs.

4. The Reporting That Was Never Switched On

At the close of voting, the idealized module says, the backend sends the staff a final report email: total votes, the Top 20 most voted, some geographic statistics on the IPs. The function exists, it's called `sendVotingReport`, and it's written in full. There's just one problem.

WARNING

A feature built and disabled: “Phase 2” `sendVotingReport` is fully implemented but **disabled in the code**, commented out with the label “Phase 2.” It's true that the backend *contains* the report; it's false that it *sends* it. It's the twin of the `finalist` state from the previous paragraph: code present, function dormant. Presenting a commented-out capability as operational is one of the easiest ways for documentation to fall out of step with the software: the rule, for anyone writing technical documentation, is to describe what the code *does*, not what it's *ready to do if someone reactivates it*. As long as “Phase 2” stays commented out, the final report is a promise, not a function.

In Summary

The festival panel is consistent with the module's philosophy, simple switches and essential numbers, but it carries the cracks of the “switch-built” approach: a uniqueness estimate that trusts a cosmetic cookie, a ranking that can drift without noticing, a `finalist` state never used, and a report never switched on. These are the points where the real contest diverges from its narrated version, and knowing them is what separates someone who can govern it from someone who only thinks they can. The structure that

holds this panel together (guard, layout, backup) stays in Chapter 14, of which this is the festival version.

IMPORTANT
The Canon

- It's the festival instance of the admin area (Chapter 14): structure, guard, and backup live there; what stays here are the contest's switches and KPIs.
- The master switches have to be read consistently with how they're written (no `'1'` versus `'true'`).
- Show honest KPIs (unique voters by IP, aware of the `vote_count` drift, Chapter 18).
- Don't pass off as active the features built but disabled, and remove the vestigial states (a `finalist` state never set).

*
** *

Next Chapter: Social Interactions & Reactions. The last surface of the CMS, where it's the anonymous public writing into the database.

CHAPTER 20: Social Interactions & Reactions

Almost everything in the CMS is restricted writing. Publishing an article, uploading a file, composing a newsletter: each of these actions lives behind `Auth::check()`. There are two exceptions, and this chapter is about them. The **reactions** to articles and the **messages** from the contact form are the only two surfaces where an unauthenticated visitor really writes into the database. They're the most exposed front of the site, and that's exactly why they concentrate the most sophisticated defenses SimonePizziWebSite has built.

A necessary premise: the reactions exist **only** in SimonePizziWebSite. SitoRuntime and DISINTELLIGENZA don't have them; the messages surface, on the other hand, returns elsewhere as a contact form. So the chapter is largely single-site, but its lens, *how the thin stack handles anonymous public writing*, touches threads that run through the whole book: the hashed identity and the anti-abuse (Ch. 10), the sanitization (Ch. 8), best-effort email (Ch. 13), the festival vote (Ch. 18).

And there's a standout worth the read: in the same codebase **two opposite philosophies** of sanitization live side by side. The content of the articles, written by an admin you trust, is saved raw and cleaned at the moment it's shown; the text of the messages, written by anyone who passes through, is cleaned at the moment it's saved. It isn't a contradiction: it's the right choice for each context, and the two look each other in the face right here.

*
**

1. Two Surfaces, One Principle: Defend at the Entrance

Beneath the differences, the two surfaces share five traits.

Both are endpoint-routers on `REQUEST_METHOD` with a **selective gate**: in `messages.php` the `POST` is public (anyone sends a message), while `GET`, `PUT`, and `DELETE` (list, mark-read, delete) go through `Auth::check()`; the reactions are entirely public, and have no admin branch at all. The identity of whoever writes is a **derived pseudonym**, not an account and not a cookie. Integrity is entrusted to the **database**, not just to an application-level `if`. Public input is cleaned **at write-time**. And everything degrades gracefully: the reactions fall to a zero count on any error (the article page never breaks), and the notification email is best-effort, because the truth is in the saved record, not in the email that went out.

The next sections take these five traits one at a time, starting with the one the module describes a little too generously: anonymity.

✱
✱✱

2. The Anonymous Identity, and Why a Hash Isn't Anonymity

The voter has no account. They're identified by a pseudonym computed on the fly from the two pieces of data the server sees on every request: the IP address and the User-Agent.

```
// SPW reactions.php:25-29 — an anonymous pseudonym, no personal data persis
↳ ted in cleartext
$voter_hash = hash('sha256',
    ($_SERVER['REMOTE_ADDR'] ?? 'unknown') .
    ($_SERVER['HTTP_USER_AGENT'] ?? 'unknown')
);
```

It's a good idea: what ends up in the database isn't an IP address in cleartext, but a string that means nothing to whoever reads it. It's better than what the festival vote does, which saves the IPs in cleartext (Ch. 18). But here an honest clarification is needed, because it's easy to tell it better than it is. This hash **isn't salted**, and the IPv4 address space is small: a little over four billion values, which a computer tries all of in a ridiculous amount of time. Combined with a plausible User-Agent, an unsalted SHA256(IP+UA) **inverts by brute force**. It doesn't protect the IP, it only obfuscates it.

WARNING

A hash isn't anonymity: it's pseudonymization, and it has limits “We hash it, so it's anonymous and compliant” is a sentence you hear often, and it's almost always false. A hash is a deterministic function: the same input always gives the same output, and if the input space is small (IPs are) anyone can build the reverse table for themselves. What you get is pseudonymization, useful for not having the IP in cleartext in the database, not irreversible anonymity. To make it genuinely hard to invert you need a **secret salt** kept on the server and never exposed: without that, the hash is a padlock with the key hanging right beside it. It's still a step up from saving the bare IP; the only problem is claiming it's more than it is.

A minor detail in the same direction: both the reactions and the messages read the raw REMOTE_ADDR, not the anti-spoofing helper getClientIp() used elsewhere on the site (Ch. 10). Behind a proxy or a CDN this can collapse many visitors onto a single address, and make the rate limit of the next section stricter than it should be.

✱
✱✱

3. The Two-Layer Rate Limit: Why One Isn't Enough

To keep a script from inflating the counts, the reactions have a rate limit. The first layer counts the recent actions by voter_hash: above twenty a minute, the request is rejected. It seems enough, but there's a crack, and it's the heart of this chapter: the voter_hash includes the User-Agent, which is a header **chosen by the client**. Just change it on every

request to get a different hash each time, and the first layer never sees two actions from the “same” voter.

Version 1.19.0 plugged the crack with a second barrier, anchored to a key the client doesn’t control: **the IP alone**.

```
// SPW reactions.php:92-119 – two layers, because the first can be bypassed
// Layer 1: by voter_hash (IP + UA), max 20/min – but the UA is chosen by th
↳ e client
$stmtRate = $pdo->prepare("SELECT COUNT(*) FROM article_reactions
    WHERE voter_hash = ? AND created_at >= DATE_SUB(NOW(), INTERVAL 1 MINUTE
↳ "); // >= 20 -> 429

// [v1.19.0] Layer 2: by IP ONLY, max 30/min – reuses login_attempts with th
↳ e 'rea:' namespace
$rl_key = 'rea:' . substr(hash('sha256', $ip), 0, 40); // truncated to fit
↳ the reused ip_address column
$stmtIpRate = $pdo->prepare("SELECT COUNT(*) FROM login_attempts
    WHERE ip_address = ? AND attempt_time >= DATE_SUB(NOW(), INTERVAL 1 MINU
↳ TE"); // >= 30 -> 429
```

The second layer reuses the `login_attempts` table, the same one that defends the login from brute force (Ch. 10) and the newsletter from mail-bombing (Ch. 13), distinguishing the uses with a namespace prefix (`rea:`). One table, three jobs.

TIP

If the rate-limit key includes client input, you need a second layer It’s an easy and common mistake: you pick a “strong” key for the rate limit (here IP plus User-Agent, more specific than the IP alone) without noticing that part of it is under the control of the very person you want to limit. The User-Agent is decided by the browser, and an attacker changes it on every request with a single line of code: the key becomes new each time and the limit never trips. The defense is to pair a second layer on a key the client can’t forge at will, like the IP. The two layers work together: the first is precise on the normal cases, the second holds when the first gets bypassed.

4. Integrity Lives in the Schema, Not in the Code

A reaction works like a toggle: if you haven’t given it yet, a click adds it; if you already have it, another click removes it. This logic lives in the code, but that isn’t where the anti-duplicate guarantee lives. That lives in the table’s schema.

```
-- SPW migrate_reactions.php:19-27 – the UNIQUE KEY is the real anti-duplica
↳ te barrier
CREATE TABLE IF NOT EXISTS article_reactions (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    article_id    INT NOT NULL,
    reaction      VARCHAR(20) NOT NULL,
    voter_hash    VARCHAR(64) NOT NULL,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY unique_vote (article_id, voter_hash, reaction) -- a double
↳ vote impossible by construction
);
```

```
// SPW reactions.php:129-145 – the toggle in the code; the INSERT IGNORE lea
↳ ns on the UNIQUE KEY
```



```

if ($existing) { /* DELETE: remove */ }
else { $pdo->prepare("INSERT IGNORE INTO article_reactions (article_id, reac
tion, voter_hash)
VALUES (?, ?, ?)")->execute([$article_id, $reaction, $voter_hash]); }

```

The difference is subtle but decisive. An application-level check of the kind “read whether it exists, then insert” can be overtaken by a race condition: two near-simultaneous requests both read “it doesn’t exist” and both insert. The `UNIQUE KEY`, instead, refuses the duplicate regardless of what the code does, even under a double click or racing requests. It’s the principle of the database as the guardian of integrity, and it’s exactly what the festival’s denormalized vote counter lacks (Ch. 18), where the absence of an equivalent constraint leaves room for the mismatch.

NOTE

Let the database guarantee integrity When a rule says “this combination can exist only once,” the right place to enforce it is a constraint in the schema, not an `if` in the application code. The constraint holds under concurrency, survives the bugs of the code running around it, and applies to every write path, present and future. The code can take care of the experience (the toggle, the message to the user); leave integrity to the engine that really knows how to defend it.

5. The Second Surface: Messages, and the Two Philosophies of Sanitization

The reactions are half the cluster. The other half is `messages.php`, the contact form. The table creates itself on the first call (the schema travels with the code, with a `CREATE TABLE IF NOT EXISTS` and a defensive `ALTER`), the `POST` is public with its own rate limit (three messages per IP every fifteen minutes, counted on its own table), and the notification to the administrator is a native fire-and-forget `mail()`: if the send fails, the visitor still sees “message sent,” because the record is already saved and the email is only a secondary channel (Ch. 13).

But the point that matters is how the text coming from outside is handled. It’s cleaned **before** touching the database.

```

// SPW messages.php:86-90 – public input cleaned at WRITE-TIME: what enters
the DB is already harmless
$name = trim(strip_tags($data['name'] ?? ''));
$email = trim(filter_var($data['email'] ?? '', FILTER_SANITIZE_EMAIL));
$subject = trim(strip_tags($data['subject'] ?? ''));
$message = trim(strip_tags($data['message'] ?? ''));

```

Here the thread of public input closes, and it does so with a polarity reversed from the articles. The content of an article (Ch. 8) is written by an admin you trust, is saved raw so as not to lose its rich formatting, and is cleaned with `DOMPurify` only **at the moment of showing it** (render-time). The text of a message comes from a stranger, and is cleaned with `strip_tags` **at the moment of saving it** (write-time): what ends up in the database is already free of tags, and stored XSS is neutralized at the source. There’s even a second

safety net: the admin panel shows the message as a React text node, which rewrites the special characters on its own, and never uses `dangerouslySetInnerHTML`.

IMPORTANT

Write-time or render-time: where to clean input depends on who sends it to you

In the same codebase two opposite strategies live side by side, and neither one is wrong. Public, untrusted input has to be neutralized as early as possible, at the write: the fewer dangerous things you keep in the database, the better you sleep. Rich, trusted content (a formatted article), on the other hand, has to be preserved as it is and cleaned where fidelity matters, at the read, because cleaning it at the write would destroy the legitimate formatting. The question isn't "write-time or render-time" in the abstract, but "who do I trust for this data": the answer decides the moment. Getting the direction wrong isn't elegant: saving public input raw opens stored XSS, cleaning editorial content at the write mutilates it.

A couple of side notes, for honesty. The public `POST` has no CSRF protection, and that's correct: there's no privileged action to forge, the only sensible barrier is the rate limit. And the form's double GDPR-consent checkbox is verified **only on the client side**: whoever submits directly to the endpoint skips it. It's consistent with the idea that consent is user experience and not a technical barrier, but it's worth knowing.

**

6. Reaction vs. Vote: Tuning the Anti-Abuse to What's at Stake

SimonePizziWebSite's reactions and DISINTELLIGENZA's festival vote (Ch. 18) are, technically, the same gesture: anonymous public writing defended by a hashed identity and an anti-duplicate barrier. But they serve two different purposes, and the rigidity is tuned accordingly. The reaction is free and plural: you can give more than one to the same article, remove them, put them back, and the anti-abuse is light because the stakes are low. The festival vote is single and watched: one expression per participant, an IP barrier on a twenty-four-hour window, a master switch that closes it off entirely, because there a ranking is being decided and the incentive to cheat is real.

NOTE

Same mechanics, two tunings: weigh the anti-abuse against what's at stake

It's pointless to police a "like" as if it were a ballot box, and dangerous to treat a vote that awards a prize with the lightness of a like. The same toolbox (derived identity, uniqueness constraint, rate limit) is tuned to two levels of rigidity depending on how much the abuse costs. Understanding where the stakes lie, before writing the anti-fraud code, is what avoids both needless friction and insufficient defense.

**

7. The Micro-Interaction: Optimistic, but with a Net

On the client the reaction bar updates the count and the button state **before** the server answers: the click feels instant. If the request fails, the update is rolled back and the UI returns to the real state. It's the optimistic UI of the social platforms, with the rollback that makes it honest. Together with the degradation to a zero count seen in §1 (if the reactions don't load, the article reads all the same), it's how a "side" module never becomes a breaking point for the page.

In Summary

Anonymous public writing is the most exposed point of the CMS, and SimonePizziWeb-Site defends it with care: a derived identity instead of the IP in cleartext, integrity enforced by the schema instead of by the code, a two-layer rate limit instead of one that can be bypassed, and input cleaned at the entrance instead of trusted. The lesson that ties it all together is the one about the two philosophies of sanitization: there's no absolutely right place to clean input, there's the question "who do I trust," and the answer changes the direction of the defense. What's left to remember is also what the module *isn't*: the hash makes no one anonymous, it obfuscates them; the form's consent is a courtesy, not a barrier. Knowing that, and not telling yourself the padlock is sturdier than it is, is part of the same technical honesty that runs through the whole book. Here the tour inside the CMS ends: from the foundations of the backend to the last click of a visitor we'll never know the identity of.

IMPORTANT The Canon

- For unauthenticated public writing, derive a pseudonym with a **salted** hash: an IP+UA hash without salt isn't anonymity, it's reversible.
- If the rate-limit key includes client input (the User-Agent), you need a second layer anchored to the IP; leave integrity to the schema (UNIQUE + INSERT IGNORE).
- Sanitize public input **at write-time** (`strip_tags`) and trusted content **at the render** (DOMPurify): it depends on who sends it to you.
- Tune the anti-abuse to the stakes: a light reaction isn't defended like a watched vote.

End of Part V. The appendices gather the service materials: the checklist for starting from scratch, and the case of the fork (FDCA), the project that inherits an entire CMS, security debts included.

Appendices

Practical tools and edge cases.

APPENDIX A – Boilerplate Checklist:

Starting a New miniCMS Project

This checklist summarizes the practical steps to bootstrap a new web project (Site or Web App) based on the standards of the “Universal miniCMS Model.” For implementation details, refer to the chapters indicated. The (Ch. N) cross-references follow the numbering of the Third Edition (20 chapters + Appendices A and B).

*
**

Phase 1: Environment Setup and Initial Security

- ☐ Create the base folder structure (`public/api/, src/, scripts/`). (Ch. 2)
- ☐ Create the `public/.htaccess` file for SPA routing (React Router). (Ch. 2)
- ☐ Scaffold the database: a `public/api/.data/` folder with an `.htaccess` → Deny from all. (Ch. 2, 3)
- ☐ Create the media folder (`public/api/uploads/`) and the cache (`public/api/.cache/`).
- ☐ Configure `vite.config.ts` with the correct proxy to avoid CORS locally. (Ch. 6)
- ☐ Create `.env.local` with the development variables (`VITE_API_URL=http://localhost/...`).
- ☐ Add `db_credentials.php` and `.env.local` to `.gitignore` (never commit credentials). (Ch. 15)

Phase 2: Core Backend Configuration (PHP)

- ☐ Implement `db.php` with a lazy connection to SQLite (`PRAGMA journal_mode=DELETE, busy_timeout=5000, PRAGMA foreign_keys=ON`). Do **not** use WAL on shared hosting. (Ch. 3, 15)
- ☐ Include the auto-scaffolding in `db.php`: creation of the `.data/` folder and the `deny.htaccess` if they don't exist. (Ch. 3)
- ☐ Create `init_db.php` to generate the tables and the default admin user with `password_hash()` (and change its password right away). (Ch. 3, 10)
- ☐ Create `auth_helper.php` or `auth.php` with the `Auth::check()` class that handles `session_start()` and the JSON headers. (Ch. 5)
- ☐ Configure secure session cookies (`httponly, samesite=Strict, secure` if HTTPS). (Ch. 10)
- ☐ Apply `date_default_timezone_set('Europe/Rome')` in **every** endpoint with time logic (a timezone inconsistency makes scheduled content disappear). (Ch. 5, 12, 15)
- ☐ Hide errors in production (`display_errors = 0` or a global try-catch): a `die($e->getMessage())` on a connection failure is a leak. (Ch. 10)

Phase 3: Core Frontend Configuration (React)

- ☐ Implement `src/api.ts` with the reading of the payload's shape (the “Double Read” pattern = read the success envelope, don't clone the response) to intercept server errors. (Ch. 6)
- ☐ Configure `AdminLayout.tsx` with the auth guard (route-guard loader or check on mount) and the “Hard Logout” (`window.location.reload()`). (Ch. 14)
- ☐ Enable the “Role-Based UI” to show only the items allowed to the connected role (Admin vs. Editor): hiding the page is UX, **blocking the action is security and has to be done on the server**. (Ch. 10, 14)
- ☐ Use `key={item.id}` in the editor component to force the rich text editor to reset when the article changes. (Ch. 8)

Phase 4: Media and Content Integration

- ☐ Implement `upload.php` with automatic GD resizing and name sanitization (`uniqid()`, the extension rebuilt, not the client's). (Ch. 7)
- ☐ Add the `MediaPicker` component for the direct upload of audio and images. (Ch. 8)
- ☐ Use the editor (Tiptap) with HTML as the source of truth saved raw; the protection against XSS is sanitization **at the render** (DOMPurify), not paste-time cleaning, which is only cosmetic. (Ch. 8)
- ☐ Implement the slug logic with normalization of Italian accents if the content is in Italian. (Ch. 5)

Phase 5: SEO and Syndication

- ☐ Create `public/index.php` as the SEO Engine: a DB query → injection of the meta tags into Vite's HTML (Dynamic Rendering with UA-sniff). (Ch. 11)
- ☐ Add the rename of `index.html` → `index_react.html` in the build script if `index.php` lives in `public/`. (Ch. 2, 11)
- ☐ Create `api/rss.php` with a valid RSS 2.0 feed (correct header, RFC 822 dates, absolute URLs, escaped content). (Ch. 12)
- ☐ Add the `<link rel="alternate" type="application/rss+xml">` tag in the HTML `<head>`. (Ch. 12)

Phase 6: Optimization and Deploy

- ☐ Configure “Real Scheduling” via a SQL query on `published_at` (comparison **in the same format/timzone**, or delegated to `NOW()`). (Ch. 9)
- ☐ Configure the JSON cache with a 300s TTL for heavy listing queries. (Ch. 9)
- ☐ Set up `clean-dist.js` in the build process to remove the `.sqlite` files from the `dist/` folder (and recreate at runtime the static defenses the build strips). (Ch. 2, 14)
- ☐ Configure the `Cache-Control: max-age=31536000` header for the files in `uploads/` via `.htaccess`. (Ch. 7)

Phase 7: Security (the Nets That Emerged from the Real Cases)

The items in this phase are the defenses that, missing in at least one of the mapped sites, produced the flaws told in the book.

- ☐ **Upload with PHP-off:** in the upload folder, an `.htaccess` that disables PHP execution (the first anti-RCE barrier), plus validation by magic bytes (`fileinfo`), not by the MIME the client declares. (Ch. 7)
- ☐ **Three-rung CSRF:** a token generated server-side, sent to the client and validated on every state-changing request (POST/PUT/DELETE/admin actions). A `confirm()` is not a CSRF defense. (Ch. 10, 14)
- ☐ **Newsletter double opt-in:** two distinct tokens (confirmation and unsubscribe); the unsubscribe link needs a secret, the email in cleartext isn't enough (it isn't GDPR-compliant). (Ch. 13)
- ☐ **Automatic backup outside the document root,** with rotation and an unguessable name; a cron protected by a timing-safe secret and fail-closed. Having the emergency script doesn't replace the backup. (Ch. 14, 15)
- ☐ **Shared server-side sanitization:** the four `content` emitters (render, SEO prerender, RSS feed, newsletter) must pass through the same cleaning; just one that forgets reopens the XSS hole. (Ch. 8, 11, 12, 13)
- ☐ **Gate by role, not just by login:** the admin endpoints verify `isAdmin`, not just `isLoggedIn`; powerful actions aren't GETs without a token. (Ch. 10, 14)

Phase 8: Specific to Site Type

For Sites with a Newsletter (SitoRuntime pattern)

- ☐ Create the `subscribers` table with the **complete** double opt-in schema (`email` UNIQUE, `is_active`, `confirmation_token`, `confirmed_at`, `subscribed_at`, `subscribed_ip`, `created_at`): one table, a single CREATE. (Ch. 13, 15)
- ☐ Implement `newsletter.php` with an admin gate + public actions (subscribe, confirm, unsubscribe). (Ch. 13)
- ☐ Use the `{EMAIL_PLACEHOLDER}` pattern for the personalized unsubscribe link with a token. (Ch. 13)
- ☐ Distinguish the **throttle** from the **rate limit**: `usleep(5000000)` every 10 emails paces the send (throttle), but does **not** protect against mail-bombing; for that you need a real rate limit on the subscribe action. (Ch. 13)

For Portfolio/Personal Site (SimonePizziWebSite pattern)

- ☐ Add the `projects` table with `sort_order`, `is_visible`, `button_a`, `button_b`. (Ch. 16)
- ☐ Implement `projects.php` with the 5 HTTP methods, including PATCH for the visibility toggle and reordering. (Ch. 16)
- ☐ Create the `PortfolioGrid.tsx`, `ProjectEditor.tsx`, and `ProjectsList.tsx` components. (Ch. 16)

For Festival/Contest (DISINTELLIGENZA / FDCA pattern)

- ☐ Add the `participants`, `votes`, and `settings` tables with the `registration_active` and `voting_active` master switches. (*Ch. 17, 18*)
- ☐ Implement `participants.php` with the pending → approved/rejected workflow. (*Ch. 17*)
- ☐ Implement `votes.php` with the real anti-fraud: the barrier is the **IP + 24h window** constraint (the cookie is only cosmetic). (*Ch. 18*)
- ☐ If the festival is born as a **fork**, secure the backend from scratch: the fork inherits every flaw, and the fix doesn't follow it. (*Appendix B*)

For SQLite → MySQL Migration (SitoRuntime pattern)

- ☐ Back up the `.sqlite` **before** touching the engine, and archive it outside the repo. (*Ch. 15*)
- ☐ Create a separate `db_credentials.php` (add it to `.gitignore`). (*Ch. 15*)
- ☐ Update `db.php` with the MySQL PDO connection (`utf8mb4`, `EMULATE_PREPARES=false`, `ATTR_TIMEOUT`). (*Ch. 15*)
- ☐ Run `init_mysql.php` to create the MySQL schema on the server, then remove it. (*Ch. 15*)
- ☐ Run `migrate_to_mysql.php` for the data move (ONE-SHOT with a count check, delete afterward). (*Ch. 15*)
- ☐ Once the migration is done, **remove the old engine's fossils** from the repo (`init_db.php` SQLite, `fix_*`, `optimize_db.php`, `emergency_revert_wal.php`). (*Ch. 15*)

This checklist accompanies the Third Edition of the Universal miniCMS Model. For implementation details, refer to the .md files of the corresponding chapters and appendices.

APPENDIX B: The Life of a Fork

The Model describes how a thin-stack site is built. It doesn't say what happens when you **fork** one, and yet that's a real event in the life of these projects, and we have one example mapped from the inside: FDCA, the "Festival della Canzone Artificiale" (Festival of the Artificial Song), is born as a fork of DISINTELLIGENZA, the "Festival della Disintelligenza Naturale" (Festival of Natural Disintelligence). Same author, same engine, new theme: from the comedy of human error to generative music. This appendix tells that phase, because it has risks all its own, and the first is the most silent of all.

Chapter 2 already introduced the pattern: two festivals with the same functional base start from a copy of what already works. The advantage is independence; the risk is its exact flip side. Here we look at it up close.

*
**

1. The Backend Freezes, the Frontend Restarts

The first thing you notice about FDCA is where the fork's energy went. The `public/api/` folder is copied from DISINTELLIGENZA **byte for byte**: the same 28 files, the same logic for authentication, voting, upload, newsletter, festival, zero differences in content. All the new work is on the public *shell*: rewritten showcase pages (Home, Philosophy, Manifesto, Past Editions, Contacts), a different branding, an animated cursor. It's the natural pattern of a re-skin: you change what you see, not what works.

The consequence is that the server logic is inherited intact. And "intact" includes everything, the flaws too.

*
**

2. The Fork Inherits the Debt: Copying an Insecure Backend Multiplies It

This is the central point of the appendix. None of DISINTELLIGENZA's security vulnerabilities was fixed in the fork, because the backend is copied line by line. What was a problem on one site becomes the same identical problem on two.

Inherited vulnerability	Where it's described	Status in FDCA
Public-upload RCE chain (no-auth upload, MIME from the client, name preserved, no PHP-off)	Ch. 7	present, unchanged
Base-rung auth (no CSRF, no rate limit, no fix-ation protection)	Ch. 10	present
Newsletter with no double opt-in or token + header injection	Ch. 13	present
Destructive resets with no CSRF token	Ch. 10, Ch. 14	present
Denormalized <code>vote_count</code> (ranking drift)	Ch. 18	present
Render without DOMPurify (stored XSS)	Ch. 8	present

There's a detail that makes the story even more instructive. On DISINTELLIGENZA's repository there's an open intervention to close the upload's RCE chain (Ch. 7). That fix lives on DISINTELLIGENZA, and **doesn't touch FDCA**: the fork has its own copy of the vulnerable file, disconnected from the original. The day FDCA's backend goes online, the RCE will be replicated, identical, on a second domain, while everyone believes they've solved it.

WARNING

The fix doesn't follow the fork When you fork by copying the backend, you inherit the features and you inherit the flaws, in the same gesture. From that moment the two branches are independent: every security fix made on one **doesn't** reach the other by magic, it has to be reapplied by hand. It's easy to forget, because the fork looks like "another project," with another name and another version. But underneath it's the same code, and a flaw closed on one branch stays wide open on the other until someone closes it again, on purpose. Before forking a backend, it's worth having a list of its known vulnerabilities precisely so as not to lose sight of it after the copy.

*
**

3. The Disconnected Shell

There's a second surprise in FDCA: the new frontend **doesn't talk to the inherited backend**. There's no `src/api.ts`, not a single `fetch` call to `/api/`. The showcase pages are pure marketing, not connected to the CMS running behind them. FDCA, as it stands, is a showcase site with a full, working CMS behind it that's unreachable from its own application.

It isn't a mistake: it's a phase. The skin first, then (maybe) the rewiring. It's the disconnection typical of a restyle in progress, and on its own it does no harm, because a backend no one calls exposes nothing. The risk is latent and deferred: the day the frontend gets connected, it will inherit *operationally too* the behaviors, and the bugs, of

DISINTELLIGENZA's backend. Today's disconnection is also the reason the flaws in §2 aren't yet exploitable end-to-end, and the reason it's easy to forget about them.

✱
✱✱

4. The Version That Restarts from Zero on Code That Doesn't

FDCA presents itself as **v0.0.1**. The `package.json` says 0.0.1, the `metadata.json` talks about a "Remix," the README is the one generated by Google AI Studio. Everything signals a new product, just born. Underneath, though, runs a backend at v0.5.x with all its accumulated debt.

WARNING

The version tells of a birth, the code tells of an inheritance The version discontinuity *hides* the continuity of the debt. It's the reverse of the divergent version-strings problem you see in DISINTELLIGENZA (the sidebar, init, and package show three different numbers): there too many versions for a single codebase, here a version-zero for code that's anything but zero. In both cases the version number lies about the real state of the software. When you see a 0.0.1, don't take for granted that there's little underneath: look at the code, not the label.

✱
✱✱

5. The Fork That Writes Its Own Roadmap

FDCA includes a `ROADMAP-EVOLUZIONE-miniCMS` folder with nine chapters, AI-assisted, that plan to rebuild the miniCMS cleanly. The titles of those chapters **trace the same clusters** as this very mapping: Backend, PHP API, Frontend Bridge, Admin & Protected Routes, Festival Engine. It's a precious artifact, because it confirms from the inside the structure this manual reconstructs from the outside: the backbone of the thin stack is recognizable even to those who work inside it, to the point of wanting to rewrite it in order. A project that documents the intention to evolve its own engine is the proof that the pattern exists and is legible.

✱
✱✱

6. One Engine, Two Festivals

There's a bright side to forking too, and FDCA demonstrates it. The festival model (sign-up, selection, round-based voting, registration and voting master switches, a ranking derived from `vote_count`) is **the same** on the two sites: only the theme changes, from human error to generative music. It's the positive proof that the contest module described in Chapters 17 and 18 is a genuinely reusable component: the same engine carries two different festivals with a simple re-skin.

Forking, in short, isn't a bad thing in itself. Reusing a domain that works by changing the skin is efficient and legitimate. It becomes a problem only when the re-skin forgets

the engine: when you copy the backend and overlook that inside that copy there were the flaws too, and that from then on it's the fork's responsibility to keep them closed.

✱
✱ ✱

In Summary

Forking a thin-stack project is a real and common phase, and it has a physiology of its own: the backend freezes, the frontend restarts, the version resets, the theme changes. The specific risk is just one, but it's serious: the silent inheritance of the debt. A fork that copies the backend verbatim carries every vulnerability along with it, unchanged, while the version-zero and the new name tell of a clean product. The fix doesn't follow the fork: it has to be reapplied by hand, on a branch that now lives a life of its own. The rule, if you fork, is to keep two lists together: the one of the features to reuse and the one of the flaws not to inherit. The first is the reason you fork; the second is the reason a fork has to be treated as a project to secure from scratch, not as a copy already in order.

APPENDIX C: Testing & Deployment (an Outline)

Two topics the body of the manual touches only in passing deserve at least a starting point: how you test a thin stack without a framework handing you the tools, and how you bring it into production on cheap hosting. These are notes, not a complete guide: they point the direction and the main traps, not every step. But they serve to dispel a misconception, namely that “no framework” means “no tests” and “deploy by feel.”

✱
✱ ✱

1. Testing: No Framework Doesn’t Mean No Tests

The model gives up frameworks, not verification. If anything, its minimal form makes some tests easier, not harder.

The PHP backend. Pure logic (slug, validation, sanitization, visibility calculation) is tested with PHPUnit like any code. The endpoints, instead, are better tested *functionally*: a real HTTP request against the endpoint, and a check on the status and the shape of the payload. Here the file-based database becomes an unexpected advantage. Instead of mocking PDO (laborious and not very faithful), you open a throwaway SQLite, in memory or in a temporary file, populate it with known data, and throw it away at the end of the test.

```
// An endpoint is better tested with a real ephemeral DB than with a PDO moc
↳ k
$pdo = new PDO('sqlite::memory:');          // a test database, lives as long
↳ as the test
$pdo->exec(file_get_contents('schema_test.sql')); // a known schema
// then: call the endpoint (via HTTP or by including the file with simulated
↳ $_GET/$_POST)
// and verify the status code + the JSON structure of the response
```

The frontend. Vitest and Testing Library cover the React components. The strong point is the `api` object of Chapter 6: being a single channel over `fetch`, it’s mocked in just one place, and the component tests never touch the real network.

The tests that pay off most. In a system like this, two families of test are worth more than a thousand detail assertions:

- **Contract smoke test.** A test that calls every public endpoint and verifies the status plus the shape of the payload catches almost all the regressions of the “unstable contract” discussed in Chapter 6. It’s the safety net missing exactly where the API has no formal schema.

- **Security non-regression test.** An admin endpoint must answer 401 or 403 without a session; a `.php` file uploaded into the upload folder must not be executable (Chapter 7); a mutation without a CSRF token must fail (Chapter 10). These are the checks that keep an already-closed flaw from silently reopening.

*
**

2. Deploy: From the `dist/` to Five-Euro Hosting

The model is born to run on cheap shared hosting, and the deploy is deliberately simple: no containers, no orchestrators. But “simple” doesn’t mean “improvised.”

What the build produces. `npm run build` generates the `dist/` folder with React’s static assets. Two steps of the build process, already seen in Chapters 2 and 11, are defenses, not details: `clean-dist.js` removes the development `.sqlite` files from the distribution (a development database shipped to production is a disaster), and if `index.php` lives in `public/`, the `index.html` has to be renamed `index_react.html`, because the SEO Engine’s PHP entry point has to take precedence.

What you upload, and what you don’t. The compiled `dist/` and the `public/api/` folder with the PHP go on the server. The `db_credentials.php` file (for MySQL sites) is uploaded **by hand**, once, and never sits in the repo. You never upload the development `.data/` folder or the test files.

How you upload. The three ways, in order of robustness: a `git pull` on the server if the hosting allows it (the cleanest); a scripted SFTP deploy; manual FTP upload (the most fragile, because forgetting a file is easy). Whatever the way, it’s best for the deploy to be *repeatable*: a script in `scripts/` that always does the same steps is worth more than a procedure kept in memory.

A minimal CI. Even without complex pipelines, a single GitHub Action that on every push runs the build, the lint, and the tests from the previous section catches regressions before they reach production. The secrets (credentials, keys) live in the CI’s environment variables, never in the repository. A possible automatic deploy step via FTP/SFTP is a handy addition, but it comes later: first the safety net of tests, then the automation of delivery.

*
**

3. The Limit of These Notes

The above is a map, not the territory. A site with serious availability requirements will want separate environments (staging and production), versioned schema migrations (the debt of Chapter 15), error monitoring, and verified backups, not just executed ones. The thin stack forbids none of this: it simply doesn’t hand it to you, and it’s up to you to decide how much of that discipline your project deserves. The rule is the same as the rest of the book: start from the minimum that keeps you safe, and add only when a concrete need calls for it.

IMPORTANT

The Canon

- Test the endpoints with an ephemeral SQLite (`:memory:` or a temporary file), not by mocking PDO.
- Mock the `api` object in just one place for the React component tests.
- Keep two safety nets that pay off: a contract smoke test (status + shape of the payload) and a security non-regression test (gate, upload, CSRF).
- Build with `clean-dist` (away with the `.sqlite` files); upload `dist/` + `public/api/`, never `db_credentials.php` from the repo or the development `.data/`.
- A minimal CI (build + lint + test on every push), with the secrets in the environment variables, not in the code.

The Author

Simone Pizzi is an author and publisher. He founded Runtime Edizioni and Runtime Radio, for which he curates editorial, audio, and digital projects. He has published the short-story collection *L'Albero dei Racconti* (The Tree of Tales) and the science-fiction novella *Frequenza di Servizio* (Service Frequency).

This manual grows out of real work on four production sites — SitoRuntime, DISINTELLIGENZA, FDCA, and SimonePizziWebSite — and gathers the “thin stack” protocol he built them with: React and TypeScript for the presentation, native PHP and SQLite or MySQL for the data, with no backend framework and no oversized infrastructure.

